Database-as-a-Service Platform

A cloud-based platform that unifies the management of SQL and NoSQL databases,
providing AI-powered assistance for developers and non-technical users.

Team Members:
Hassan Hussein Azmy
Ali Mohammed Ali
Abdallah Hany Ragab
Ahmed Bahy Youssef
Aya Mohammed Sayed

Supervisor: Dr. Rania Ramadan Abdel-Dayem
Eng. Mustafa Shokry
Department of Computer Systems Engineering
Faculty of Engineering, Benha University
Graduation Project 2024-2025
June 2026

Acknowledgments

Acknowledgments We would like to express our sincere gratitude to our esteemed professors and supervisors for their invaluable guidance, support, and encouragement throughout the course of our graduation project.

Our heartfelt thanks go to **Dr. Rania**, our supervisor, for her exceptional mentorship, insightful feedback, and unwavering support, which have been pivotal to the success of our project. We are deeply appreciative of her dedication and expertise, which have greatly enriched our learning experience.

To all our professors who have contributed to our academic journey and helped shape our growth as aspiring professionals, we extend our deepest appreciation. Your efforts and guidance have been instrumental in making this project a reality, and we are truly thankful for your contributions.

Abstract

Modern software development heavily relies on data, yet managing databases remains complex due to the coexistence of SQL and NoSQL systems, fragmented tools, and steep learning curves. Existing cloud services simplify hosting but lack an integrated, intelligent environment for developers. This project proposes a unified, AI-powered Database-as-a-Service (DBaaS) platform that supports both SQL (PostgreSQL) and NoSQL (MongoDB) databases. Key features include visual schema design, integrated data manipulation, query execution, AI-assisted guidance, and flexible cloud deployment. The platform aims to streamline workflows, reduce development overhead, and improve accessibility for both novice and experienced developers.

Contents

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Motivation

Databases are essential to modern software systems, supporting data storage, retrieval, analysis, and application logic. The increasing reliance on data-driven decision-making has elevated the need for efficient, scalable, and accessible database management systems, including relational databases (SQL) like PostgreSQL and MySQL, and non-relational databases (NoSQL) such as MongoDB and Cassandra.

Database development, however, is often complex due to fragmented workflows involving multiple tools for schema design, data manipulation, query execution, and cloud hosting. While cloud-based services like AWS RDS, Supabase, Firebase, and MongoDB Atlas simplify infrastructure management, they do not provide fully integrated development environments, leaving inefficiencies and steep learning curves for developers.

Advances in artificial intelligence, including Retrieval-Augmented Generation (RAG), offer opportunities for automated schema generation, optimized queries, and natural language interactions, yet AI integration in existing tools remains limited.

This project addresses these challenges by developing a unified, cloud-based Database-as-a-Service (DBaaS) platform supporting PostgreSQL and MongoDB. It combines visual schema design, seamless data manipulation, cloud storage, and AI-powered assistance, providing an intuitive, efficient, and accessible environment for both novice and experienced developers.

1.2 Problem Statement

Modern software development relies heavily on data, yet the diversity of database technologies, particularly the coexistence of SQL and NoSQL systems, imposes significant challenges on developers. Managing multiple tools and interfaces for schema design, query execution, data manipulation, and visualization creates fragmented workflows, steep learning curves, and increased error rates.

While AI can automate repetitive tasks, assist with query formulation, and accelerate onboarding, most existing database tools lack meaningful AI integration. Similarly, cloud-based offerings focus on hosting rather than providing a unified development environment.

This underscores the need for a cloud-based, AI-powered Database-as-a-Service (DBaaS) platform that integrates SQL and NoSQL management, intuitive schema design, and intelligent query support, streamlining workflows and improving accessibility for both novice and experienced developers.

1.3 Suggested Solution

To address the challenges of fragmented database management and the complexity of handling multiple database types, the proposed solution is the development of an AI-powered, cloud-based Database-as-a-Service (DBaaS) platform. This platform will unify the management of SQL and NoSQL databases within a single, intuitive interface, enabling developers and non-technical users to work more efficiently and confidently.

Key features of the solution include:

- **Unified Database Management:** The platform will support both SQL (PostgreSQL) and NoSQL (MongoDB) databases, allowing users to manage multiple types of databases without switching between different tools.
- **Schema Design and Visualization:** Users will be able to create and visualize database schemas through a simple, interactive interface. The platform will support manual schema creation, CSV/JSON uploads, and semi-automated schema generation using natural language input.
- **Integrated Data Manipulation:** A Query Editor will allow execution of both SQL and NoSQL queries, while a table-like interface will enable easy addition, updating, and deletion of data.
- **Flexible Cloud Provider Integration:** Unlike traditional platforms that lock users into one cloud provider, our solution will allow users to deploy on their preferred cloud This ensures portability, reduces vendor lock-in, and gives users freedom to choose based on cost, region, or performance needs.

By combining AI-assisted automation, cloud infrastructure, and a unified interface, this DBaaS platform aims to streamline workflows, reduce development overhead, and empower users to interact with databases more efficiently and intelligently.

1.4 Project scope

This project focuses on the design and development of a cloud-based Database-as-a-Service (DBaaS) platform that unifies SQL and NoSQL database management within a single, user-friendly system. The platform aims to simplify database operations and enhance productivity by integrating intelligent assistance and modern cloud technologies.

The system supports PostgreSQL as a relational database and MongoDB as a NoSQL database, allowing users to manage different data models through one interface. It provides tools for

schema design and visualization, including manual creation, file-based imports (CSV and JSON), and schema generation using natural language descriptions.

To improve usability, the platform incorporates AI-assisted features based on Retrieval-Augmented Generation (RAG), enabling natural language querying, automated query generation, and schema explanation. A unified query editor and a table-based data management interface are included to facilitate efficient data manipulation and query execution.

The project also covers secure cloud storage with version control, allowing users to manage, restore, and export database projects. To ensure flexibility and avoid vendor lock-in, the platform is designed to support multi-cloud deployment, enabling users to choose their preferred cloud provider through containerized and cloud-agnostic architecture.

Finally, the scope includes basic administrative monitoring and essential security mechanisms such as authentication, authorization, and controlled access to database resources.

1.5 Project aim & objectives

The aim of this project is to design and develop a cloud-based Database-as-a-Service (DBaaS) platform that unifies the management of SQL and NoSQL databases within a single, intelligent, and user-friendly environment. The project seeks to simplify database operations by providing integrated tools for schema design, data manipulation, and query execution, while reducing the reliance on multiple fragmented database management tools. To achieve this, the platform supports PostgreSQL and MongoDB, offers visual and file-based schema creation, and incorporates AI-assisted features based on LLM AI agents to enable natural language querying, automated query generation, and schema understanding. Additionally, the system is designed to support secure cloud storage with version control, administrative monitoring, and robust access control mechanisms. A key objective of the project is to ensure multi-cloud deployment flexibility, allowing users to select their preferred cloud provider and avoid vendor lock-in through a cloud-agnostic, containerized architecture.

1.6 Project Methodology

A software development approach is a structured way of overseeing the creation of a software project. It typically covers aspects such as determining which features to include in the current version, setting release timelines, assigning tasks to team members, and defining the testing process.

1.6.1 Software Development Method

The software development model defines the structured approach used to build the system. Choosing the right model depends on factors such as project size, complexity, and flexibility.

Our project is of medium size with rapidly evolving requirements, requiring a development approach that is adaptable to change. To achieve this, we follow a process that emphasizes collaboration and interaction among team members. The development is structured around short-term plans, which are prepared jointly by the project supervisor and the team. Based on these factors and the comparison shown in Table 1.1, agile software development methodologies were selected.

1.6.2 Agile Software Development Methodologies

Agile development methodologies are iterative project management approaches that prioritize collaboration, flexibility, and feedback. They break projects into smaller parts, called sprints, with regular reviews and adjustments to allow developers to respond to evolving requirements.

Agile development methodologies are important because they:

- Enable faster delivery and quick adaptation to change.
- Enhance collaboration among team members and stakeholders.
- Increase product relevance through user-centric development.
- Reduce cost overruns and improve time to market.

A comparison between the most commonly used agile methodologies is shown in the table below.

Criteria	**Scrum**	**Kanban**	**Extreme Programming (XP)**
Focus	Time-boxed iterations (Sprints)	Continuous flow of work	Engineering practices for quality and responsiveness
Key Roles	Product Owner, Scrum Master, Development Team	No defined roles	Developer, Tester, Customer

Continued on next page

Criteria	Scrum	Kanban	Extreme Programming (XP)
Iteration Length	Fixed (2–4 weeks)	Continuous delivery	Short (1–2 weeks)
Work Breakdown	Prioritized backlog	Visualized tasks (Kanban board)	Small features with continuous feedback
Planning	Sprint planning	Pull-based planning	Iterative feature planning
Customer Involvement	High	Moderate	Continuous
Flexibility to Change	Limited during sprint	Highly flexible	Highly flexible
Team Collaboration	Cross-functional teams	Specialized resources	Self-organizing teams

Table 1.1: Comparison among Scrum, XP, and Kanban agile methodologies

Our project involves a small, cross-functional, and self-organized team with clearly defined roles and fixed deliverables. Based on these characteristics and the comparison above, the Scrum methodology was selected.

1.6.3 Scrum Methodology

Scrum is an Agile framework designed to support effective collaboration in complex software development projects. It emphasizes iterative progress, accountability, and continuous improvement. Work is divided into time-boxed iterations called sprints, typically lasting two to four weeks. Each sprint begins with a planning meeting and ends with a review and retrospective.

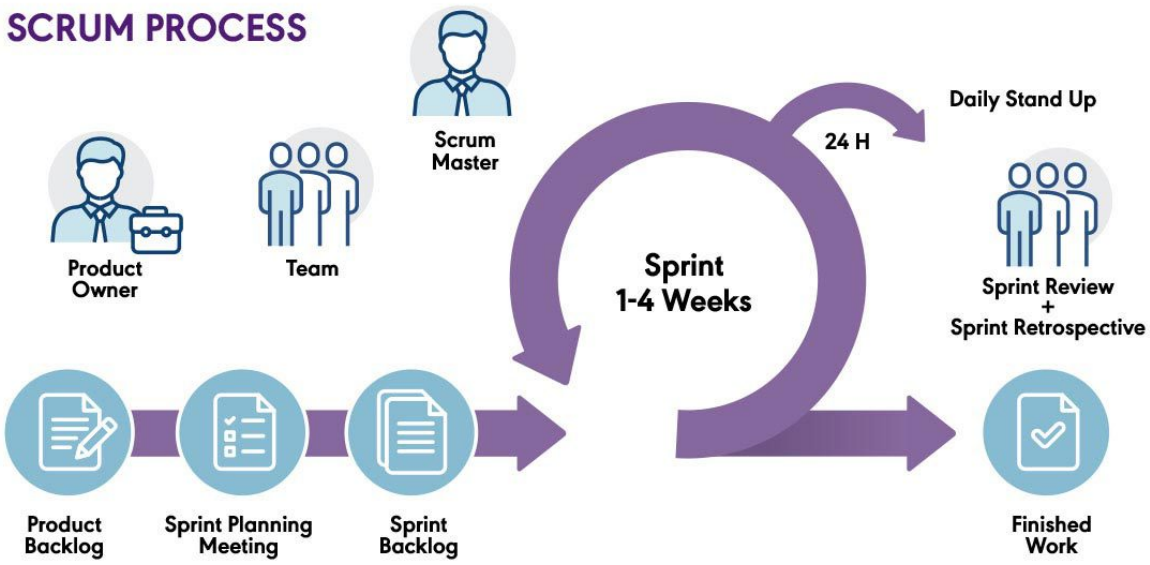


Figure 1.1: Scrum Methodology Process

The roles in Scrum are defined as follows:

- **Product Owner (PO):** Responsible for representing stakeholders, defining requirements, and prioritizing the product backlog.
- **Scrum Master:** Ensures adherence to Scrum principles, removes impediments, and facilitates communication within the team.
- **Scrum Team:** A group of professionals responsible for delivering the product increment during each sprint.

The Primary Phases of the Scrum Model

1. **Initiation:** Defining the project vision, identifying stakeholders, and forming the Scrum team.
2. **Planning and Estimation:** Selecting backlog items and planning sprint activities.
3. **Implementation:** Executing sprint tasks and updating progress continuously.
4. **Reviewing:** Evaluating sprint outcomes and identifying improvements.
5. **Releasing:** Delivering the product increment and conducting retrospectives.

1.6.4 Justification for Technology Choices

- **Frontend – Flutter:** Flutter enables a single codebase for web and mobile applications, ensuring faster development, consistent design, and seamless integration with backend services and AI features.

- **Backend – Golang / Spring Boot:** These frameworks provide high performance, scalability, and reliability, making them suitable for handling concurrent requests and multiple database systems.
- **Databases – PostgreSQL and MongoDB:** PostgreSQL manages structured relational data, while MongoDB supports flexible and unstructured data, offering versatility across use cases.
- **LLM AI Agent – AutoGen:** This tool supports natural language interaction, automated schema generation reducing the learning curve for users.
- **Cloud Deployment – AWS / GCP:** Cloud platforms ensure scalability, reliability, secure storage, and flexible deployment while avoiding vendor lock-in.

By integrating these technologies, the proposed platform achieves a robust, scalable, and user-friendly environment that enhances productivity and simplifies database management for both novice and experienced developers.

1.7 Project Plan

Phase / Task	Duration	Start Date	End Date
Brainstorming and Idea Generation	1 month	August	August
Searching for project ideas	2 weeks	August	August
Initial discussions among team	2 weeks	August	August
Idea Selection and Supervisor Discussion	1 month	September	September
Define project idea and novelty	2 weeks	September	September
Final agreement with supervisor	2 weeks	September	September
Initial System Analysis	10 days	01 Oct	10 Oct
Problem definition	3 days	01 Oct	03 Oct
System analysis diagrams	7 days	04 Oct	10 Oct
Studying Related Topics	3 weeks	10 Oct	01 Nov
Studying required technologies	3 weeks	Oct	Nov

Phase / Task	Duration	Start Date	End Date
Implementation Phase 1	6 weeks	15 Nov	31 Dec
UI implementation	3 weeks	Nov	Dec
Backend and database development	3 weeks	Nov	Dec
Documentation Work	2 weeks	24 Jan	07 Feb
Writing Chapter 1 and Chapter 2	2 weeks	Jan	Feb
Implementation Phase 2	5 weeks	07 Feb	13 Mar
System integration and AI connection	5 weeks	Feb	Mar
Testing and Debugging	2 weeks	15 Mar	28 Mar
Unit and integration testing	2 weeks	Mar	Mar
Final Documentation and Submission	4 weeks	Apr	May
Final GP book writing	2 weeks	Apr	Apr
GP book submission	2 weeks	May	May
Final Discussion	3 days	June	June

Table 1.2: Project Plan Overview

Chapter 2: Related Work

2.1 Background

2.1.1 Learning About DBaaS Systems

Database-as-a-Service (DBaaS) represents a novel paradigm inspired by cloud computing principles [2]. It is defined as a cloud service responsible for managing the underlying infrastructure and resources required by cloud databases [3]. DBaaS enables organizations, whether large enterprises or small businesses, to rely on flexible and reliable data storage services. The model shifts the burden of managing the Database Management System (DBMS) and data storage away from a client-server architecture—where the data owner controls these elements—to a third-party cloud architecture [3].

In the DBaaS environment, the provider assumes responsibility for crucial administrative and maintenance tasks, including maintaining the database software, resource management, backup/restore, high availability, and point-in-time recovery of the service [4]. This structure obviates the need for customers to purchase costly hardware and software, handle upgrades, or employ professionals for administrative roles [2].

2.1.2 Understanding DBaaS Concepts

A foundational concept in DBaaS is multi-tenancy, which describes the design principle within the cloud for sharing computing resources among multiple, concurrent tenants. DBaaS providers implement various architectures to enable this dynamic resource sharing, aiming to reduce costs and enhance flexibility [2].

The architectural landscape of DBaaS is primarily categorized based on the degree of multi-tenancy—the layer at which resources are shared:

a) **Separate database (Virtual Multi-Tenancy)**
This is the simplest strategy, achieving a high level of isolation, typically at the virtualization layer. Each tenant receives a database in an isolated virtual machine (VM), as exemplified by Amazon AWS. This approach abstracts the installation and configuration of the database system from the user. However, requiring a separate operating system for each deployed database can lead to higher resource consumption compared to containerization [2].

b) **Shared database, separate schema**
This architecture achieves a higher degree of multi-tenancy, where multiple tenants share the underlying database system, but isolation is maintained through dedicated schemas for each tenant. Providers benefit from lower costs through efficient resource consumption and reduced maintenance, and they may serve multiple tenants with fewer database licenses [2].

c) **Shared database, shared schema (Organic Multi-Tenancy)**
Also referred to as the shared everything architecture, this represents the highest level

of multi-tenancy and the lowest level of isolation. The efficient consumption of resources results in the lowest costs. In this complex design, all tenant data are stored together in one database. Maintaining isolation requires advanced schema-mapping techniques (such as Basic Layout, Pivot Table Layout, or Universal Table Layout) that link data to the respective tenants [2].

The core architecture generally includes the Database Management System (DBMS), the Cloud infrastructure (providing physical resources like computing power and storage), the Database Server (managed by the provider), and the API (offering a standardized interface for customer interaction) [5].

2.1.3 The Evolution of DBaaS Systems

The concept of DBaaS has developed in line with the growth of cloud computing. The foundation of DBaaS is rooted in the Software as a Service (SaaS) paradigm, utilizing the economic advantage provided by the "economy of the scale" principle, where applications and system environments are provided once for thousands of users [6]. Early first-generation SaaS systems utilized relational DB technology, often extending it with complex application logic to handle multi-tenancy requirements. To overcome the resulting infrastructure complexity and heavy maintenance needs, large providers (like Google, Amazon, and Yahoo!) began implementing platforms specifically for database services, promoting the idea of data outsourcing [6].

A significant milestone in the evolution was the introduction of Amazon Web Services' Relational Database Service (RDS) in 2009. Initially, cloud database systems provided simple, container-like data management services, typically featuring put/get semantics for data blocks of unknown structure, and sometimes lacking support for application semantics or complex model management aspects [6].

Today, the evolution continues with offerings that integrate both traditional RDBMS and newer systems like NoSQL (e.g., document-oriented databases like MongoDB) and newSQL technologies. These systems are designed to keep the ACID properties of traditional relational databases while leveraging shared-nothing architecture for scaling out [6]. This continuous advancement means major vendors now offer a variety of database services tailored to specific application needs [5].

2.1.4 Challenges in DBaaS Implementation

DBaaS providers face significant challenges, particularly concerning resource management, isolation, and security:

Technical and Architectural Challenges

a) **Maintaining Isolation while Minimizing Overhead**
The primary challenge is enabling multiple tenants to work in isolation while avoiding excessive resource overhead per tenant [2].

b) **Complexity of Shared Architectures**
In shared database architectures, providers must implement tenant-specific monitoring, backup/restore strategies, and billing techniques, which increases complexity. Furthermore, migrating a tenant to a different system is complex in a shared schema architecture because tenant data are mixed across various tables [2].

c) **Resource Management in Dynamic Environments**
DBaaS resource management is an online problem, requiring continuous decisions on which replicas to assign to which nodes, as tenants frequently arrive, depart, and exhibit high variance in their resource demands over time [4].

d) **Handling Resource Over-Subscription**
Providers often over-subscribe resources to improve cluster utilization and reduce costs. However, this necessitates intelligent placement decisions to prevent resource demand from exceeding capacity, leading to resource violations [4].

Security Challenges

a) **Data Confidentiality and Breaches**
Sensitive data stored in cloud databases are subject to critical concerns regarding confidentiality, as unauthorized access or data breaches pose significant risks [5].

b) **Reduced Isolation in Multi-tenancy**
The shared memory model inherent in DBaaS architectures means each virtual server could potentially access the memory address space of others, creating risks of unauthorized access to confidential information [5].

c) **Unsecured Interfaces and APIs**
Low-security interfaces or APIs are potential attack vectors.Robust interfaces should incorporate user authentication, data encryption, and access control measures. A high-quality API architecture is necessary to mitigate these risks [5].

d) **Data Integrity and Availability**
Data integrity is another significant challenge, as data can be intentionally or unintentionally modified or deleted by authorized or unauthorized users, leading to corruption or loss. Additionally, data privacy concerns arise when cloud providers fail to ensure that users' data is not accessed or used without consent [5].

e) **Malicious Insiders**
Malicious insiders present significant security risks. Attackers leveraging extensive cloud services may exploit internal vulnerabilities. Cloud service consumers are responsible for

securing encryption keys. If keys are exposed and consumers fail to retain copies, insider attacks become feasible. Relying solely on cloud providers for data protection puts consumers at risk [5].

2.1.5 Benefits of DBaaS Systems

Despite challenges, DBaaS offers substantial advantages that drive widespread adoption:

a) **Cost Savings and Economic Advantages**
Customers are relieved of the need for upfront capital investments in hardware and software. The service utilizes a pay-per-use basis, which, combined with multi-tenancy, offers the "economy of the scale" principle, leading to significant cost reduction. Furthermore, the diminished need for full-time database administrators reduces employment costs.

b) **Increased Focus on Core Competencies**
By outsourcing IT maintenance and infrastructure complexities, enterprises can redirect their focus toward their core business activities [2, 6].

c) **High Availability and Disaster Recovery**
DBaaS provides built-in multi-zone replication, automated backups, and failover mechanisms. This level of reliability is difficult to achieve in traditional on-premise deployments [3].

d) **High Scalability and Flexibility**
DBaaS provides access to dynamically scalable computing infrastructure on demand. The platform demands dynamic scaling, both horizontally and vertically, which is closely linked to the underlying database system's capabilities. Customers can choose database services tailored precisely to their needs [3].

e) **Reduced Administrative Burden**
DBaaS abstracts the installation and configuration of the database system. Providers handle maintenance work such as backups, upgrades, and restores, reducing the administrative burden on the tenant [2].

2.2 Related Works

2.2.1 Introduction to Related Works in DBaaS Systems

Database-as-a-Service (DBaaS) has emerged as a transformative paradigm in cloud computing, enabling organizations to manage, store, and access data without the need for on-premises infrastructure or dedicated database administrators. In this section, we explore related works in the realm of DBaaS, examining key research, innovative implementations, and industry best practices that have shaped this service model. By reviewing existing literature and case studies, we aim to understand the evolution of DBaaS systems, the architectures employed by major

providers, and the practical challenges faced in deploying secure, scalable, and high-performance cloud databases.

We will present a comparative overview of several DBaaS solutions, highlighting their key features, service models, and performance characteristics. This analysis sets the foundation for our proposed DBaaS implementation, guiding the design of a solution optimized for small to medium-sized enterprises (SMEs), balancing performance, scalability, and security in today's dynamic cloud computing landscape.

Platform	**SQL**	**NoSQL**	**Schema Viz.**	**Lock-in**	**AI**
RDS	✓	✗	Low	High	✗
DynamoDB	✗	✓	Low	High	✗
Google SQL	✓	✗	Low	High	✗
Firesbase	✗	✓	✗	High	✗
Azure SQL	✓	✗	Basic	High	✗
Atlas	✗	✓	Limited	Low	✓ (Limited)
PlanetScale	✓	✗	Limited	Medium	✗
NeonDB	✓	✗	Low	Medium	✗
IBM Cloud	✓	✓	✗	High	✗
Supabase	✓	✗	ERD	Medium	✓ (Limited)
Proposed DBaaS	✓	✓	**Full Visual Builder**	**Low**	✓ **(Full)**

Table 2.1: Comparison of different DBaaS platforms across key features

2.2.2 Comparison of Cloud Database Services

Amazon RDS

Amazon RDS functions as a managed relational database that sits in the cloud and connects directly to your application servers. The application interacts with RDS using standard SQL queries over a secure connection. Amazon handles the infrastructure underneath, including storage, CPU, memory, backups, and replication. For high availability, RDS can replicate data across multiple availability zones, and read replicas can serve read-heavy workloads. Essentially, your application only sees a database endpoint; the scaling, maintenance, and failover are abstracted away.

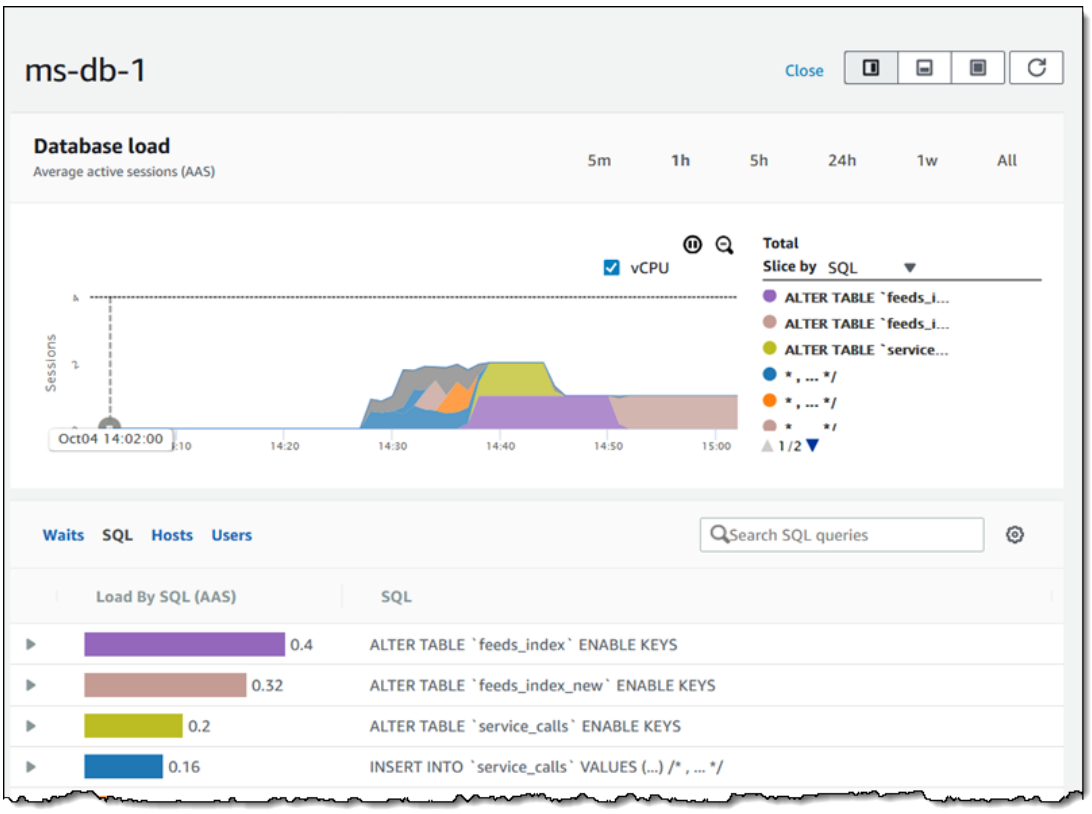


Figure 2.1: Amazon RDS console

Azure SQL Database

Azure SQL Database works similarly, acting as a fully managed SQL Server in the cloud. Applications connect using T-SQL through standard database drivers or APIs. Azure manages the underlying infrastructure, scaling, patching, and replication. It can operate in serverless mode, where compute scales automatically based on demand, or provisioned mode for predictable workloads. The database is accessible through secure endpoints, and geo-replication allows global applications to read and write data efficiently.

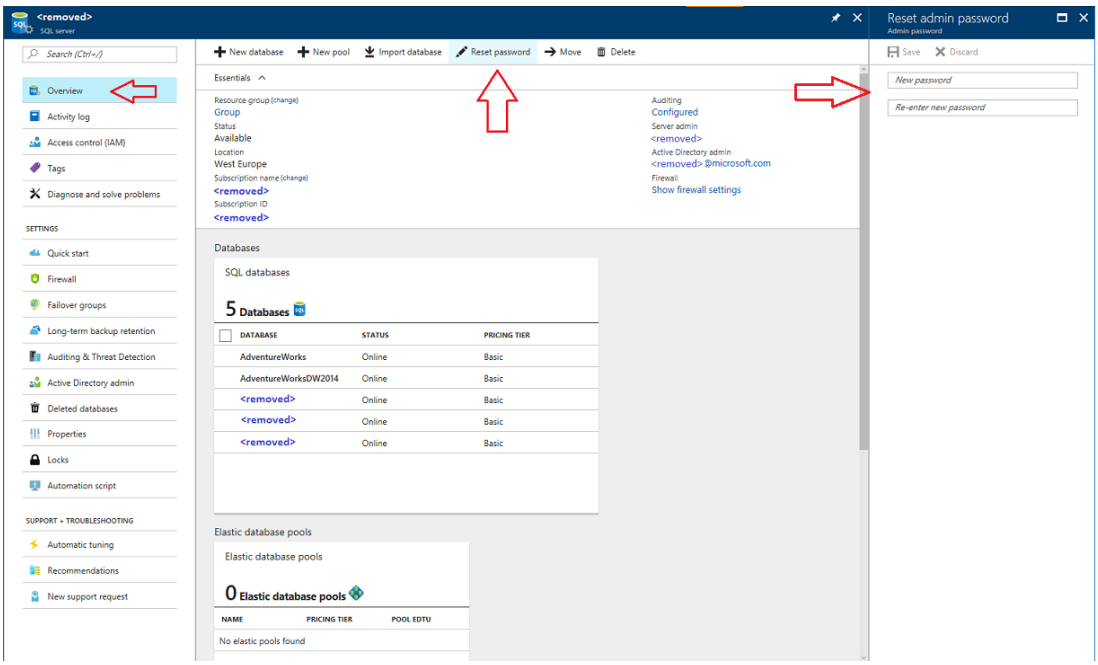


Figure 2.2: Azure SQL Database console

MongoDB Atlas

MongoDB Atlas is a cloud-hosted, document-based NoSQL database. Applications communicate with Atlas using MongoDB drivers or APIs, sending queries in the MongoDB Query Language. Atlas handles sharding and replication in the background, distributing data across multiple nodes or regions to provide scalability and resilience. Because it is schema-less, applications can store and retrieve complex, nested JSON documents without rigid schemas. Global clusters allow users from different regions to access data with low latency.

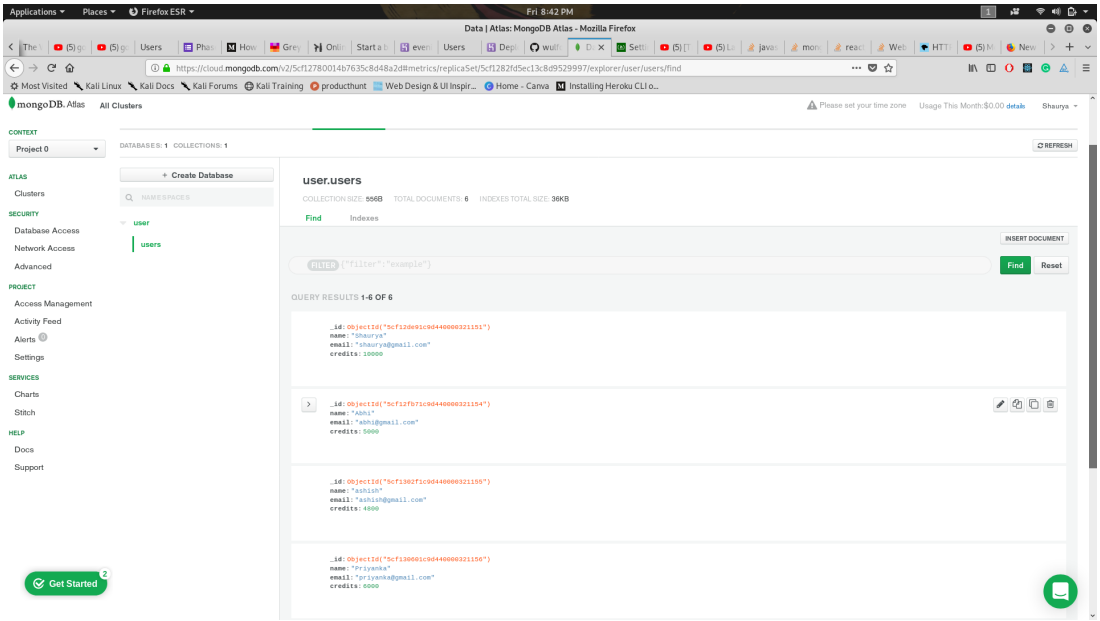


Figure 2.3: Monogodb console

Firebase

Firebase connects directly to client applications—web, iOS, or Android—through SDKs. When an app writes or reads data, Firebase automatically synchronizes changes in real time across all connected clients. The infrastructure handles scaling, replication, and offline support, so developers don't need to manage servers. Security rules and authentication integrate directly with the database, making it simple to control access at a fine-grained level.

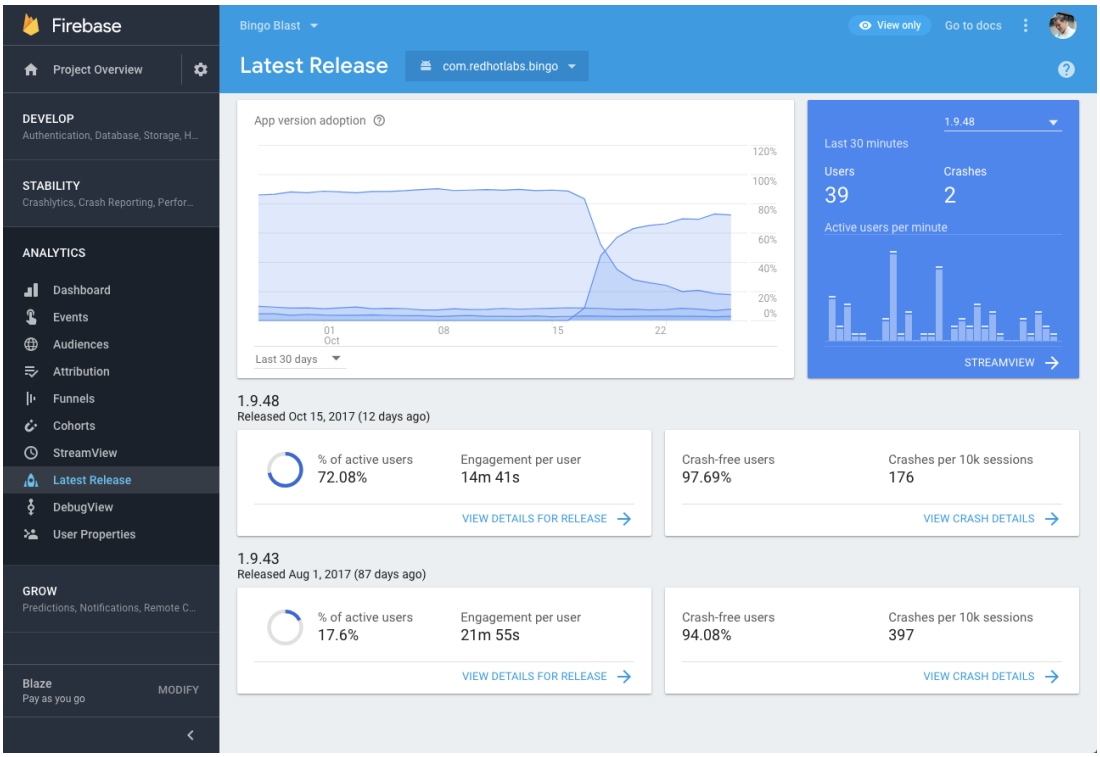


Figure 2.4: Firebase console

Supabase

Supabase uses PostgreSQL as its core, but adds an API layer on top, exposing RESTful and GraphQL endpoints for applications. Applications can also subscribe to real-time changes via websockets. Supabase manages the database infrastructure, real-time updates, authentication, and storage. This means developers can interact both with SQL queries and API calls without managing servers. Real-time subscriptions allow applications to instantly react to database changes, similar to Firebase, while keeping the benefits of a structured relational database.

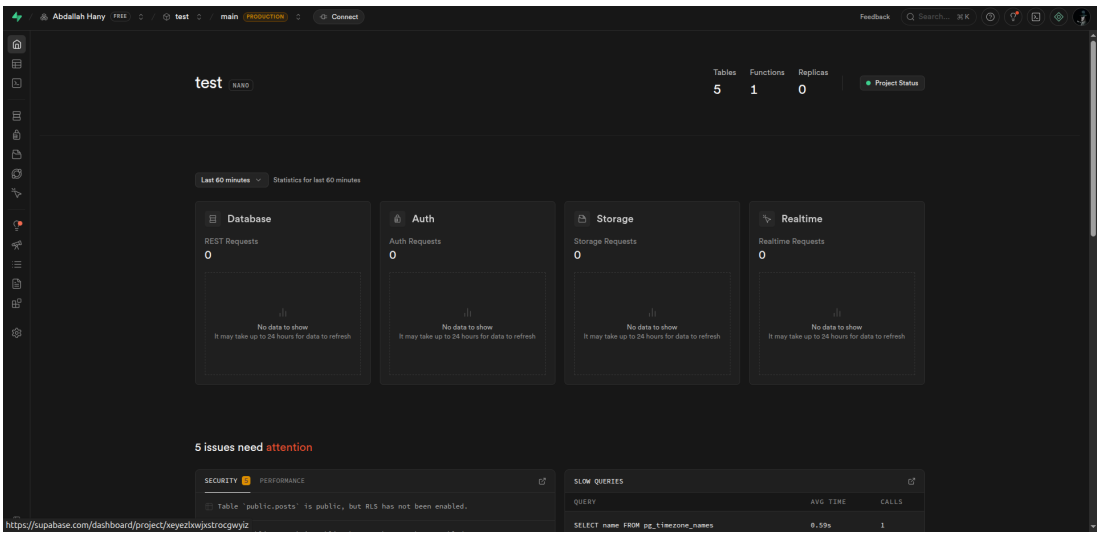


Figure 2.5: Supabase console

2.3 Containerization and Virtualization

2.3.1 Virtualization

Virtualization is a fundamental concept in modern cloud computing architectures, serving as a resource allocation method that facilitates the delivery of on-demand computing services. It is the process of running a virtual instance of a computer system [7].

Mechanism of Virtualization (VMs)

Hypervisor-based virtualization, which creates Virtual Machines (VMs), works by virtualizing hardware resources to provide the abstraction of an entire computer system. This approach allows multiple operating systems (OSes) to coexist on a single physical machine [8].

1. **Hypervisors:** Virtualization relies on software known as Hypervisors or Virtual Machine Monitors (VMMs), which run and monitor the VMs [7].
    - **Type 1 (Bare Metal):** These hypervisors interact directly with the host device's physical hardware. An example of a Type-1 hypervisor is Xen [7].
    - **Type 2 (Hosted):** These are software applications installed on top of the host operating system, similar to any regular application [7].

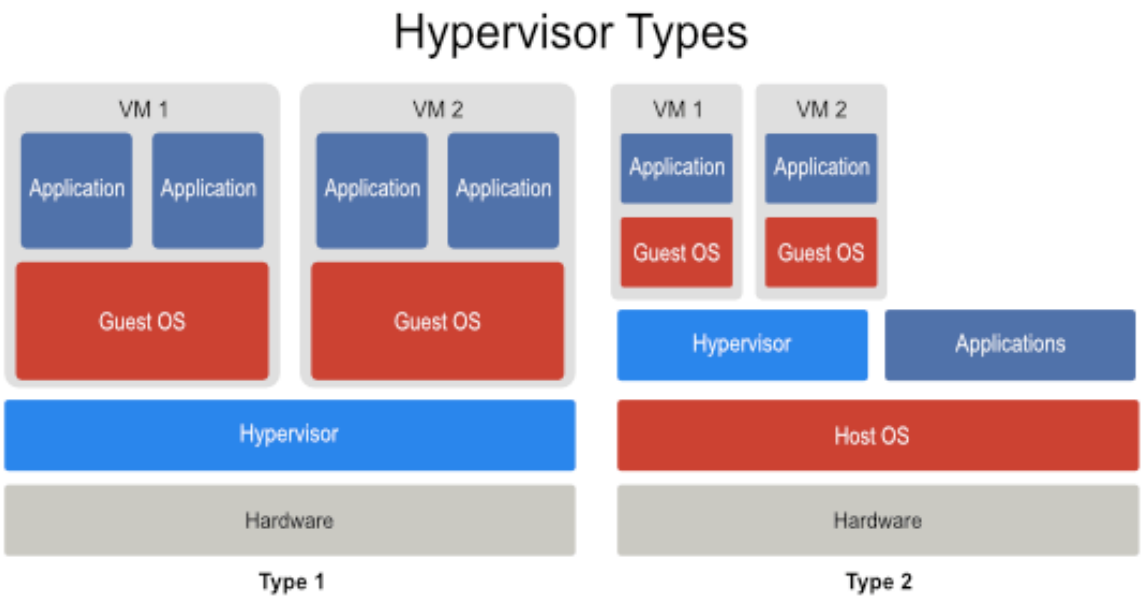


Figure 2.6: Hypervisor types

2. **Resource Allocation and Isolation:** In this model, a guest OS runs on top of the hypervisor (or host OS in Type 2). Hardware devices are virtualized, often using device emulators [8]. Crucially, each VM is provisioned with a complete, separate operating system [9].
    - **Strong Isolation:** VMs provide strong performance isolation because they separate the guest operating systems, meaning no data structures in the guest OSes are shared among VMs. This isolation is critical for ensuring that the resource consumption or failure of one VM does not affect other co-located VMs [9].
    - **Resource Overhead:** Because each deployed database or application requires its own full operating system, virtualization often leads to higher resource consumption and larger memory footprints compared to containerization. VMs are known for having slower spin-up/down times, requiring several seconds for booting [9].

2.3.2 Containerization

Containerization is an alternative resource allocation method and is sometimes referred to as OS-level virtualization or system virtualization in which only the necessary services and packages can be installed on a minimal operating system. These resources are all independent so working with their dependencies and managing them becomes easier. Applications can be abstracted from the environment in which they actually run, which allows container-based applications to be deployed easily and consistently [7].

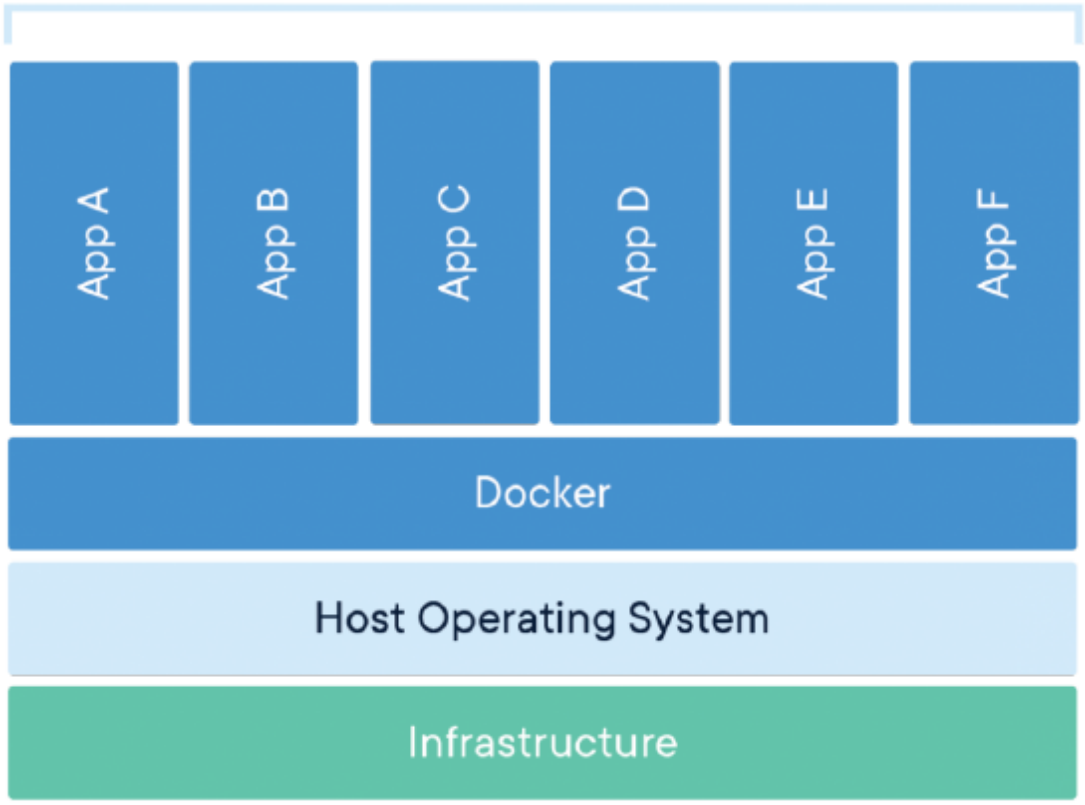


Figure 2.7: Containerized Environment making use of Docker

Mechanism of Containerization

1. **Shared Kernel Architecture:** Containers create multiple virtual units in the userspace but, unlike VMs, they share the same host kernel. This inherent sharing is why containers are considered extremely lightweight, as they eliminate the need for running a complete operating system inside each virtual environment [8, 9].
2. **Minimal Resources:** In a containerized environment, only the necessary services and packages needed to run a specific application are installed on a minimal operating system. Applications are abstracted from the environment in which they actually run [7, 9].
3. **Isolation Mechanisms:** Containers are isolated from each other through private namespaces (which partition resources to groups of processes) and cgroups (control groups, which manage and limit resources like CPU, memory, and I/O) at the OS level. Namespaces impose instance-specific resource visibility, while cgroups limit the amount of system resources used by an instance [9].
4. **Efficiency:** The design of containerization, exemplified by tools like Docker, aims to overcome the performance overheads associated with VM device emulation and running full guest operating systems. Containers are highly portable, energy and resource efficient, and significantly quick during boot-up, often requiring only a few milliseconds [8, 9].

2.3.3 Comparison and Key Differences

The primary distinction between virtualization (VMs) and containerization lies in the level at which they provide isolation and the degree to which they share the host system's resources:

Containers shift the layer of virtualization from hardware to the OS level, which generally gives them a performance advantage due to negligible virtualization overheads compared to VMs. They are often found to be more efficient in performance for SaaS (Software as a Service) and PaaS (Platform as a Service) offerings [8, 7].

Several studies have analyzed the relationship between containerization, virtualization, and cloud service orchestration. Research shows that the gap between virtualization and containerization is smaller than that in deployment and orchestration, motivating further work. Containers can run on small devices and Linux distributions, while IaaS offers cost savings by offloading infrastructure management. Virtualized applications can be deployed in IaaS environments, and containers are increasingly used in PaaS for fine-grained resource control due to faster startup times and better performance than VMs. Major companies like Amazon, Google, and Facebook provide robust cloud platforms. Technically, virtualization (e.g., KVM) involves separate guest OSs on emulated hardware, whereas containerization (e.g., Docker) runs atop the host OS and provides application-level APIs. In SaaS, tenants operate securely on a shared multi-tenant system as if they have dedicated software instances [7].

Feature	**Virtualization**	**Containerization**
Level of Abstraction	Hardware (via Hypervisor)	OS/Kernel (via Namespaces/cgroups)
Operating System	Each VM runs a full, separate Guest OS	Containers share the single Host OS kernel
Isolation	Strong (Hardware-level partitioning)	Weaker (OS-level isolation)
Resource Footprint	Large (due to full OS image)	Small/Lightweight (minimal packages)
Startup Time	Slower (requires booting OS, several seconds)	Faster (milliseconds)

Table 2.2: Comparison between containerization and virtualization

2.3.4 Database Performance

The study examined four popular relational database management systems: MySQL, PostgreSQL, Microsoft SQL Server, and Oracle. To evaluate performance, the researchers measured the

execution times of fundamental database operations, specifically INSERT, UPDATE, DELETE, and SELECT queries. Each test scenario was repeated 100 times across four increasing datasets, with the largest dataset containing 1,000,000 records in the primary table [1].

Two primary hypotheses guided the research:

1. H1 assumed that databases running on Docker containers would be more efficient than those on virtual machines.
2. H2 assumed that Oracle would be the most efficient database regardless of the environment.

The results confirmed the first hypothesis (H1), showing that databases running on Docker containers outperform instances running on virtual machines in terms of performance [1].

However, the findings rejected the second hypothesis (H2). The study found that the PostgreSQL database demonstrated a definite advantage in performance over the other analyzed databases in terms of SQL query execution speed. PostgreSQL consistently achieved the shortest execution times across INSERT, DELETE, and UPDATE scenarios [1].

While containers often outperform VMs in general application deployment speed and resource utilization, the optimal choice for databases, particularly in Database-as-a-Service (DBaaS) environments, is nuanced and depends heavily on the workload characteristics, especially concerning Input/Output (I/O) operations and isolation requirements. The MySQL database obtained comparable results to PostgreSQL, generally showing only slightly longer execution times. In contrast, the Oracle system was typically ranked third or last among the systems tested, depending on the specific scenario, and MS SQL Server often performed the worst, especially with larger datasets in DELETE and UPDATE operations [1].

Database	Average execution time of database queries (ms)			
	Database no 1	Database no 2	Database no 3	Database no 4
MySQL container	4,059	4,080	4,578	4,410
MySQL virtual machine	9,047	9,986	9,040	9,382
PostgreSQL container	1,665	1,664	1,660	1,655
PostgreSQL virtual machine	2,549	2,747	2,562	2,344
Microsoft SQL Server container	7,404	7,839	11,835	13,978
Microsoft SQL Server virtual machine	9,000	13,447	18,651	20,595
Oracle container	9,505	10,072	15,023	14,218
Oracle virtual machine	10,628	10,658	10,223	11,314

Figure 2.8: Average query execution times for the scenario Research Programme No. 1 [1].

Database	Average execution time of database queries (ms)			
	Database no 1	Database no 2	Database no 3	Database no 4
MySQL container	4,107	4,804	4,502	4,871
MySQL virtual machine	9,332	9,046	9,219	11,429
PostgreSQL container	1,893	2,142	3,687	3,200
PostgreSQL virtual machine	2,668	2,743	4,653	3,998
Microsoft SQL Server container	11,883	18,183	75,155	308,270
Microsoft SQL Server virtual machine	16,745	22,428	80,450	364,955
Oracle container	9,478	8,797	16,369	13,422
Oracle virtual machine	10,539	9,575	17,543	23,677

Figure 2.9: Average query execution times for the scenario Research Programme No. 2 [1].

Database	Average execution time of database queries (ms)			
	Database no 1	Database no 2	Database no 3	Database no 4
MySQL container	4,135	4,626	4,486	5,086
MySQL virtual machine	9,506	8,699	9,357	12,791
PostgreSQL container	1,922	2,179	3,817	3,381
PostgreSQL virtual machine	3,161	3,079	4,790	4,092
Microsoft SQL Server container	13,312	19,109	78,860	309,793
Microsoft SQL Server virtual machine	18,797	25,219	84,461	371,181
Oracle container	48,591	56,526	63,944	59,614
Oracle virtual machine	50,727	54,686	60,560	62,214

Figure 2.10: Average query execution times for the scenario Research Programme No. 3 [1].

In summary, the research concluded that the containerized environment offers superior performance for relational databases compared to virtualization, and that PostgreSQL currently leads in efficiency among the tested systems [1].

2.3.5 Databases and Workload Sensitivity

Database Management Systems (DBMSs) are a common and critical service in the cloud, where disk I/O performance and performance isolation are crucial. DBMS workloads are typically update-intensive and issue frequent fsync calls to maintain file system consistency, which significantly relies on journaling mechanisms [8].

The I/O Performance Anomaly in Containers

Contrary to the general belief that containers (like LXC) inherently outperform hypervisor-based virtualization (like KVM) due to less overhead, research has shown that for DBMS workloads, KVM often outperforms LXC in both I/O performance and isolation [8].

Contrary to the common assumption that container-based virtualization offers superior performance due to lower overhead, prior studies indicate that hypervisor-based virtualization (e.g., KVM) can outperform containers (e.g., LXC) for database workloads. This performance discrepancy primarily arises from file system journaling behavior [8]..

In containerized environments, all containers share a single host kernel and a common journaling module, which serializes commit operations across containers. As a result, database workloads may experience increased latency and reduced throughput due to interference from unrelated I/O activities [8]..

In contrast, virtual machines maintain independent file systems and journaling modules, enabling parallel commit operations and minimizing cross-tenant interference. Empirical results demonstrate that KVM can achieve up to 64higher throughput than LXC for MySQL workloads, highlighting its suitability for I/O-intensive DBMS applications [8].

Performance Isolation and Resource Guarantee

For DBaaS, maintaining strong isolation and resource guarantees is mandatory to meet service level agreements.

1. **Weaker Isolation in Containers:** Since containers share kernel data structures and buffer caches at the OS level, performance isolation becomes weaker and harder to achieve. The journaling issue further compounds this; I/O requests originating from the journaling module run outside the container's control group (cgroup) and are not accurately accounted for the container that initiated the commits. As a result, containers may violate resource limits imposed by cgroups [8].
2. **VM Isolation for DBaaS:** VMs provide strong isolation because they run separate guest OSes [8]. Resource reservation within a multi-tenant database context, such as Microsoft SQL Azure, is crucial for performance isolation, ensuring that one tenant's workload is unaffected by co-located tenants [10]

2.3.6 Conclusion on Database Suitability

For high-performance, mission-critical, and highly consolidated Database-as-a-Service (DBaaS) environments where disk I/O and strict performance isolation (SLAs) are paramount, virtualization (VMs) is typically the better-suited choice. This is due to the fundamental architecture: VMs offer stronger performance isolation and bypass the bottleneck created by the shared journaling mechanism inherent in OS-level virtualization [9].

However, if the priority is fast deployment, efficient resource utilization, and lighter overheads for platforms like PaaS that can tolerate the shared I/O stack, containers are often preferred [7].

The overall trend seen in the market is that virtualized environments are better suited for IaaS (Infrastructure as a Service) infrastructures, where robustness is valued highly, while containers are found to be efficient for PaaS and SaaS offerings [7].

2.4 AI Agents

The concept of an *intelligent agent* has been central to artificial intelligence (AI) since the field's earliest formulations. Historically, the agent abstraction was introduced to formalize computational systems that *sense* an environment, *decide* on actions, and *act* in order to achieve objectives or to reduce error according to some criterion. This sense–think–act loop draws on intellectual antecedents that span multiple disciplines. Norbert Wiener's foundational exposition of cybernetics emphasized feedback, control, and communication in both biological and engineered systems and thereby provided a formal vocabulary for thinking about closed-loop behavior that adapts to an environment [11]. Early symbolic AI adopted and developed agent taxonomies and architectures oriented around planning, search, and knowledge representation; these symbolic systems relied on explicit rule sets, logical inference, and handcrafted world models to produce action sequences in constrained domains. Complementing symbolic approaches, the study of natural language interaction—epitomized by Joseph Weizenbaum's ELIZA—demonstrated both the promise and the pitfalls of programmatic conversational behavior: ELIZA's pattern-matching and re-assembly rules generated the *appearance* of dialogic competence while exposing the gap between surface textual fluency and semantically grounded, environment-anchored agency [12]. The interplay of these strands—feedback control (cybernetics), symbolic planning (classical AI), and rudimentary conversational interfaces—set the stage for later statistical and neural methods that would substantially change the substrate on which agents are built.

During the late twentieth and early twenty-first centuries, statistical and machine learning approaches matured into practical tools for perception and decision making. Reinforcement learning (RL) formalized agent learning as the optimization of a cumulative return within an environment modeled as a Markov Decision Process (MDP) or, more generally, a partially observed MDP (POMDP). Canonical methods—temporal difference learning, Q-learning, and actor-critic algorithms—provided the algorithmic foundations for training agents interacting sequentially with environments (see, e.g., the canonical text by Sutton and Barto). The introduction of deep neural function approximators into RL—most prominently demonstrated by Deep Q-Networks (DQN) that learned to map raw pixel observations to actions with human-level performance on Atari games—showed that end-to-end learning could produce effective task-oriented controllers from high-dimensional sensory inputs [13, 14]. Nonetheless, RL and earlier agentic systems were often *narrow*: they were designed and optimized for specific tasks or domains, required substantial task-specific engineering (reward shaping, environment simulators), and typically lacked the flexible natural-language interface that characterizes modern open-ended human–machine interaction.

The advent of large pretrained language models (LLMs) in the 2010s and early 2020s created a new capability frontier. Transformer architectures, introduced by Vaswani *et al.*, provided scalable, attention-based sequence models that could be trained efficiently on massive corpora and that learned long-range contextual dependencies critical to linguistic competence [15]. The paradigm of self-supervised pretraining followed by task-specific adaptation (fine-tuning or prompting) demonstrated that very large autoregressive or encoder–decoder models internalize rich linguistic, factual, and procedural priors that can be exploited in zero- and few-shot regimes; GPT-3, for example, empirically illustrated how scaling model size and pretraining data substantially improves generalization and in-context learning ability [16]. Concurrently, retrieval-augmented methods provided mechanisms for coupling parametric model knowledge with non-parametric external memories (e.g., dense passage retrieval), enabling models to consult large knowledge stores at inference time to reduce factual errors and to extend effective context beyond the model's token window [17].

Together, these advances suggested a radical reconceptualization of agent design: rather than constructing narrowly engineered controllers, one could position LLMs as *agentic cores* ("brains") that combine broad world knowledge, language understanding, and flexible reasoning with engineered periphery components (memory stores, tool APIs, execution middleware) to realize richer autonomous behavior. Starting circa 2022–2023, the research literature began to formalize this perspective and to introduce concrete methodologies for turning LLMs into autonomous or semi-autonomous agents. Two influential motifs emerged in this literature: (i) *tool augmentation*—training or prompting LLMs to decide when and how to call deterministic or specialized tools (calculators, search engines, code execution sandboxes) to improve accuracy and grounding [18]; and (ii) *reasoning–acting interleaving*—techniques that permit the model to produce explicit reasoning traces interleaved with action tokens, thereby enabling verification, iterative planning, and error correction (e.g., the ReAct paradigm) [19]. These developments reframed the old sense–think–act cycle: the LLM often serves simultaneously as an interpreter of natural language, a planner of multi-step procedures, and a controller that issues executable instructions, while external modules supply grounding and verifiable computation.

The remainder of this section surveys canonical architectural patterns for LLM agents found in contemporary research, formalizes their decision processes, and discusses practical mediation mechanisms such as retrieval-augmented generation and schema-based tool invocation.

2.4.1 Architectural Taxonomy for LLM Agents

Contemporary LLM agent architectures can be characterized along a design spectrum that ranges from *single-model policy agents* to *modular planner–executor systems* and to *tool-augmented hybrid systems*. Each design exhibits distinct trade-offs in interpretability, sample efficiency, computational cost, and suitability for long-horizon tasks.

Single-Model Policy Agents (LLM as a Policy)

The most straightforward architecture treats a pretrained LLM as a stochastic policy π_θ that maps a textual *context* c_t to a distribution over tokens, which are then parsed into higher-level *actions* executed in the environment. Formally, at discrete time step t the model yields

$$\text{token}_{t+1} \sim p_\theta(\cdot \mid c_t),$$

and a higher-level action a_t is obtained by deterministically parsing the emitted token sequence (for example, decoding a JSON object into an API call). One may therefore express the induced policy over environment actions as

$$\pi_\theta(a_t \mid s_t) \;=\; \sum_{x \in \mathcal{T}(a_t)} p_\theta(x \mid c_t), \tag{2.4.1}$$

where $\mathcal{T}(a_t)$ denotes the set of token sequences that parse to action a_t, and s_t denotes the environment state (possibly only partially observed and implicitly represented within c_t). Objective-wise, training or adapting such a policy aims to maximize an expected return,

$$J(\theta) \;=\; \mathbb{E}_{\pi_\theta, \mathcal{E}}\left[\sum_{t=0}^{T} r_t\right],$$

with r_t the environment reward and the expectation taken over environment dynamics and the policy's stochasticity.

In practice, single-model policies are commonly augmented with retrieval systems (retrieval-augmented generation, RAG) to extend effective context beyond the architecture's maximum token window: a retrieval module returns a set of relevant documents or knowledge snippets which are concatenated (or summarized) into the prompt given to the model, thereby reducing hallucination and improving factual accuracy [17]. Tool use is implemented through a middleware layer that maps specific token patterns to API invocations; recent work shows that models can be trained to *decide* both *when* to call a tool and *how* to incorporate the tool's output into subsequent token prediction. The Toolformer methodology is an example of a self-supervised procedure in which candidate API call sites are discovered and judged by their impact on language modeling likelihood, producing models that learn to employ calculators, search, Q&A endpoints, and other utilities selectively [18].

Practical considerations. Single-model policies are attractive for their simplicity and for minimizing engineering overhead: a single LLM instance, given a carefully designed prompt and a retrieval front end, can accomplish a wide range of tasks. However, this design places the entire burden of multi-step planning, long-term state management, and error correction on token-level generation. Token budget constraints limit how much historical context can be

considered directly, and complex, safety-sensitive tool invocations require conservative validation middleware to avoid executing malformed or adversarially crafted actions.

Modular Planner–Executor Architectures

A more robust and interpretable design decouples high-level planning from low-level execution by creating a hierarchical pipeline. In this pattern, an LLM may serve as the *planner* that emits a structured plan $P = \text{Plan}(c_t)$ consisting of subgoals $\{g_1, \ldots, g_k\}$; each subgoal is then dispatched to an *executor* (which may be another LLM with a focused prompt or a deterministic tool) that performs the concrete actions. Concretely, the operational loop alternates:

1. **Planner:** produce $P_k = \text{LLM}_{\text{plan}}(c_t)$, a plan expressed in natural language or in a structured schema (e.g., a sequence of API calls).
2. **Executor:** for each subgoal $g \in P_k$, invoke an executor module (specialized LLM or deterministic function) to carry out g, obtaining outcome o_g.
3. **Feedback:** aggregate $\{o_g\}$ into an updated context c_{t+1} and replan if necessary.

This decomposition affords several technical advantages. First, it improves interpretability: intermediate plans and subgoal outcomes are explicit and inspectable, enabling human audits or automatic verification checks. Second, it enables targeted fine-tuning: the planner and executor may be adapted independently (for example, use a smaller specialized model for deterministic execution tasks while retaining a larger model for high-level planning). Third, hierarchical control facilitates retries and conditional branching: if an executor fails, the planner can revise the plan based on the observed failure mode.

A particularly effective variant interleaves explicit *reasoning traces* with actions; this is the design principle behind the ReAct paradigm. In ReAct, the LLM alternates between producing *reasoning tokens* r_t (chain-of-thought style traces that document internal deliberation) and *action tokens* a_t (which may correspond to tool calls or textual actions). Symbolically one may write the trajectory as

$$[r_1, a_1, r_2, a_2, \ldots] \sim \pi_\theta(\cdot \mid c_t),$$

and empirical evidence suggests that producing explicit, human-interpretable reasoning traces while acting improves both performance and debuggability on tasks that require multi-step planning and interaction with external knowledge sources [19].

Training regimes. Modular agents are typically trained using a mixture of supervised fine-tuning on (context, plan, execution) triples and reinforcement learning where planners are scored based on downstream executor outcomes. The modularity reduces the scale of data required for each subcomponent and allows for curriculum design in which simpler plans are learned first and complexity gradually increases.

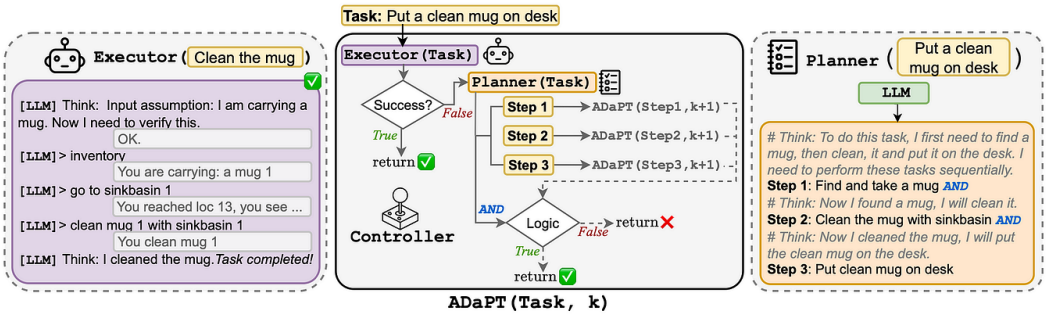


Figure 2.11: Planner–Executor architecture for schema generation.

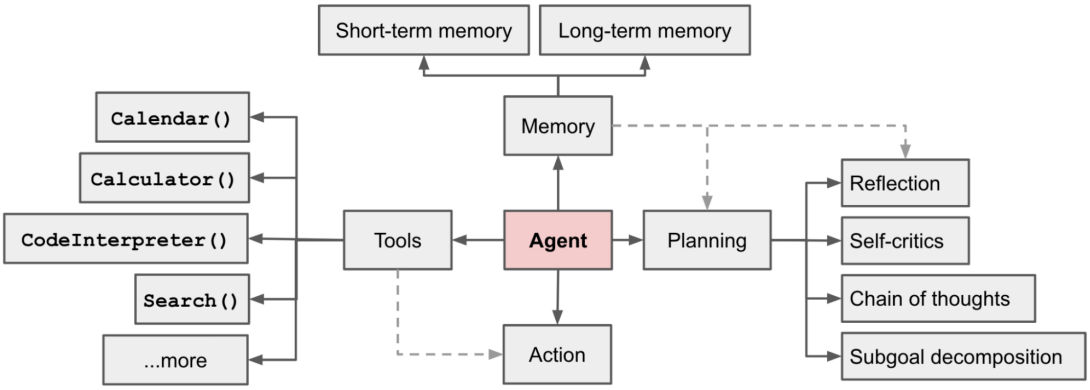


Figure 2.12: AI Agent Tool-Calling Architecture.

Tool-Augmented and Meta-Learned Tool Use

A dominant engineering pattern for high-fidelity agent behavior is explicit *tool augmentation*. In this design, a set $\mathcal{U} = \{u_1, \ldots, u_m\}$ of external utilities (search, calculator, code runner, database queries, domain APIs) is registered with the agent platform. The LLM learns (or is prompted) to select an element $u \in \mathcal{U}$ and to provide schema-conforming arguments; middleware then invokes u, and its output is reintegrated into the prompt or into a memory store. The canonical pipeline is:

1. **Parse:** map a structured subsequence of tokens to an API call call$(u; \alpha)$ where α denotes arguments.
2. **Invoke:** execute the tool in a sandboxed runtime, returning output o_u.
3. **Embed:** convert o_u to text and/or vector embeddings, and insert into the agent's context (possibly after summarization) for subsequent steps.

Work such as Toolformer demonstrates that models can be trained in a largely self-supervised fashion to identify beneficial tool call sites and to rewrite prompts so that the tool output improves downstream language modeling likelihood; the Toolformer objective augments the base language modeling objective with a selection criterion that promotes tool calls that increase predictive utility [18]. This meta-learning of tool usage is powerful because it combines the flexible, generalizable reasoning of a neural model with the deterministic correctness of specialized

utilities.

Safety and validation. Tool invocation requires defensible safety mechanisms. Middleware typically implements (i) schema validation on arguments before invocation, (ii) sandboxing and resource limits during execution, and (iii) post-invocation verification (e.g., cross-checking computational results or enforcing type/format invariants) prior to re-inserting outputs into the model context.

2.4.2 Multi-agent environments, roles, and emergent coordination

As a continuation of the architectural patterns surveyed above (single-model policies, planner–executor decompositions, and tool-augmented hybrids), recent work situates those agent designs inside *multi-agent systems* (MAS) constructed from LLMs. In this paradigm, an LLM is no longer only an isolated decision maker but a node in a network of interacting agents that exchange messages, coordinate on subtasks, share memory, and jointly form world models. Research converges on several core design dimensions that determine the behavior, scalability, and interpretability of LLM-based MAS: (1) specification of agent *profiles* (capabilities, tool sets, priors, and prompt templates); (2) selection of *interaction topologies* (who can talk to whom and when); (3) communication protocol design (natural language vs. structured schemas, synchronous vs. asynchronous messaging); (4) environment and world-model fidelity (abstract dialogue arenas vs. embodied or web-like environments); and (5) metrics and theoretical constructs for quantifying emergent collective properties such as synergy, division of labor, and coalition formation [20].

Multi-agent LLM research uses a range of environment models that vary by fidelity, observability, and action expressiveness. At one end are *abstract dialogue arenas*: controlled, language-only settings in which agents exchange messages, negotiate plans, or play coordination games. At the other end are *embodied* or *web-like* environments where agents manipulate objects, navigate interfaces, or perform multi-step tasks requiring grounding, tool invocation, and state tracking. Representative platforms include TextWorld, ALFWorld, and WebArena . Formally, a multi-agent environment can be described as a stochastic game $(\mathcal{S}, \mathcal{A}_1, \ldots, \mathcal{A}_n, P, R_1, \ldots, R_n)$, where $\mathcal{S}$ is the global state space, $\mathcal{A}_i$ the action space of agent i, P the transition kernel $P(s_{t+1} \mid s_t, a_{1,t}, \ldots, a_{n,t})$, and R_i the reward function for agent i [21]. For LLM agents, the action space $\mathcal{A}_i$ is hybrid: (i) natural-language utterances $u \in \mathcal{U}$, (ii) structured messages (JSON/Protobuf payloads), and (iii) tool invocations $t \in \mathcal{T}$. Observations are typically partial: $o_{i,t} = \mathcal{O}_i(s_t)$, and the LLM-based policy is $\pi_{\theta_i}(a_{i,t} \mid h_{i,t})$, where $h_{i,t}$ merges observations, memory, role prompt, and tool outputs.

Functional specialization is a recurring pattern: agents are assigned roles—planner, critic, executor, retriever, summarizer, tool-wrapper, user proxy—that map to distinct prompts, tool permissions, and priors. Each agent i has a profile $P_i = (\mathcal{C}_i, \mathcal{T}_i, \pi_{\theta_i}^{\text{init}})$. Tasks T can be decomposed

into subtasks $T = \bigcup_j t_j$ and allocated via $A : \{t_j\} \to \{1, \ldots, n\}$. Explicit role definitions improve throughput and correctness compared to homogeneous teams. Systems like HuggingGPT and AutoGen exemplify planner/selector/executor decompositions, orchestrating specialized agents [22, 23]. The team policy can be expressed as

$$\Pi((a_{1,t}, \ldots, a_{n,t}) \mid h_{1,t}, \ldots, h_{n,t}) = \mathcal{C}\big(\pi_{\theta_1}(a_{1,t} \mid h_{1,t}), \ldots, \pi_{\theta_n}(a_{n,t} \mid h_{n,t})\big),$$

where $\mathcal{C}$ enforces coordination constraints.

A principal research question is whether LLM-based MAS produce *emergent* behaviors beyond individual capabilities. Information-theoretic tools such as Partial Information Decomposition (PID) quantify synergy by decomposing predictive information into unique, redundant, and synergistic components across agents' message streams. Empirical results suggest that shared rewards, repeated interactions, and role heterogeneity promote measurable synergy and emergent communication protocols.

2.4.3 Communication in Multi-Agent LLM Systems: Protocols, Emergent Languages, and Grounding

Communication is the operational substrate of coordination in LLM-based multi-agent systems (LLM-MAS). In these systems, the efficacy of task decomposition, planning, and execution hinges on how agents exchange information, propagate state updates, and negotiate subtasks. From an engineering perspective, communication is both a design choice—human-readable natural language versus structured machine-readable payloads—and a learning phenomenon—agents may develop emergent protocols that optimize efficiency over interpretability. The interplay between these dimensions depends on the use case, safety and interpretability requirements, and environment information structure.

Communication modalities and message schemas

Two canonical modalities dominate contemporary LLM-MAS implementations:

Natural-language messaging. Agents exchange text in human languages (e.g., English). Advantages include interpretability, auditability, and the ability for human overseers to intervene in real time. However, natural language introduces inherent ambiguity, verbosity, and potential misalignment with formalized actions. To mitigate these issues while preserving interpretability, many systems adopt templated messages or structured-natural hybrids. For example, responses may include a machine-parseable JSON snippet preceded or followed by natural-language explanation. Formally, let $m_{i,t} = \text{NL+JSON}(a_{i,t}, r_{i,t})$ be the message generated by agent i at

time t, where $a_{i,t}$ is the intended action and $r_{i,t}$ is the human-readable rationale. This approach underpins frameworks such as AutoGen and HuggingGPT [23, 22, 20].

Structured messaging. Structured messages encapsulate actions, subtask proposals, beliefs, or state updates in formats such as JSON or Protobuf. Formally, an agent's message $m_{i,t} \in \mathcal{M}_i$ must satisfy a schema $\mathcal{S}_i$ and can be deterministically parsed into action sequences:

$$\mathcal{F}_i : m_{i,t} \mapsto \{a_{i,t}^{(1)}, \ldots, a_{i,t}^{(k)}\}.$$

Structured messaging reduces ambiguity, supports formal verification, and enables safe execution of tool invocations. Hybrid approaches—structured payloads augmented with natural-language summaries—combine auditability and machine safety, allowing human interpreters to review agent reasoning while automated processes consume formal payloads [18].

Emergent communication and agent protocol learning

Emergent communication studies in LLM-MAS extend classical referential games and signaling theory to the context of pre-trained language priors. When agents repeatedly interact under bandwidth constraints, efficiency pressures, or task-specific rewards, they may develop compressed, idiosyncratic protocols $E : \mathcal{H}_{i,t} \mapsto m_{i,t}$ that diverge from human language. Here $\mathcal{H}_{i,t}$ represents the agent's interaction history, memory, and retrieved context. Iterated learning and generative pressures amplify inductive biases learned during pretraining, producing structured, compositional codes optimized for internal model representations. Empirical studies demonstrate that these protocols often exhibit:

- **Compression and efficiency**: messages minimize token length while maximizing actionable content.
- **Task-specific syntax**: emergent grammars are tailored to internal reasoning chains and tool invocation sequences.
- **Opacity to humans**: while efficient, emergent codes may be unintelligible without translation layers.

Design tradeoffs must balance interpretability and efficiency. Constraining agents to human-readable formats preserves auditability, essential in domains such as clinical decision support, finance, or customer automation. Allowing controlled emergent protocols can improve throughput in internal pipelines but requires monitoring, translation agents, or intermittent supervision to ensure traceability [24].

The formal study of communication in LLM-MAS therefore spans hybrid message design, emergent protocol learning, and strategic interactions. By integrating these perspectives, system

designers can craft agents that communicate efficiently, maintain safety, and adapt dynamically to multi-agent coordination challenges.

2.4.4 AutoGen framework

AutoGen is an open-source, research-grade multi-agent conversation framework developed to enable complex LLM applications where multiple agents converse, invoke tools, and cooperate to achieve high-level goals. The AutoGen technical report and accompanying documentation describe the framework primitives, conversation orchestration, memory and tool integration, and practical implementation patterns; the exposition below synthesizes those descriptions and makes explicit the engineering and algorithmic structure underlying AutoGen's design [25, 26, 27].

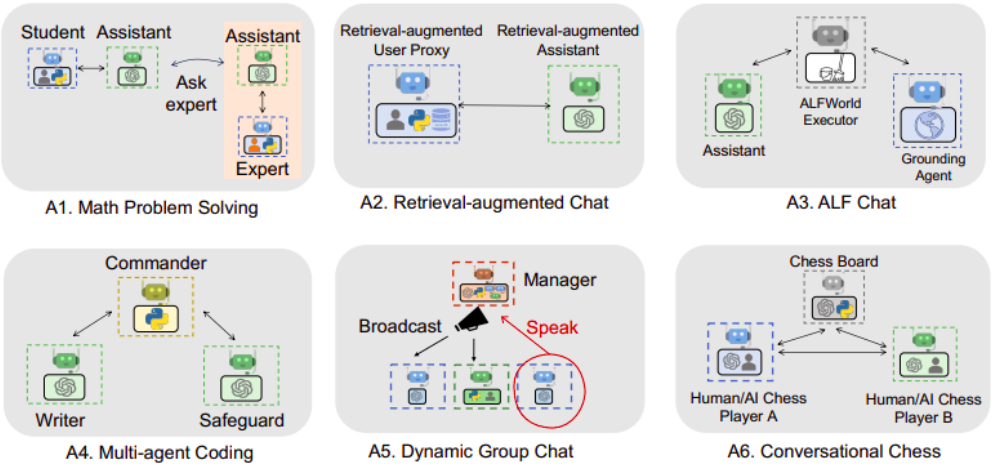


Figure 2.13: Examples of AutoGen multi-agent interaction diagram.

AutoGen core architecture and abstractions

AutoGen's runtime is organized around a small set of first-class abstractions that map directly to conversation programming constructs and to the LLM prompting pipeline:

- **Agent.** An *agent* is a composable object encapsulating (i) a *role profile* (system prompt template, instruction scaffolding, and model configuration); (ii) a *tool registry* (a set of authorized external functions/APIs with type signatures); and (iii) a *memory interface* (episodic logs, summaries, or handles to persistent vector stores). Agents are interoperable: a single LLM backbone can instantiate multiple agents by switching role templates and tool permissions at runtime. Agents may be executed locally or via external LLM APIs. The AutoGen paper formalizes agents as tuples $A_i = (R_i, \mathcal{T}_i, \mathcal{M}_i, \lambda_i)$ where R_i is the role prompt, $\mathcal{T}_i$ the tool set, $\mathcal{M}_i$ the memory access interface, and λ_i the model invocation parameters (temperature, max tokens, stop tokens). [25]

- **Dialogue/Conversation Manager.** The Conversation Manager (CM) is the orchestration layer that (a) schedules turns among agents, (b) constructs per-turn prompts by merging role prompts, recent dialogue, and retrieved memory, (c) validates and parses agent outputs, and (d) mediates tool invocation and result reinsertion. The CM implements pluggable scheduling policies (round-robin, priority queues, event triggers) and ensures token-budget and API-call budgets are respected. [25, 27]

- **Tool / Plugin Interface.** Tools are registered with signatures and safety metadata. The CM maps structured tokens emitted by agents (e.g., well-formed JSON) to tool calls and routes responses back into the conversation context. Tool execution occurs in sandboxed runtimes with pre/post-condition checks and resource limits (timeouts, memory caps). [25]

- **Memory Stores.** AutoGen supports pluggable backends (in-memory logs, persistent text stores, and vector indices). Memory is structured into layers—episodic history, compressed summaries, and semantic indices for retrieval. The CM exposes retrieval APIs used in prompt construction and implements background compaction/summarization pipelines to keep the prompt footprint manageable. [25, 27]

- **Message Schemas and Validators.** AutoGen encourages schema-driven exchanges. Agents are expected to emit outputs conforming to developer-declared JSON/Protobuf schemas; the CM validates outputs and applies fallback corrective strategies if validation fails (clarification request, heuristic repair, verifier invocation). [25]

These abstractions convert conversational programming tasks into a disciplined prompt-construction and execution pipeline: when it is an agent's turn, the CM computes a prompt P by composing the agent's role prompt R_i, the retrieved memory M_{ret}, and recent dialogue H_{recent}; then the LLM is invoked to produce a token sequence that is post-processed into actions or tool calls.

Conversation orchestration logic

AutoGen conceptualizes multi-agent coordination in terms of a small set of reusable *conversation patterns*. Each pattern specifies how turns are selected, how context is propagated across interactions, and how intermediate state is summarized and reused. Rather than exposing a single low-level scheduling loop, AutoGen emphasizes higher-level abstractions that balance modularity, controllability, and cost efficiency.

The AutoGen framework identifies several canonical interaction patterns:

- **Two-agent conversation.** The simplest pattern involves a pair of agents engaging in a bounded multi-turn dialogue, typically initiated via an explicit chat invocation. The result of such a conversation is encapsulated in a structured artifact that includes the full

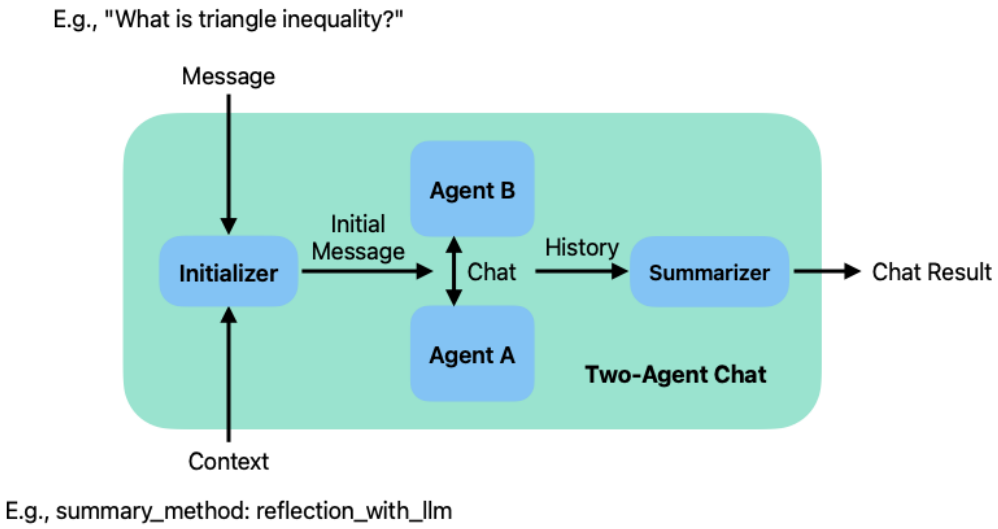


Figure 2.14: Two-agent conversation pattern

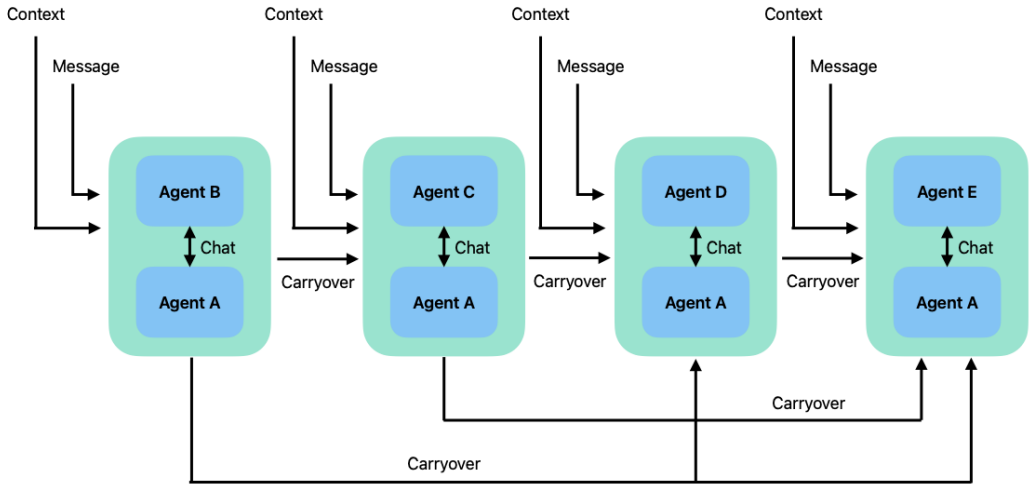


Figure 2.15: Sequential converstation pattern

message trace, a concise summary, and token or cost statistics. This pattern is well suited for focused subproblems or bilateral reasoning.

- **Sequential (carryover) conversations.** More complex workflows are expressed as a sequence of two-agent conversations, where each interaction produces a summary that is injected as carryover context into the subsequent one. This enables staged task decomposition (e.g., analysis → planning → execution) while controlling context growth through configurable summarization policies.

- **Group conversations.** In this pattern, multiple agents participate in a shared dialogue. A central design choice concerns

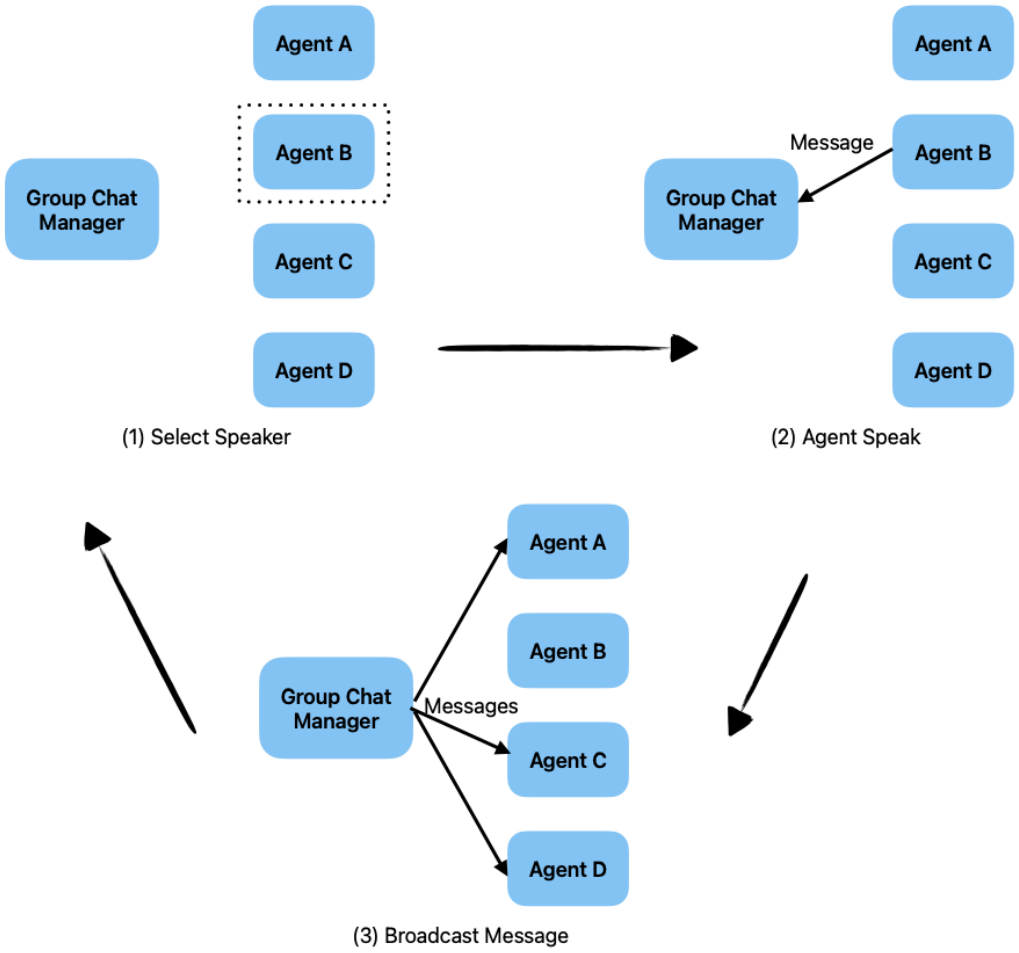


Figure 2.16: Group conversation pattern

speaker selection : In group conversations, AutoGen delegates turn-taking to a *Group Chat Manager*, which is responsible for selecting the next speaking agent according to a configurable strategy. Currently supported strategies include: *round_robin*, which deterministically cycles through agents in a fixed order; *random*, which samples the next agent uniformly at random; *manual*, in which turn selection is deferred to human input; and *auto*, the default strategy, where an LLM-based policy selects the next agent based on the evolving conversational context. These strategies offer a spectrum of trade-offs between determinism, human oversight, flexibility, and computational cost, and may be combined with additional constraints or custom selection functions to enforce domain-specific interaction policies..

- **Nested conversations.** Nested or packaged conversations encapsulate an internal multi-agent workflow behind a single agent interface. From the perspective of the outer orchestration layer, the nested workflow behaves as a single agent, enabling hierarchical composition and reuse of common interaction templates.

Design considerations. The choice of conversation pattern depends on the structure of the task and operational constraints. Two-agent and unconstrained group conversations favor flexibility and exploratory reasoning, whereas sequential and constrained group patterns support predictable pipelines and bounded costs. Nested conversations facilitate modular system design by isolating complexity and promoting reuse.

Overall, AutoGen's conversation patterns provide a declarative layer for multi-agent orchestration, shifting the focus from low-level scheduling mechanics to higher-level interaction design and enabling systematic reasoning about cost, context, and control.

Tool integration and verification

AutoGen treats tools as typed, sandboxed functions. Tool registration includes a name, a typed signature, resource constraints, and an optional validator. The mapping from textual output to a tool invocation is mediated by a parser $\mathcal{P}$ that extracts structured arguments from the produced tokens. The tool invocation pipeline is:

$$\text{tokens} \xrightarrow{\mathcal{P}} \text{args} \xrightarrow{\text{schema\_check}} \text{call} \xrightarrow{\text{sandbox}} \text{result} \xrightarrow{\text{postprocess}} \text{context\_update}.$$

Key safety primitives described in the AutoGen report are:

- **Pre-condition / schema checks:** require tool arguments to satisfy developer-declared invariants prior to execution.
- **Sandboxing:** run untrusted code and resource-heavy tools within containerized, time-boxed environments to limit side-effects.
- **Post-condition validation:** verify result types and ranges before merging outputs into agent contexts; if validation fails, flag to verifier agents or request agent clarification.

AutoGen documents practical examples where tool outputs are immediately embedded into the conversation and re-tokenized for subsequent reasoning (e.g., code execution results, database query responses). The framework supports serializing tool outputs both as human-readable summaries and as vector embeddings for retrieval.

Memory architecture and retrieval

AutoGen adopts a multi-tier memory model:

Episodic memory: the raw conversation log (recent n turns). Used for short-term coherence and context.

Semantic summaries: periodic LLM-produced summaries that compress long histories into concise artifacts for long-horizon tasks.

Indexed semantic store: vector embeddings of important artifacts (summaries, tool outputs, factual assertions) stored in an ANN index (approximate nearest neighbor) for retrieval-augmented prompting.

Retrieval forms an explicit stage in prompt construction. Given a retrieval query derived from current goal g, the CM performs:

$$M_{\text{ret}} = \text{TopK}_{\text{ANN}}\big(\text{embed}(q(g)),\ k\big)$$

and inserts the canonicalized textual summaries of M_{ret} into the agent prompt. The report emphasizes cost-aware retrieval: selecting top-k that maximize relevance while satisfying token budget constraints (e.g., prefer compressed summaries over raw long dialogues). AutoGen also supports retention policies and scheduled summarization routines to limit unbounded growth of stored artifacts. [25, 27]

AutoGen formalizes a conversation-first programming model that maps naturally to contemporary LLM capabilities: it treats agents as role-templated LLM instances with controlled tool access, it enforces schema validation and sandboxed tool execution for safety, and it places memory retrieval and cost-aware prompt assembly at the heart of the orchestration pipeline. The framework's modular abstractions and rich developer tooling have enabled reproducible research on agentic workflows while illuminating practical tradeoffs (cost, latency, hallucination) that remain active areas of investigation.

2.5 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a hybrid framework in natural language processing that integrates external information retrieval with large pretrained generative models, enabling the model to dynamically fetch relevant documents from a knowledge corpus and condition its responses on that retrieved content [17]. This approach was introduced to address limitations of conventional generative language models, such as static knowledge confined to training data, difficulty in updating world information, lack of provenance for generated facts, and poor performance on knowledge-intensive tasks [17]. By combining parametric memory (the generative model) with non-parametric memory (an external retriever and index), RAG improves factual accuracy, reduces hallucinations, and allows the model to provide more contextually informed outputs, making it particularly effective for tasks such as open-domain question answering, summarization, and knowledge-grounded dialogue [18, 19].

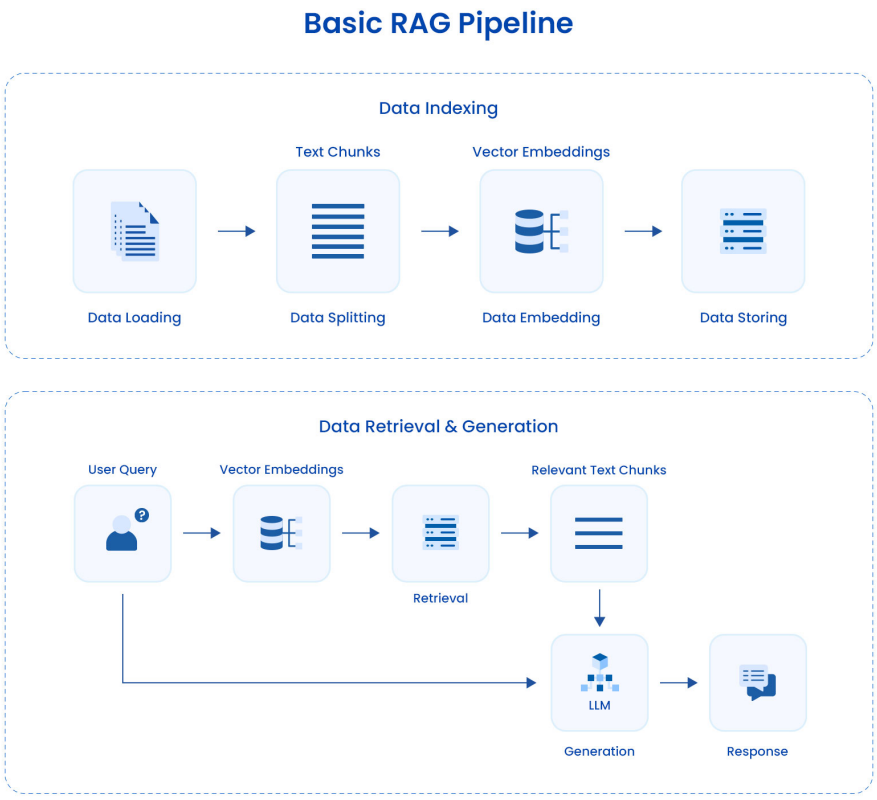


Figure 2.17: Simple Retrieval-Augmented Generation Pipeline

2.6 Text to Schema

The task of translating natural-language requirement descriptions into a logical relational schema differs fundamentally from the more widely studied problem of natural-language-to-query translation (Text→SQL). In the Text→SQL literature, the database schema is typically assumed to be predefined, stable, and correct; learning and inference methods are therefore explicitly conditioned on existing table structures, column names, and foreign-key relationships when mapping user utterances to executable queries. By contrast, the Text→Schema problem requires the schema itself to be constructed from free-form or semi-structured textual specifications, including the identification of relations, attributes, primary and foreign keys, and relevant domain constraints. This upstream formulation shifts the research focus away from query semantic parsing alone and toward long-standing database design questions that traditionally require expert human judgment.

From a research perspective, Text→Schema integrates concerns from natural language understanding, requirements engineering, and database theory. The task encompasses not only syntactic interpretation of text but also semantic reasoning about domain concepts, data dependencies, and integrity constraints. Decisions made during schema construction—such as entity granularity, key selection, relationship modeling, and normalization level—have enduring

consequences for query expressiveness, integrity enforcement, and system performance. Consequently, Text→Schema cannot be reduced to a simple extension of Text→SQL; instead, it represents a distinct problem space requiring dedicated models, representations, and evaluation methodologies [28].

Formally, the Text→Schema task may be characterized as the generation of a normalized logical schema, commonly targeting third normal form (3NF), together with explicit key and referential constraints, given a natural-language description of requirements and optionally limited auxiliary information such as naming conventions or example records. This formulation highlights several methodological challenges that are peripheral in Text→SQL research but central here: robust extraction of domain entities and inter-entity relationships from ambiguous prose; inference of functional dependencies and candidate keys in the absence of instance data; disambiguation of cardinalities and participation constraints; and formal representation of implicit domain rules such as value ranges, enumerations, and optional attributes. Addressing these challenges transforms schema generation into a multi-faceted research problem that intersects database design theory, knowledge representation, and natural language processing [28, 15, 16].

The classical roots: relational model, ER modelling, normalization & functional dependencies

Relational model and normalization. Relational theory provides the conceptual and mathematical foundations for the Text→Schema problem. The relational model formalizes data as relations composed of tuples and attributes, while normalization theory defines a hierarchy of normal forms designed to reduce redundancy and prevent update anomalies [29]. Properties such as lossless-join decomposition and dependency preservation provide formal criteria for assessing schema correctness. Any automated schema-generation approach must therefore grapple with these theoretical constraints, ensuring that generated schemas satisfy well-established principles of relational design rather than merely reflecting surface-level linguistic structure.

The database literature further emphasizes that normalization decisions involve trade-offs between theoretical purity and practical performance considerations. While higher normal forms reduce redundancy, they may introduce additional joins that impact query efficiency, particularly in analytical or latency-sensitive workloads. As a result, schema design is widely regarded as a balance between formal dependency reasoning and anticipated usage patterns, a balance that automated systems must approximate when operating solely on textual requirements [30].

Entity–Relationship modelling and conceptual design. The Entity–Relationship (ER) model established the dominant conceptual abstraction used in database engineering to bridge informal domain understanding and formal logical schemas [31]. By representing entities, relationships, attributes, and cardinalities, ER modeling enables designers to reason about domain structure independently of implementation details. Empirical and methodological

studies in conceptual modeling consistently demonstrate that inaccuracies at this stage—such as misidentified entities, incorrect relationship arities, or missing associative entities—propagate into flawed logical schemas and integrity violations.

Automated conceptual modeling from text has therefore long been recognized as a critical but challenging research problem. Linguistic heuristics, pattern-based extraction, and ontology-assisted methods have all been explored to derive ER-style representations from requirements documents. However, the literature also emphasizes that free-form natural language often underspecifies cardinalities and constraints, necessitating either conservative assumptions or human validation. These findings underscore the importance of robust conceptual extraction as a prerequisite for reliable Text→Schema systems [31].

Functional dependencies and inference rules. Functional dependencies (FDs) form the theoretical basis of normalization and schema decomposition. Armstrong's axioms provide a sound and complete inference system for reasoning about FDs and computing dependency closures. Building on this foundation, extensive research has focused on discovering FDs automatically from relational data. Algorithms such as TANE and its successors identify exact or approximate dependencies by analyzing value distributions and equivalence classes within datasets [32].

Despite their effectiveness, these data-driven FD discovery techniques presuppose the availability of representative instances, an assumption that does not hold in early-stage schema design driven by textual requirements. Consequently, while classical FD theory supplies formal tools for normalization and correctness verification, it does not directly address the problem of inferring semantic dependencies from natural language descriptions. This limitation has been repeatedly acknowledged in the database literature and remains a central challenge for automated schema generation in text-only settings [32].

2.6.1 Implications for automated schema generation

Taken together, the classical literature implies two core requirements for any automated Text→Schema approach. First, systems must reliably extract conceptual structure from natural language, including entities, relationships, attributes, and cardinality constraints, despite the inherent ambiguity and incompleteness of textual requirements. Second, they must obtain or hypothesize functional dependencies and keys sufficient to support normalization, either through auxiliary data, domain knowledge, or interaction with stakeholders.

In the absence of annotated schemas or instance data, automated approaches typically adopt pragmatic strategies documented in the requirements-engineering and database-design literature: eliciting additional specification through clarification, applying conservative default modeling patterns (such as surrogate keys or minimal dependency assumptions), or leveraging external

knowledge sources to propose plausible dependency structures. Each strategy involves trade-offs between automation, correctness, and the risk of introducing unfounded assumptions. Consequently, prior research consistently emphasizes the importance of iterative refinement and human-in-the-loop validation when moving from informal requirements to formal database schemas [29, 31, 32, 33].

2.6.2 Automated conceptual modelling from requirements text

Automated derivation of conceptual representations (entities, relationships, attributes, cardinalities) from natural-language requirements has been an active strand of research for several decades. Early approaches relied on linguistic heuristics and rule-based pipelines that map surface syntactic patterns (noun phrases, possessives, subject–verb–object triples) to candidate entity and relation hypotheses. These methods demonstrated that a non-trivial portion of conceptual structure can be recovered automatically from semi-structured requirements, but evaluations repeatedly highlighted their sensitivity to textual style and vocabulary variation: extraction quality tends to degrade on unrestricted prose, and the hardest tasks are identification of associative entities, optionality, and accurate cardinality assignment [34, 35].

Subsequent work integrated richer resources and methods to improve robustness. Ontology-assisted methods and statistical co-occurrence techniques were explored to provide semantic grounding for candidate concepts and to resolve certain types of lexical ambiguity (synonymy and polysemy). For example, ontology-alignment techniques can map extracted terms to standardized concepts and help identify hierarchical relations that inform attribute grouping and inheritance modelling. Empirical studies that combined pattern-based extraction with domain ontologies or lexicons reported greater precision in entity recognition and improved ability to detect common attribute groupings, but they still relied on human validation for constraints and cardinalities in most evaluated cases [35, 34].

A recurring finding in the conceptual-modelling literature is that free-text requirements tend to under-specify structural constraints: many requirements documents omit explicit statements of keys, cardinalities or participation constraints, either deliberately (to avoid over-commitment) or inadvertently. Consequently, automated systems face the dual challenge of (i) extracting explicit structure where present, and (ii) gracefully handling under-specification through conservative defaults, clarification dialogues, or by leveraging external knowledge sources to hypothesize plausible constraints. Several works in requirements engineering emphasize interactive elicitation (questionnaires, scenario probing, and stakeholder interviews) as a pragmatic complement to automation, pointing to hybrid workflows where an automated extractor proposes a draft conceptual model which is iteratively refined with human input [36].

2.6.3 Recent advances: large language models and schema-related tasks

The emergence of large pretrained language models (LLMs) has shifted the landscape of research on text-driven structure inference. LLMs exhibit broad world knowledge and strong pattern completion abilities that can be harnessed to propose schema elements, name conventions, and even likely functional dependencies purely from natural-language descriptions. Early empirical work applying LLMs to schema-centric tasks includes studies of schema matching and schema transformation where pretrained models were used to align heterogeneous schemas or to generate structural proposals for downstream tasks; these studies report promising results but also emphasize variability and error modes that require verification [37, 38].

Complementary lines of research have examined the use of LLMs as components in multi-step or multi-agent pipelines for complex tasks. Architectures that decompose problems into specialized subtasks (analysis, drafting, review, testing) and assign distinct LLM actors to each subtask have shown empirical benefits in reducing cascading errors and enabling targeted verification steps [23, 39]. Related research on tool use and agentic behavior (e.g., Toolformer, ReAct) demonstrates that models can be prompted or trained to invoke external tools, perform iterative reasoning with environment interactions, and produce structured outputs such as code or DDL when given suitable prompts and scaffolding [18, 40]. While these advances do not solve the conceptual and dependency-inference challenges identified earlier, they provide new mechanisms for integrating external knowledge sources, enacting clarification dialogues, and encoding multi-step verification into automated pipelines [18, 23].

As a concrete example, experimental studies on schema matching show that LLMs can often propose plausible correspondences between differently named attributes or suggest sensible normalizations, but that their outputs require post-hoc validation and constrained generation to ensure referential integrity and formal correctness [37]. Similarly, pilot projects that apply LLM-based prompts to generate relational DDL from textual descriptions report useful first-draft schemas that accelerate human designers, while also highlighting hallucinations and inconsistent key choices when the model is not constrained or validated.

2.6.4 Large language models and multi-agent systems

Recent rapid advances in large pretrained language models (LLMs) have created new opportunities for tasks that require broad semantic knowledge and flexible pattern induction. LLMs have been shown to perform remarkably on tasks such as code generation, structured-data synthesis, and schema-related operations (e.g., schema matching and attribute alignment) when given appropriate prompts and context. Empirical studies indicate that pretrained LLMs can often propose plausible table/column names, suggest normalization splits, and hypothesize candidate constraints based on common-sense and domain knowledge encoded during pretraining [37, 38].

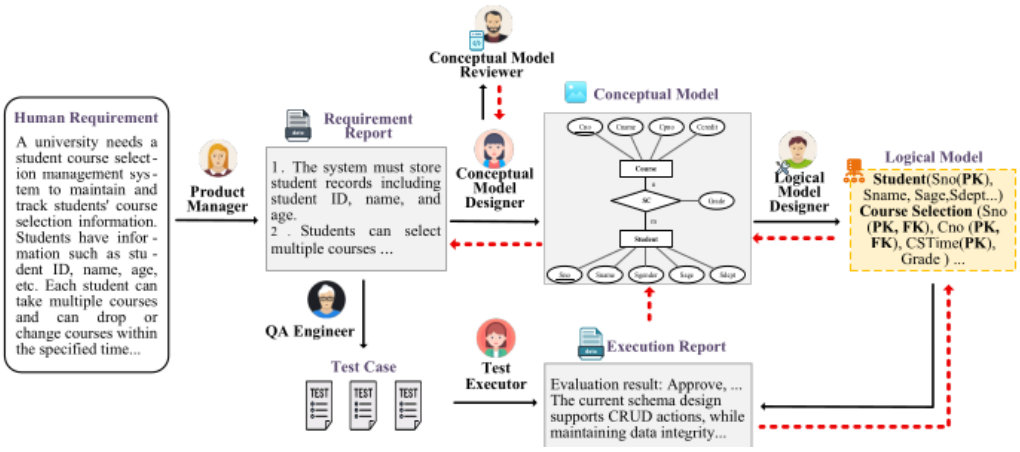


Figure 2.18: SchemaAgent Design

Concurrently, a growing body of research investigates multi-agent and agentic architectures that organize LLMs into specialized subcomponents (analysis, drafting, reviewing, testing) or allow them to call external tools in a controlled manner. Works demonstrating structured LLM tool use and agentic reasoning patterns (e.g., ReAct-style reasoning and acting [19], or multi-agent conversation frameworks [23]) show that decomposing complex problems into distinct reasoning roles and supplying tool affordances (execution, retrieval, or validation) reduces certain classes of errors and can improve overall reliability for multi-step tasks. For example, LLMs augmented with tool invocation capabilities can check whether generated SQL/DDL executes cleanly in a sandbox and can use the execution feedback to revise outputs [18, 39].

Building on this line of research, Wang et al. [28] proposed SchemaAgent, a multi-agent framework specifically designed for automated relational database schema generation. SchemaAgent assigns specialized roles to agents corresponding to the phases of schema design (requirement analysis, conceptual modeling, logical modeling), and incorporates dedicated reviewer and QA/test executor roles to detect and correct errors throughout the workflow. This controlled role assignment, combined with iterative feedback loops, reduces error propagation, improves schema quality, and ensures adherence to relational constraints. Empirical evaluation on the RSchema benchmark demonstrates that the multi-agent approach significantly outperforms direct prompting or chain-of-thought methods across multiple LLMs, highlighting the potential of agentic LLM systems to automate complex, knowledge-intensive tasks while maintaining reliability. These studies collectively show that while agentic and tool-augmented paradigms do not eliminate the inherent challenges of semantic dependency inference from text, they offer practical mechanisms for error localization, lightweight verification, and iterative refinement, which are crucial for high-integrity schema generation [28, 37].

2.6.5 Evaluation challenges in schema generation

Evaluating automatically generated schemas presents conceptual and practical challenges that have been recognized across database, ontology, and conceptual-modelling literatures. Unlike query-generation tasks where execution provides an objective correctness signal, schema design admits multiple valid representations for the same requirements. Design choices such as surrogate versus natural keys, degree of normalization, whether to introduce associative entities, and denormalization for performance produce a *space of acceptable schemas* rather than a single ground-truth. This multiplicity complicates the construction of canonical benchmarks and demands granular evaluation strategies.

The research community has therefore adopted mixed evaluation regimes. Component-level metrics (entity and attribute recall/precision, key detection rates, foreign-key recovery) enable objective comparisons of discrete aspects of schema proposals. Human expert assessments remain indispensable for judging global design quality, trade-offs, and workload-driven decisions. In addition, task-oriented evaluation (measuring how well generated schemas support downstream queries, analytics, or API needs) provides pragmatic evidence of utility even when structural mismatches exist between automatic outputs and a particular gold standard [28, 41, 42].

Recent benchmark creation efforts for requirement→schema tasks respond directly to these issues by decomposing evaluation into measurable components and by proposing adjudication procedures for cases where multiple semantically-equivalent schemas exist. Still, the literature acknowledges that no single automatic metric captures all aspects of design quality and that standardized protocols for multi-gold annotation, expert adjudication, and downstream task evaluation are necessary to enable reproducible research in Text→Schema [28].

Manually evaluating each schema is prohibitively time-consuming and makes it difficult to quantitatively assess model performance. Consequently, SchemaAgent introduces an automated evaluation approach using manually annotated schemas in RSchema as the ground truth. Each schema undergoes multiple rounds of evaluation to ensure correctness and quality. A schema consists of five key components: relation schema (table), attribute, primary key, foreign key, and constraint. The evaluation metrics include average F1 score and exact matching accuracy (Acc.) between predicted and ground truth values. Accuracy is set to 1 only if the F1 score equals 1; otherwise, it is 0.

Relation schema (Table). Since generative models may produce different tokens with similar meanings, three alignment methods are used to match predicted schema names with ground truth:

1. *Synonym Matching:* WordNet is used to extract synonyms of predicted values and verify inclusion of the ground-truth value.
2. *Similarity Matching:* all-MiniLM-L6-v2 measures semantic similarity with a threshold $\delta_0 = 0.6$.

3. *String Matching:* The longest common substring between predicted and ground-truth values is checked against $\delta_1 = 0.75$.

For a schema with multiple tables, the table-level F1 score is computed as:

$$F1_{\text{table}} = 2 \times \frac{P \times R}{P + R} \tag{2.6.1}$$

where

$$P = \frac{|R_{\text{gt}} \cap R_{\text{pred}}|}{|R_{\text{pred}}|}, \quad R = \frac{|R_{\text{gt}} \cap R_{\text{pred}}|}{|R_{\text{gt}}|} \tag{2.6.2}$$

R_{gt} is the set of relation schemas in the ground truth, and R_{pred} is the predicted set. The average error rate of this metric is 1.8%, demonstrating reliability compared with manual evaluation.

Attributes. The same three alignment methods are applied to attribute sets in matched tables. Attribute F1 is computed as:

$$F1_{\text{attrs}} = \frac{1}{|R_{\text{gt}}|} \sum_{R_{\text{gt},i} \in R_{\text{gt}}} F1_{\text{attrs}}(R_{\text{gt},i}) \tag{2.6.3}$$

where $F1_{\text{attrs}}(R_{\text{gt},i})$ compares the attributes of the i-th ground-truth table with the predicted table.

Keys Primary and foreign keys require exact matching, with Acc. set to 1 only when predicted and golden sets are identical. Data type accuracy is computed only for successfully matched attributes.

These metrics, combining semantic alignment and exact matching, enable automated, reproducible assessment of generated schemas. Manual evaluation of 20 randomly selected schemas (over 100 tables) confirms that automated metrics are consistent with expert judgment, providing a reliable quantitative signal for model performance.

2.6.6 Synthesis and open directions

Collectively, the prior literature indicates that Text→Schema is a challenging but tractable research direction that benefits from hybrid approaches. Core open problems include: (1) principled methods to infer functional dependencies from text or small-sample exemplars; (2) robust disambiguation strategies for cardinalities and participation constraints; (3) workload-aware normalization that integrates expected query patterns into design decisions; and (4) standardized evaluation frameworks that accommodate multiple valid schemas. Promising methodological building blocks include classical FD theory and instance-aware FD miners, semantic grounding via ontologies or schema corpora, and the new generation of LLM-based tools and multi-agent frameworks for hypothesis generation and verification [43, 32, 35, 37, 28].

2.7 Text to SQL

2.7.1 Foundational Evolution of Natural Language to SQL Parsing

The translation of natural language queries (NL) into syntactically valid and semantically accurate Structured Query Language (SQL) expressions—historically formalized as semantic parsing—has undergone distinct evolutionary paradigms [44, 45]. Early technical approaches relied entirely on rule-based statistical semantic parsers and deterministic translation patterns [45]. These frameworks evaluated input streams at the token level, mapping linguistic inputs directly onto rigid, single-table target database schemas via hardcoded template sketches or syntax maps [45]. While computationally lightweight, these early systems lacked scalability and generalizability, rendering them highly fragile when exposed to natural language synonyms, lexical variations, or modified structural schemas [45].

The field shifted significantly with the implementation of deep neural networks, specifically sequence-to-sequence (Seq2Seq) neural architectures and Graph Neural Networks (GNNs) [45]. These neural parsers substantially enhanced natural language intent understanding and vocabulary variation handling [45]. To preserve structural validity during the generation phase, these systems began utilizing Abstract Syntax Trees (AST) and structural decoders to ensure that the output adhered to standard database syntax [44, 45].

The introduction of Pre-trained Language Models (PLMs) like BERT and T5 established the next major milestone, drastically improving schema alignment and intent parsing [45]. PLM-driven models achieved approximately 80% accuracy on standard benchmarks like Spider 1.0 [45]. However, their performance deteriorated rapidly when exposed to extra-hard query complexities or highly nested schema states [45].

In the contemporary era of Large Language Models (LLMs), the task has been recast as an adaptive generation and in-context execution problem [44, 45]. Modern decoder-only and encoder-decoder architectures (e.g., GPT-4, Claude 3.5, and DeepSeek) leverage extensive parameters and emergence capabilities to analyze natural language logic and database relations directly within the prompt context window, eliminating the need for rigid structural sketches or decoding constraints [44, 45].

2.7.2 The Modular Pipeline Taxonomy in the LLM Era

While classical Text-to-SQL translation was long treated as an end-to-end monolithic generation task, recent advancements have enforced a highly structured, modular design pattern [45]. Modern systems segment the Text-to-SQL lifecycle into three explicit execution phases: Pre-Processing, Core Translation, and Post-Processing [45].

Pre-Processing Strategies

The pre-processing stage is responsible for minimizing information noise and augmenting input text before it reaches the language model backbone [45]. This phase consists of three core operational modules:

- **Schema Linking:** This module isolates and maps specific natural language tokens to their corresponding structural items (tables and columns) within the database [45]. In early systems, this was achieved via exact or approximate string matching [45]. Modern systems deploy deep cross-encoders or specialized In-Context Learning (ICL) routines to dynamically score and rank structural elements by relevance, mitigating cross-domain vocabulary alignment gaps [45].
- **Database Content Retrieval:** To satisfy literal filter constraints embedded in `WHERE` or `HAVING` clauses, systems must extract precise cell values from the physical database [45]. Because injecting whole database files into LLM prompts is structurally impossible, frameworks construct complex indexing and retrieval strategies [45].
- **Additional Information Acquisition:** This module infuses external domain knowledge, formulaic definitions, metadata descriptions, and format constraints into the generation template [45]. This step prevents error propagation caused by a model's unfamiliarity with specialized corporate jargon or cross-domain rules [45].

Core Translation Frameworks

The core translation layer handles the conversion of structured semantic contexts into executable relational code, defined by specific encoding and decoding design choices [45]:

- **Encoding Strategies:** Input serialization generally relies on *Sequential Encoding*, where language queries, schema metadata, and inline comments are flattened into a single linear string [45]. Alternatively, *Graph-Based Encoding* constructs highly interconnected graph abstractions that capture the physical database topology (e.g., primary-to-foreign key relationships), mapping them using Graph Attention Networks to explicitly preserve the schema design [45].
- **Decoding Strategies:** Standard implementations leverage *Greedy Search* or *Beam Search* decoding paths [45]. However, advanced pipelines use *Constraint-Aware Incremental Decoding* frameworks, such as PICARD [45]. These systems intercept token generation at each step, verifying the partial statement against the formal SQL grammar rules and schema-level constraints to prevent structural or syntactical errors before execution [45].

- **Intermediate Representations (IR):** To mitigate the wide structural gap between unconstrained natural language and rigid SQL, architectures can translate queries into a grammar-free intermediate language [45]. Frameworks like NatSQL remove complex syntax components (such as implicit `FROM` and `JOIN` tables) to yield a simplified abstraction layer, which can later be deterministically expanded into database-specific SQL [45].

Post-Processing and Self-Correction

Once an initial SQL statement is output by the model, post-processing layers are activated to check validity and execute refinement loops [45]:

- **Output Consistency and Voting:** To minimize model randomness, pipelines sample multiple execution paths simultaneously [45]. Frameworks use self-consistency voting or keyword clustering methods to select the most consistent candidate from the parallel generation runs [45].
- **Execution-Guided Self-Correction:** This technique uses real-time runtime feedback to execute automated debugging loops [45]. By capturing execution states, syntax warnings, or empty result logs directly from the database server engine, the pipeline passes the raw error text back into specialized refinement blocks, mimicking human debugging processes [45].

2.7.3 Multi-Agent Collaboration and Role-Based Delegation

The limits of monolithic, single-agent pipelines when exposed to massive enterprise schemas have driven the development of multi-agent collaborative frameworks [44, 46, 45]. Rather than forcing a single LLM configuration to process the entire database state, reason through multi-hop questions, and write hundreds of lines of dialect code, these frameworks assign specialized roles to distinct agents that collaborate via shared memory contexts [44, 46, 45].

The Schema Linker (Selector Agent)

The Selector agent handles the task of schema linking and prompt optimization [44, 45]. When exposed to real-world industrial environments containing hundreds of tables and thousands of columns, injecting the complete data description creates severe information interference, misleads column matching, and incurs unsustainable API token costs [44, 45]. The Selector isolates a minimal relevant sub-schema based on semantic similarity to the user's question [44, 45]. It drops irrelevant tables and retains a strictly bounded column footprint per table—ensuring that the subsequent generation layers receive a clean, high-density context prompt [44].

The Logic Decomposer

For complex analytical questions requiring nested subqueries, common table expressions (CTEs), or multi-table joins, single-pass generation frequently falls into reasoning traps [44, 47, 45]. The Decomposer agent handles task breakdown by fracturing the natural language question into a sequence of simpler, executable sub-questions [44, 45]. Operating via sequential Chain-of-Thought (CoT) logic, it builds sub-SQL components progressively [44, 45]. The execution output or logical structure of an early subquery serves as the structural foundation for the next analytical block, ensuring that each component is validated before final assembly [44].

The Semantic Reviewer and Refiner

A critical vulnerability in modern implementations is the semantic consistency gap: models frequently output syntactically clean SQL queries that execute perfectly without throwing database errors, but return incorrect data because the query logic does not align with the user's actual intent [46, 45]. To address this, current research integrates a dual-agent check system:

The SQLReviewer agent is engineered for latent semantic error detection [46]. It adopts the software engineering principle of *Rubber Duck Debugging*, systematically parsing and explaining the generated SQL code block line-by-line to evaluate whether the joins, selections, and filters precisely match the user's intent [46]. If a logical conflict or execution crash is flagged, the query is passed to the SQLRefiner agent [46]. The Refiner acts as an automated debugging engine, utilizing failure memory reflection and logging historical execution history to iteratively correct the SQL statement until it successfully passes through both validation gates [46, 45].

2.7.4 Architectural Calibration: Fine-Tuning vs. In-Context Learning

Deploying Text-to-SQL capabilities within corporate settings requires careful optimization across training and inference dimensions [45, 48]. Recent engineering studies from 2025 highlight the major performance and structural trade-offs between zero-shot/few-shot In-Context Learning (ICL) and domain-specific Supervised Fine-Tuning (SFT) [45, 48].

Evaluation Benchmarks and Metrics

System calibration and model accuracy are measured using two standardized evaluation parameters [45, 48]:

1. **Valid Efficiency Score (VES):** Evaluates the computational efficiency of the generated SQL queries. Unlike basic accuracy metrics, VES measures the execution time and resource

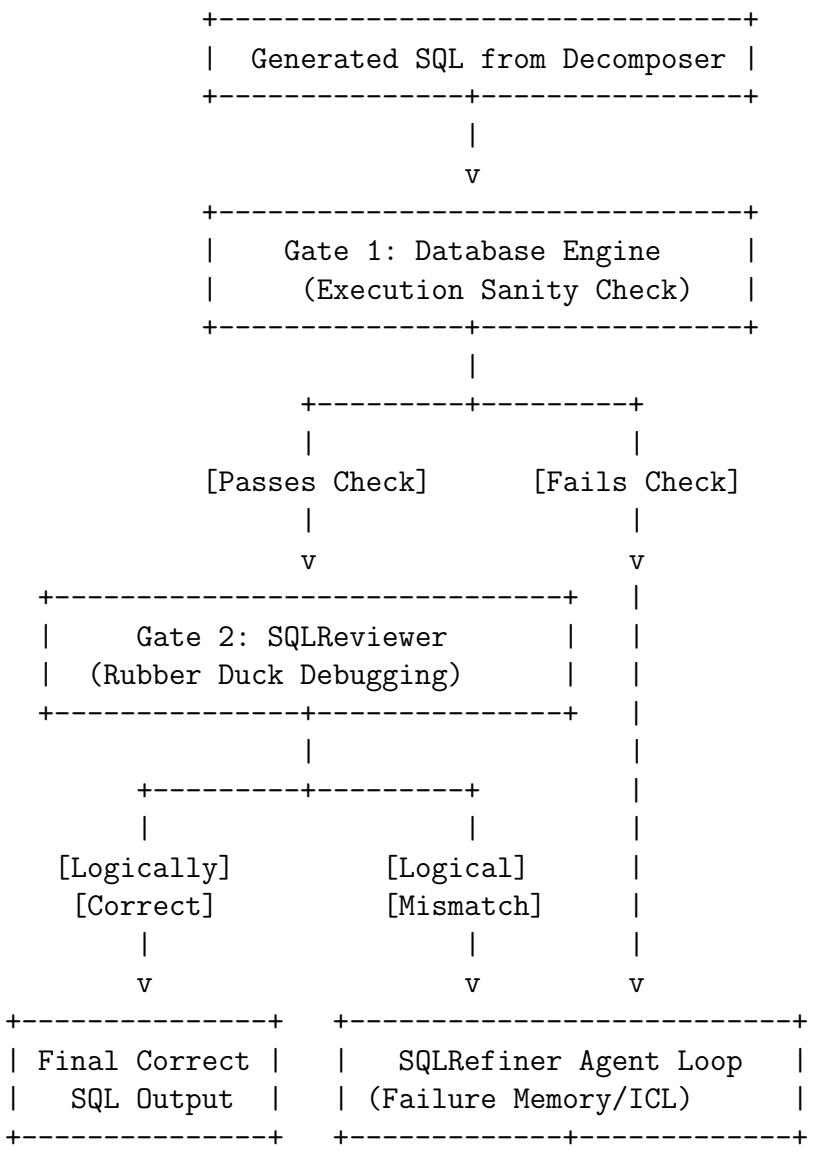


Figure 2.19: Dual-Gate Verification and Refinement Flow

optimization of the model-generated query relative to the ground-truth SQL. This metric ensures that the system produces not only semantically correct but also highly performant code suitable for massive production environments [45, 48].

2. **Execution Accuracy (EX):** Measures the precise proportion of generated queries that return a data result identical to the ground-truth query when executed against the physical target database [45, 48].

Trade-offs and the "Inverse-Skilled Bias"

Empirical model evaluations establish distinct trade-offs between closed-source API prompting and local fine-tuning [45, 48]:

Table 2.3: Strategic Trade-offs between ICL and SFT

Optimization Strategy	Pros	Cons	Data Privacy	Resource Requirements
In-Context Learning (ICL) *(e.g., GPT-4, Claude 3.5)*	Zero retraining time; rapid adaptation via prompt changes; elite general reasoning.	High token consumption; volatile prompt sensitivity; extreme API costs at scale.	Low (Exposes queries and metadata to external servers).	High API budget; zero local hardware overhead.
Supervised Fine-Tuning (SFT) *(e.g., Specialized open models)*	Captures deep structural domain context; stable schema mapping; near-zero prompt overhead.	Requires high-quality, human-curated training pairs; risk of overfitting baseline text patterns.	High (Enables fully air-gapped, local deployment).	Local GPU clusters required for initial parameter calibration.

2.7.5 Frontier Challenges: Real-World Enterprise Workflows

While traditional models achieved high scores on classic benchmarks like Spider 1.0 (91.2%) and BIRD (73.0%), recent 2025 research reveals that these platforms struggle when deployed within true corporate production environments [47]. The introduction of the **Spider 2.0** benchmark highlights a massive complexity gap between academic configurations and industrial database operations [47].

Real-World Schema and Architecture Traits

Production-grade systems differ from academic benchmarks across several key areas [47]:

- **Scale and Scope:** Production databases move away from small, clean, local tables, transitioning to terabyte-scale cloud data lakes (e.g., Google BigQuery, Snowflake) characterized by massive schemas that frequently contain over 1,000 columns across nested relational structures [47].
- **Nested Records and Unstructured Fields:** Industrial systems frequently abandon flat tables, storing complex data in nested `STRUCT` arrays, spatial geography points, or loose `JSON` objects directly within a single column [47]. Models cannot resolve these schemas via standard column linking, requiring multi-step extraction reasoning and specialized dialect function mapping [47].
- **Dialect Divergence:** Production applications require navigating subtle, platform-specific syntax nuances across diverse database engines—including SQLite, DuckDB, PostgreSQL, ClickHouse, and Snowflake [47]. This variance demands that agents dynamically query

platform documentation to extract specific analytical functions (e.g., `DATE_TRUNC` or `ST_DISTANCE`) to match regional data constraints [47].

Chapter 3: System Analysis

3.1 Problem Description & Analysis

The rapid evolution of software development has placed data at the core of modern applications, necessitating robust Database Management Systems (DBMS). However, the current landscape presents significant friction for developers, particularly students and early-stage engineers. The problem can be decomposed into five distinct areas: workflow fragmentation, high learning barriers, lack of intelligent assistance, infrastructure overhead, and vendor lock-in risks.

3.1.1 Fragmentation of the Development Ecosystem

A primary challenge facing developers is the diversity of database technologies. Modern applications often require a mix of Relational (SQL) databases such as PostgreSQL and NoSQL databases such as MongoDB to handle different data structures. Managing these disparate systems forces developers to rely on a fragmented ecosystem of standalone tools.

- **Context Switching:** Developers must frequently switch between different environments for schema design, query execution, data manipulation, and cloud hosting.
- **Tool Isolation:** SQL and NoSQL databases typically possess unique configurations, syntaxes, and interfaces that do not interoperate.
- **Operational Inefficiency:** This lack of integration introduces unnecessary overhead, slows development cycles, and increases the likelihood of inconsistencies and errors. Current cloud offerings focus primarily on hosting rather than providing a fully integrated development workspace that combines schema visualization and query editing in one cohesive environment.

3.1.2 High Learning Curve and Complexity

For students and entry-level developers, the complexity of traditional database management is a significant barrier to entry.

- **Configuration Burden:** Setting up and maintaining database environments requires substantial technical knowledge regarding installation, configuration, and resource management.
- **Syntax & Structure:** Beginners often struggle to understand complex schema relationships and query syntaxes without intuitive, visual aids. Existing tools often fail to provide integrated support for schema visualization or semi-automated schema generation, forcing users to invest substantial time just to operate the system.

3.1.3 Absence of Context-Aware AI Assistance

While Artificial Intelligence has transformed many aspects of software engineering, it remains underutilized in database management tools.

- **Manual Query Formulation:** Developers frequently waste time writing repetitive queries or attempting to decipher complex joins and aggregation pipelines manually.
- **LLM Agent-Powered Schema Generation:** Existing platforms rarely leverage autonomous LLM agents to analyze database content and generate or infer schema structures automatically. Such agents can interpret natural language requests (e.g., "show customers who purchased X") and translate them into structured queries, provide schema visualizations, and suggest optimized relationships between tables or collections.
- **Schema Understanding:** Developers dealing with unfamiliar or large datasets often lack intelligent tools to explain the relationships between tables or collections. LLM agents can act as interactive assistants, generating schemas, highlighting dependencies, and providing explanations for complex structures, reducing cognitive load.

3.1.4 Infrastructure Management and Cost Overhead

Traditional on-premise database management imposes heavy burdens regarding Capital Expenditure (CAPEX) and specialized labor.

- **Operational Burden:** Managing database infrastructure requires significant investment in hardware, software licensing, and specialized staff for maintenance, backups, and patching.
- **Resource Inefficiency:** Traditional models often lead to over-provisioning, where organizations purchase excess capacity for hypothetical future needs, resulting in waste.
- **Complexity of Scale:** As data grows, manual scaling becomes complex and error-prone. While DBaaS mitigates this by converting CAPEX to OPEX, setup on hyperscale providers can still be expensive and difficult for Small and Medium Enterprises (SMEs) to configure correctly.

3.1.5 Vendor Lock-In and Deployment Rigidity

Reliance on major hyperscale providers such as AWS, Azure, and Google Cloud introduces strategic risks.

- **Ecosystem Traps:** Hyperscalers often design services as walled gardens, making it difficult to migrate workloads or data due to proprietary APIs and high egress fees.
- **Lack of Portability:** Deep integration with a specific vendor ecosystem restricts organizational flexibility.
- **Cost Unpredictability:** Complex pricing models and hidden fees make monthly IT budgeting difficult.
- **Multi-Cloud Need:** There is growing demand for cloud-agnostic architectures using containerization technologies such as Docker and Kubernetes to avoid lock-in risks.

3.1.6 Technical Trade-offs: Virtualization vs. Containerization

From a system architecture perspective, hosting databases introduces technical trade-offs. Containers offer lightweight and fast deployment suitable for PaaS and SaaS platforms but pose challenges in I/O performance isolation compared to Virtual Machines (VMs). The system design must optimize database performance within containerized environments while maintaining reliability and deployment agility.

3.1.7 Summary of the Problem Gap

The market lacks a unified, AI-powered DBaaS platform that supports both SQL and NoSQL models while providing a low-code or no-code interface for schema design. Existing solutions such as AWS RDS, Supabase, and MongoDB Atlas typically specialize in a single database model or focus primarily on infrastructure rather than developer experience and AI-driven accessibility. This project addresses this gap by integrating LLM-agent-powered schema generation and visualization into a single, cohesive solution.

3.2 Requirement Engineering

The system requirements are divided into functional requirements, which define the specific behaviors and services the platform must provide, and non-functional requirements, which specify quality attributes such as scalability, portability, and security.

3.2.1 Functional Requirements

The platform is designed to eliminate tool fragmentation by consolidating essential database operations into a single unified lifecycle.

Unified Management (Multi-Model Support)

- The system shall provide a centralized dashboard capable of managing both Relational (SQL) and Non-Relational (NoSQL) databases simultaneously.
- The platform must support PostgreSQL for structured relational data and MongoDB for flexible, document-based data.
- **Justification:** This requirement supports a best-of-breed selection approach, recognizing that different workloads require different database engines (e.g., SQL for transactional integrity and NoSQL for flexible schemas) rather than a monolithic solution.

Schema Operations & Visual Design

- **Visual Builder:** The system shall include a drag-and-drop or interactive graphical interface for creating tables and defining entity relationships (ERD), abstracting complex syntax from the user.
- **File Import:** The system shall allow users to initialize projects by uploading existing CSV or JSON files, automatically inferring the initial schema structure.
- **LLM Agent-Powered Generation:** The platform shall implement an autonomous LLM agent module capable of converting natural language prompts (e.g., "Create a schema for an e-commerce store") into executable database structures, suggesting normalized relations, indexes, and validation rules, and producing visual ERD artifacts for user review.
- **Interactive Refinement:** The LLM agent shall support iterative, conversational refinement of generated schemas (user can accept, modify, or ask the agent to refactor), preserving audit logs of changes for reproducibility.

Data Manipulation & Query Execution

- **Table Interface:** The system shall provide a spreadsheet-like interface for performing CRUD (Create, Read, Update, Delete) operations without requiring manual query writing.
- **Unified Query Editor:** The system shall support execution of both SQL queries and NoSQL aggregation pipelines within a single integrated environment.
- **Agent-Assisted Query Generation:** An LLM agent shall translate user natural language requests into syntactically correct SQL or aggregation pipelines, explain query semantics, and propose optimized alternatives (e.g., index hints, projection reduction) when applicable.
- **Context:** These features directly address ecosystem fragmentation by eliminating the need for developers to switch between isolated tools for data entry and query execution.

AI Assistance (LLM Agent-Centric)

- **Conversational Assistant:** The system shall integrate a persistent LLM agent that is schema-aware (has canonicalized knowledge of the current project schema and metadata) and can answer questions, generate queries, and propose schema edits grounded in the actual database contents and structure.
- **Explainability & Traceability:** The LLM agent shall produce human-readable rationales for any generated schema change or query and shall attach links to the specific schema elements, sample rows, or tool outputs that informed its suggestion.
- **Multi-Agent Workflows:** For complex tasks (e.g., data migration, normalization, or cross-store joins), the platform shall support orchestrated LLM agents with specialized roles (planner, verifier, executor) to propose plans, validate transformations, and execute safe operations under policy constraints.
- **Human-in-the-Loop Controls:** The system shall allow users to require explicit approval before any destructive operation suggested by an agent (e.g., DROP, mass UPDATE), and record approvals in an auditable history.
- **Market Alignment:** This requirement aligns with the industry trend toward LLM-driven developer tooling while emphasizing agentic correctness, provenance, and human oversight rather than generic retrieval pipelines.

Cloud Storage & Versioning

- **State Management:** The system shall securely store database projects in the cloud and maintain version histories that enable rollback to previous states.
- **Export & Restore:** Users shall be able to export schemas and data or restore projects from saved versions to ensure data resilience and recovery.
- **Reliability:** Automated backup and restore mechanisms shall be implemented to reduce administrative overhead and improve service reliability.
- **Agent-Aware Checkpoints:** The platform shall capture agent proposals and transformations as part of version history (including agent prompts, outputs, and validation results) to enable reproducible rollbacks and post-hoc audits.

3.2.2 Non-Functional Requirements

These requirements ensure that the system remains scalable, portable, secure, and suitable for production deployment.

Scalability via Containerization

- **Architecture:** The system shall utilize Docker for containerization.
- **Resource Efficiency:** Containers shall share the host operating system kernel, reducing resource overhead and enabling faster startup times compared to Virtual Machines (VMs).
- **Justification:** Although VMs provide stronger isolation for I/O-intensive workloads, containerization is selected for its superior efficiency in PaaS and SaaS environments, enabling dynamic horizontal scaling.
- **Database Performance Considerations:** The system shall include configuration guidelines and optional node affinity/IO provisioning primitives (e.g., hostPath, CSI volumes with provisioned IOPS) to mitigate containerized I/O bottlenecks for production database workloads.

Flexibility (Cloud-Agnostic Design)

- **Vendor Lock-In Mitigation:** The system architecture shall be cloud-agnostic, supporting deployment across multiple providers (e.g., AWS, GCP, Azure).
- **Portability:** By relying on standards-based technologies such as containers and Kubernetes instead of proprietary cloud APIs, the system shall allow seamless workload migration with minimal refactoring.
- **Strategic Value:** This approach mitigates hyperscaler walled-garden strategies that rely on proprietary services and data egress fees.

Security & Monitoring

- **Access Control:** The system shall implement robust authentication and authorization mechanisms (e.g., OAuth/OIDC, RBAC) to manage user access to database resources.
- **Tenant Isolation:** Strong tenant isolation shall be enforced to prevent unauthorized access to sensitive data in multi-tenant environments (network policies, namespace separation, encrypted volumes).
- **Administrative Monitoring:** An administrative dashboard shall be provided to monitor system performance, user activity, tool/agent usage, and resource utilization to support compliance and proactive management.
- **Agent Safety Controls:** The platform shall enforce agent capability scoping (which agents can execute which tools), rate limits on agent-initiated operations, and verification layers (schema validators, transaction dry-runs) before executing critical changes.

- **Auditability:** All agent interactions, approvals, tool calls, and schema/data mutations shall be logged with cryptographic timestamps to support forensic analysis and regulatory audits.

3.3 Feasibility Study

The feasibility study evaluates the technical, economic, and operational viability of developing a custom AI-powered DBaaS platform. This analysis demonstrates that the proposed system addresses key market gaps while leveraging an architecture optimized for performance, scalability, and cost efficiency.

3.3.1 Technical Feasibility

The primary technical challenge of the proposed system lies in the efficient lifecycle management of database instances. Unlike conventional solutions that rely on heavyweight virtualization or external orchestration platforms such as Kubernetes, this project adopts a lightweight custom orchestration layer implemented in Golang.

Custom Orchestration Strategy (Golang)

- **Suitability for System Programming:** The backend is implemented in Golang due to its strong concurrency model based on goroutines and its suitability for system-level programming. This enables the orchestrator to efficiently solve the online resource allocation problem, dynamically placing database instances as tenants join or leave the system with minimal latency.
- **Direct Docker Integration:** The orchestrator interfaces directly with the Docker Engine API rather than introducing the abstraction overhead of Kubernetes. This approach reduces system complexity and resource consumption while enabling fine-grained control over container lifecycles (start, stop, restart) and resource constraints via cgroups.
- **Performance:** Empirical research shows that containerization introduces negligible overhead compared to virtual machines, achieving millisecond-level startup times. By avoiding hypervisor-related overhead, the system maximizes hardware utilization and responsiveness.

Database Engine Compatibility

- The platform supports PostgreSQL for relational workloads and MongoDB for document-oriented workloads. Studies indicate that PostgreSQL deployed in containerized environ-

ments can meet or exceed virtual machine deployments for many common operations such as INSERT and DELETE when I/O is provisioned appropriately.

- Although containerized databases share the host kernel, potentially leading to I/O contention in dense multi-tenant environments, this risk is mitigated through strict resource isolation policies enforced by the custom orchestrator (cgroup limits, CPU/memory quotas, I/O throttling), addressing the noisy-neighbor problem.

AI Integration (LLM Agent-Centric)

- **LLM Agent for Schema Generation and Assistance:** Rather than depending on a retrieval-augmented generation pipeline, the system employs persistent, schema-aware LLM agents that analyze project artifacts (uploaded CSV/JSON samples, existing schema metadata, example rows) to infer, generate, and iteratively refine database schemas. These agents output concrete DDL/DDL-like constructs (CREATE TABLE/collection, field types, indexes, constraints) and produce human-readable rationales and ERD visualizations for developer review.

- **Agent-Oriented Embedding and Local Indexing:** For internal use (e.g., fast lookup of previously generated schema fragments, example rows that support agent explanations), the system maintains localized embedding indices and metadata stores consumed by the agent pipeline; these indices are operational auxiliaries to the agents (not a global retrieval service) and are primarily used to ground agent proposals in concrete evidence from the user's project.

- **Planner/Verifier Agent Pattern:** Complex changes (schema migration, cross-store normalization) are executed via coordinated multi-agent workflows: a *planner agent* proposes a migration plan, a *verifier agent* simulates and validates transformations (dry-run against a sandbox), and an *executor agent* carries out approved steps under human supervision. This pattern reduces destructive errors and improves trustworthiness of automated transformations.

- **Operational Constraints and Safety:** Agents operate with strict capability scoping and policy constraints: they cannot perform destructive operations without explicit user approval; every proposed change is logged along with the agent prompt and evidence used for that decision to preserve auditability and reproducibility.

- **Feasibility and Integration:** Modern LLM APIs and on-premise model runtimes make these agentic workflows technically feasible; the system focuses on orchestration, schema grounding, and controlled execution rather than on generic large-scale document retrieval pipelines.

3.3.2 Economic Feasibility

The economic analysis highlights the cost efficiency of the proposed platform compared to hyperscale cloud DBaaS offerings.

Total Cost of Ownership (TCO) Reduction

- **Avoiding Hyperscaler Premiums:** Surveys indicate that a majority of organizations consider large public cloud providers costly, leading to unpredictable IT budgets. By adopting a cloud-agnostic design capable of running on bare metal or lower-cost infrastructure, the proposed system significantly reduces total ownership costs.
- **Resource Efficiency:** Traditional database deployments often suffer from over-provisioning. The proposed platform employs automated scaling managed by the custom Golang orchestrator, ensuring that resource usage closely matches actual demand and minimizing idle capacity costs.

Mitigation of Vendor Lock-In

- **High Switching Costs:** Hyperscale providers typically impose vendor lock-in through proprietary APIs and data egress fees.
- **Strategic Agility:** By relying on open container standards such as Docker and avoiding proprietary DBaaS APIs, the proposed platform eliminates vendor lock-in risks and ensures deployment flexibility across diverse infrastructure environments.

3.3.3 Operational and Market Feasibility

The operational and market feasibility analysis confirms strong demand for a unified, AI-driven DBaaS solution.

Market Growth

- The global cloud database market is projected to grow substantially over the coming decade, driven by increased digitalization and demand for scalable data management solutions. This growth indicates a substantial addressable market for new DBaaS platforms.

The Unified Tool Gap

- Developers currently operate within a fragmented ecosystem, switching between isolated tools for SQL and NoSQL database management. There is a clear absence of a unified platform that combines multi-model database support with an intuitive visual interface suitable for students and small-to-medium enterprises.

LLM Agents as Baseline Expectation

- The database market is transitioning toward AI-native systems where conversational, agent-driven schema generation, schema understanding, and model-assisted operations are expected features. Integrating LLM agents as first-class participants in the development lifecycle positions the platform competitively while preserving human oversight and auditability.

Reduced Administrative Burden

- The platform automates operational tasks such as backups, scaling, and patching, significantly reducing administrative overhead. Agentic assistance reduces developer time spent on schema design and query authoring, enabling users to focus on application logic rather than database maintenance.

3.4 System Modeling & Diagrams

3.4.1 Event Table

#	EVENT NAME	TRIGGER	SOURCE	USE CASE	RESPONSE	DEST.
1	Developer login to the system	• Developer's Info • Login request	Developer	Login Developer	• Show dashboard • Error if login fails	Developer

2	Developer Update profile	• Developer Id • Details info	Developer	Update profile	1. Updated profile	Developer
3	Developer creates a new project	• New Project Request • Details (name, DB)	Developer	Create a new Project	• Nav to Schema Tab • Save project records	Developer
4	Developer deletes project	1. Project Id	Developer	Delete Exist Project	1. Updated List of projects	Developer
5	Create project schema	• Schema Gen. request • Natural language	Developer	Create Schema	1. Visualize Generated Schema	Developer
6	Developer modifies Entities	• Entities info • Create/Edit Entities	Developer	Modify Entities	• Query response msg • Update Entity in DB	Developer

7	Developer executes query	• Query statement • Project Id	Developer	Execute Query	• Query response msg • Show results	Developer
8	Developer modifies data	• CRUD request • Project Id	Developer	CRUD operations	• Response message • Updated DB	Developer
9	Developer Requests Restore	1. Restore Request (Select Point-in-Time)	Developer	Restore Project	1. Restore requested backup	Developer
10	Developer Upgrades Tier	1. Subscription Change Request	Developer	Manage Subscription	1. Update Tenant Quotas	Developer
11	Developer forgets password	• Email address • Reset request	Developer	Forget Password	• Send reset link • Update password	Developer
12	Developer deletes account	• Delete request • Verification	Developer	Delete Account	1. Permanent deletion of data	Developer

Table 3.1: System Event Table

3.4.2 Activity Diagrams

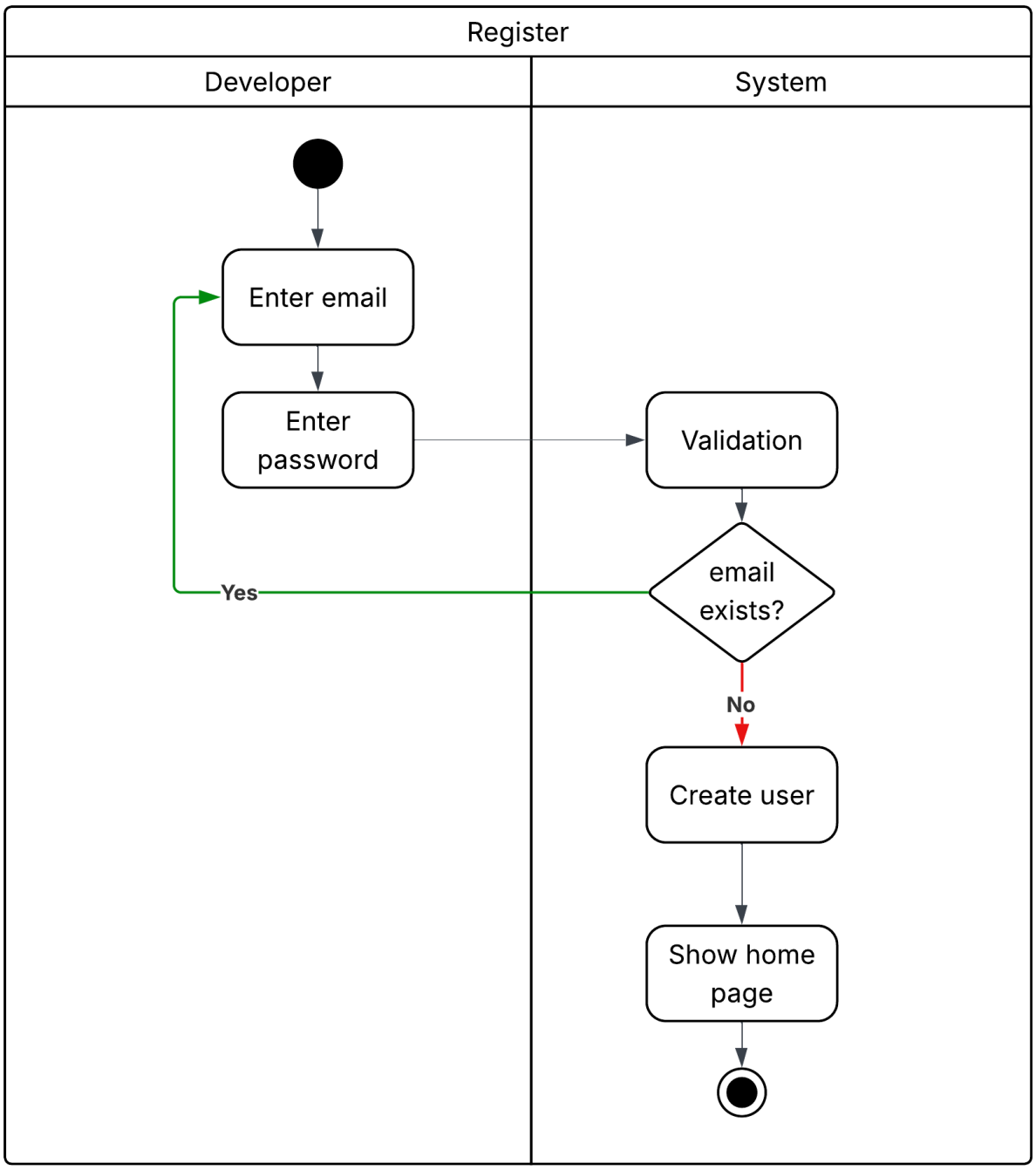


Figure 3.1: Register activity diagram

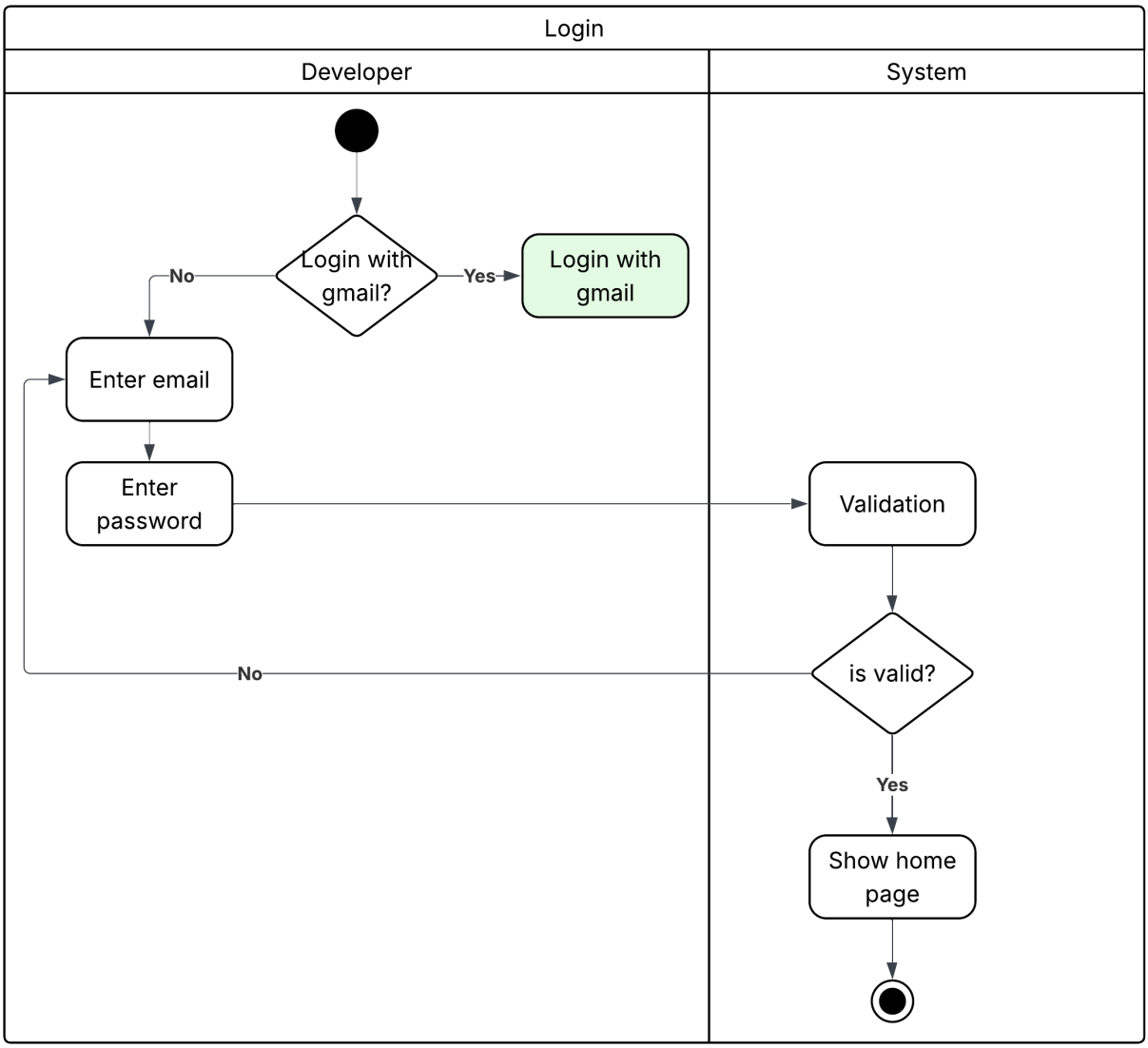


Figure 3.2: Login activity diagram

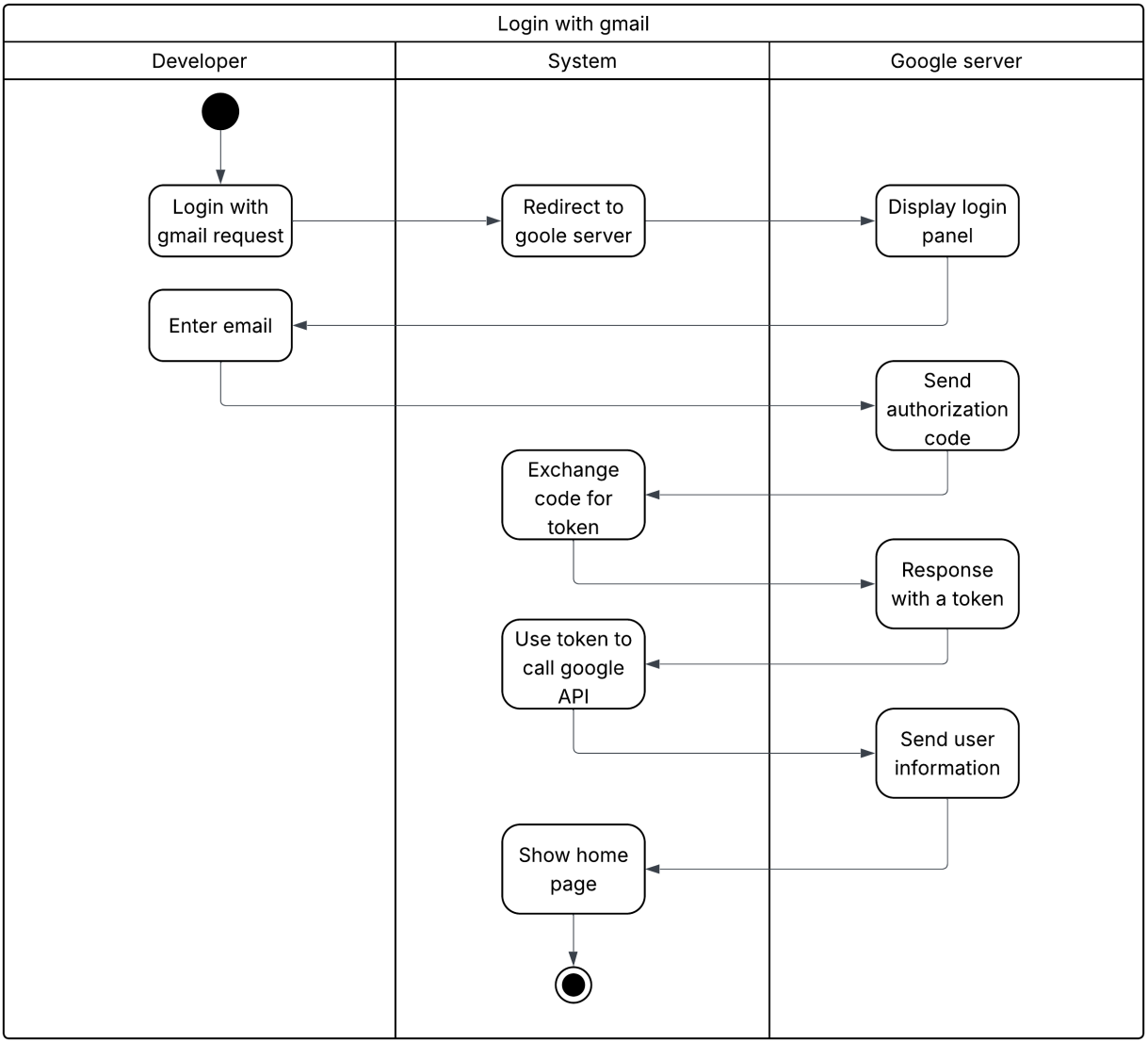


Figure 3.3: Login with g-mail activity diagram

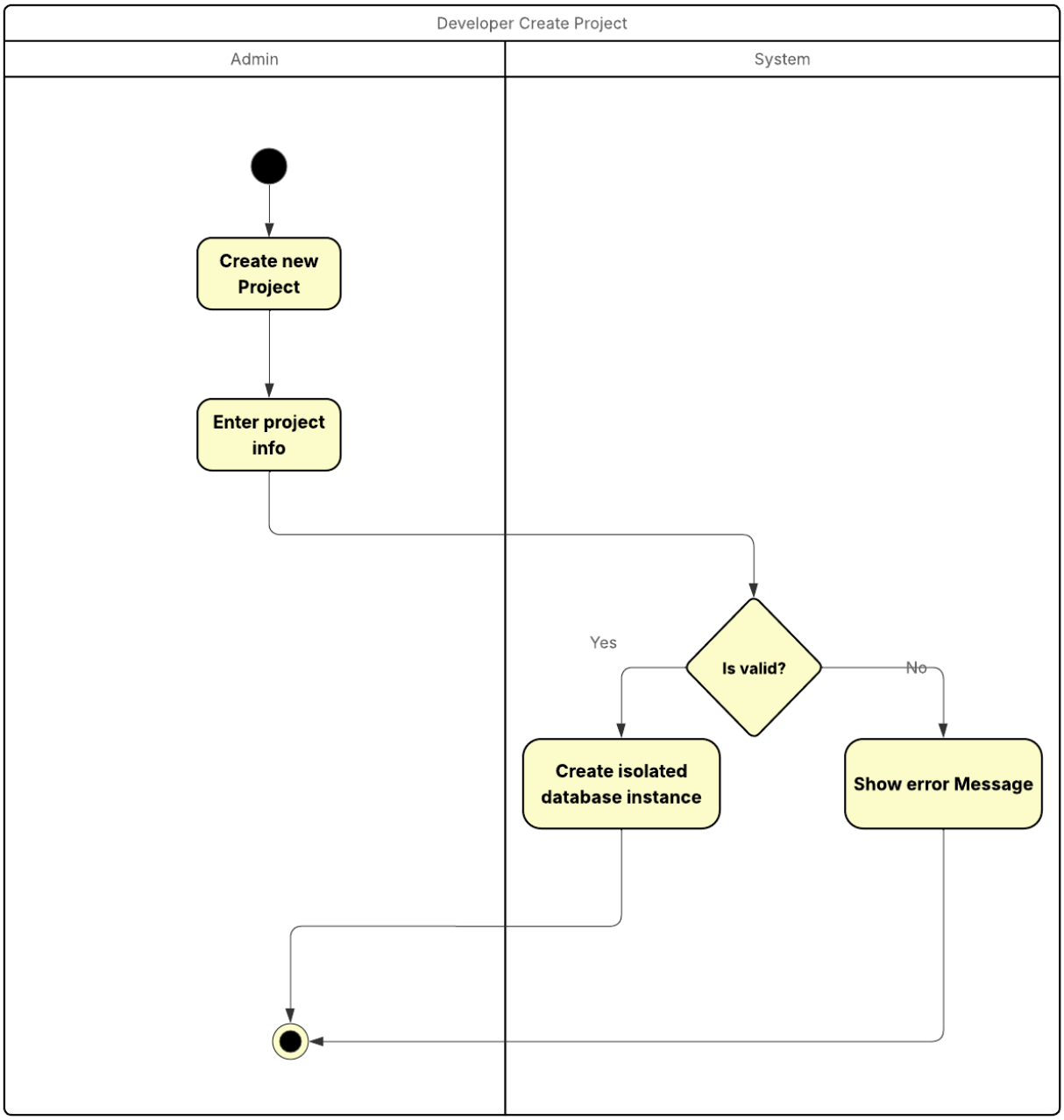


Figure 3.4: Create project activity diagram

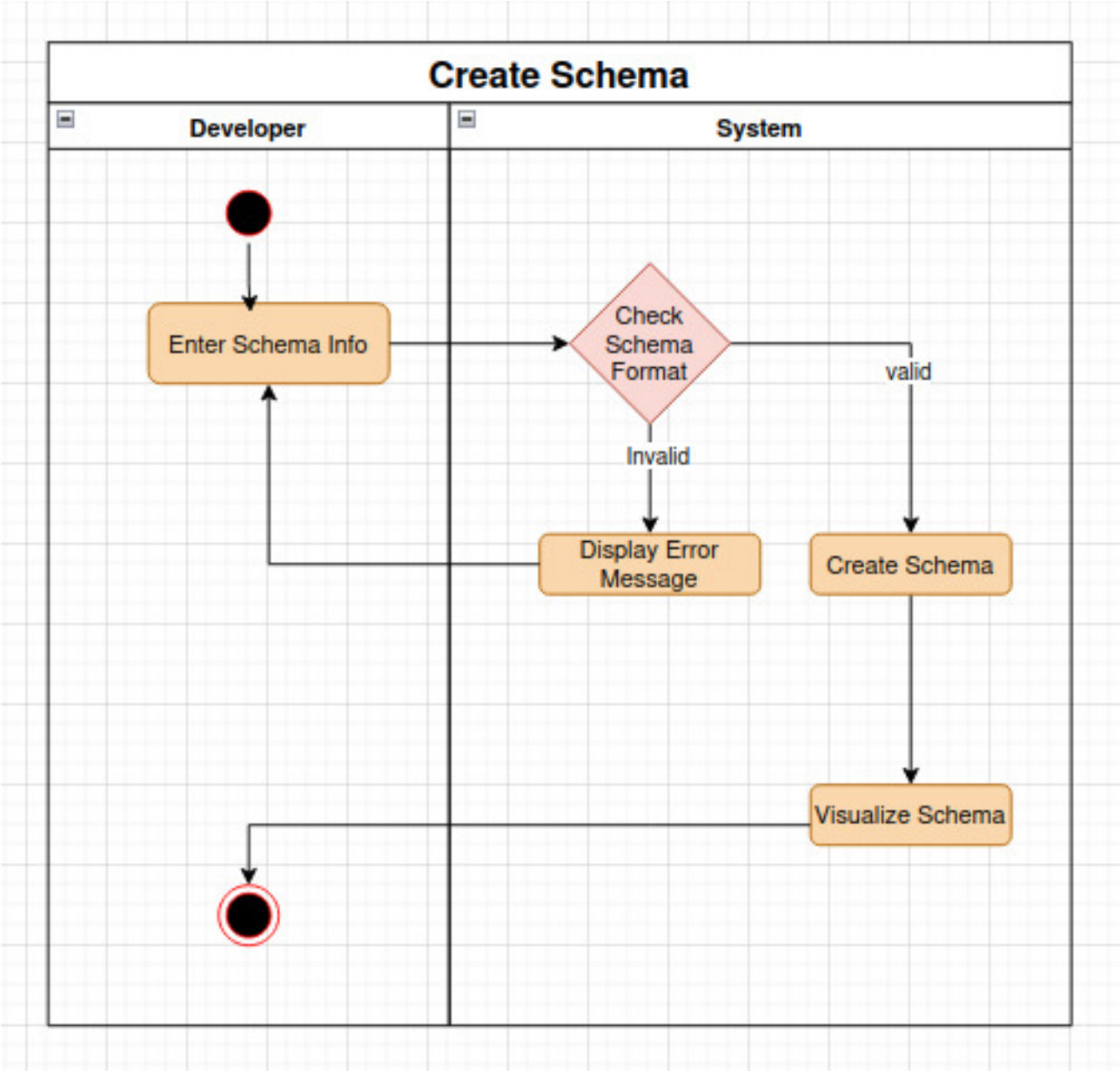


Figure 3.5: Create schema activity diagram

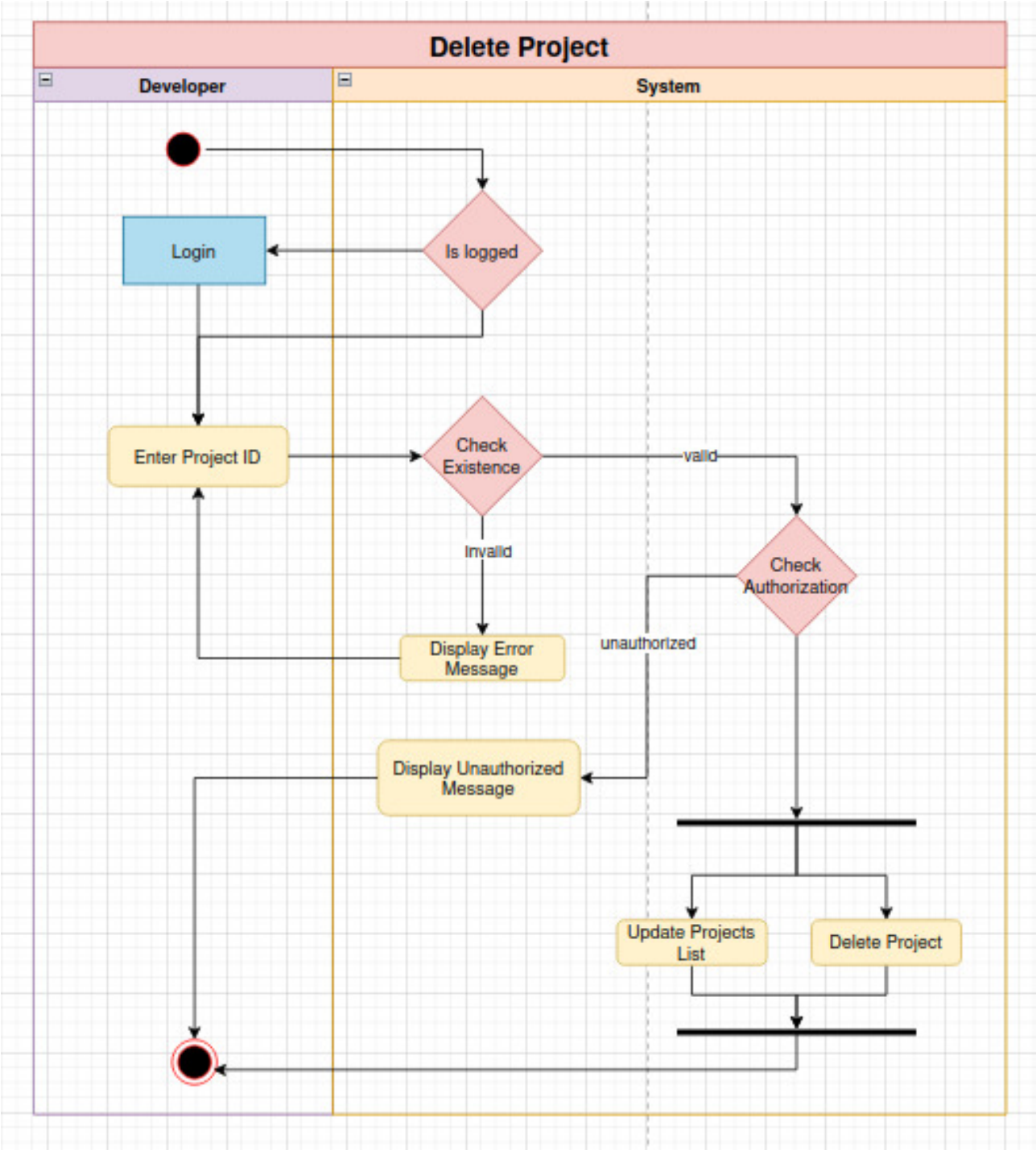


Figure 3.6: Delete project activity diagram

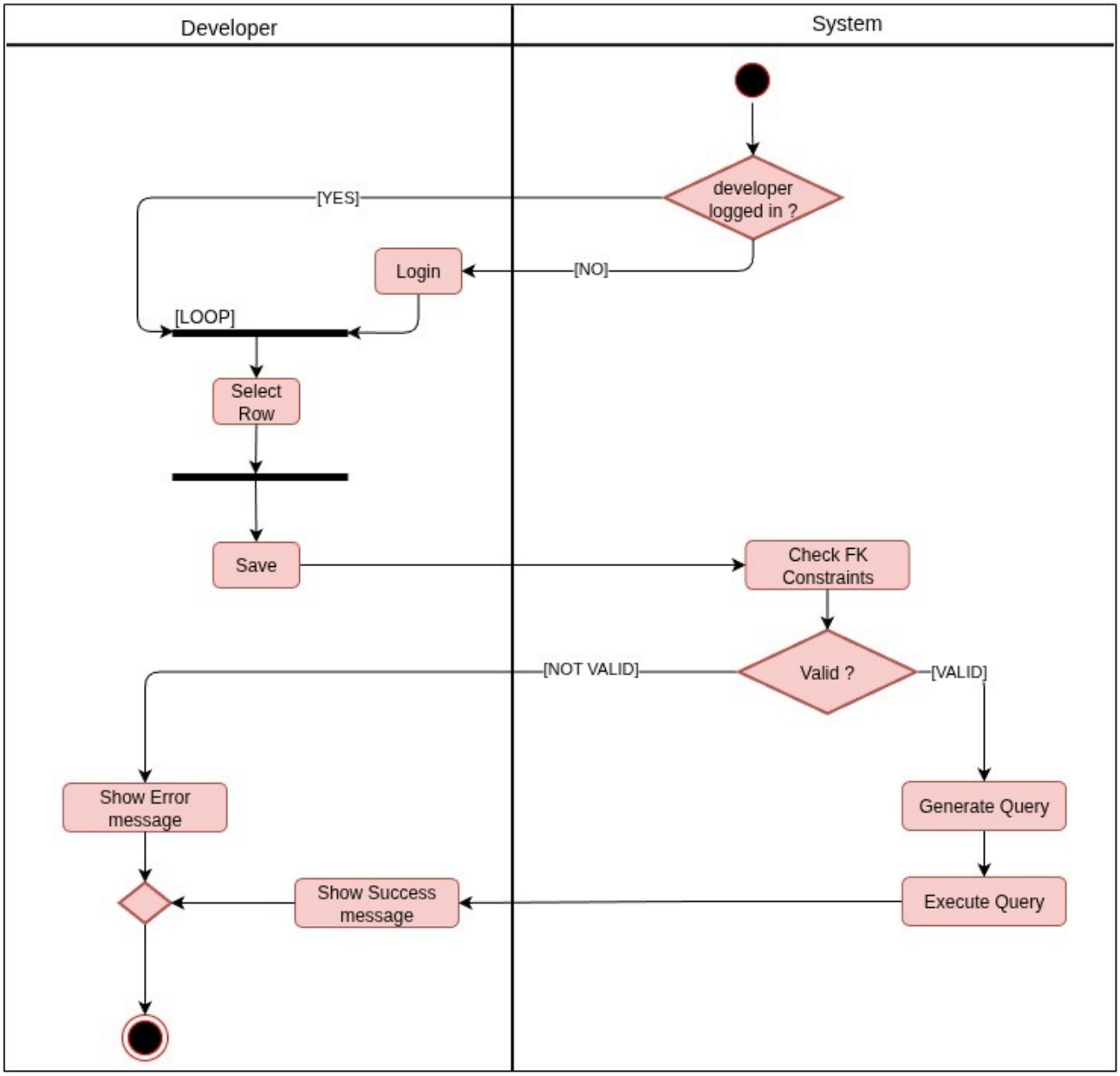


Figure 3.7: Delete rows activity diagram

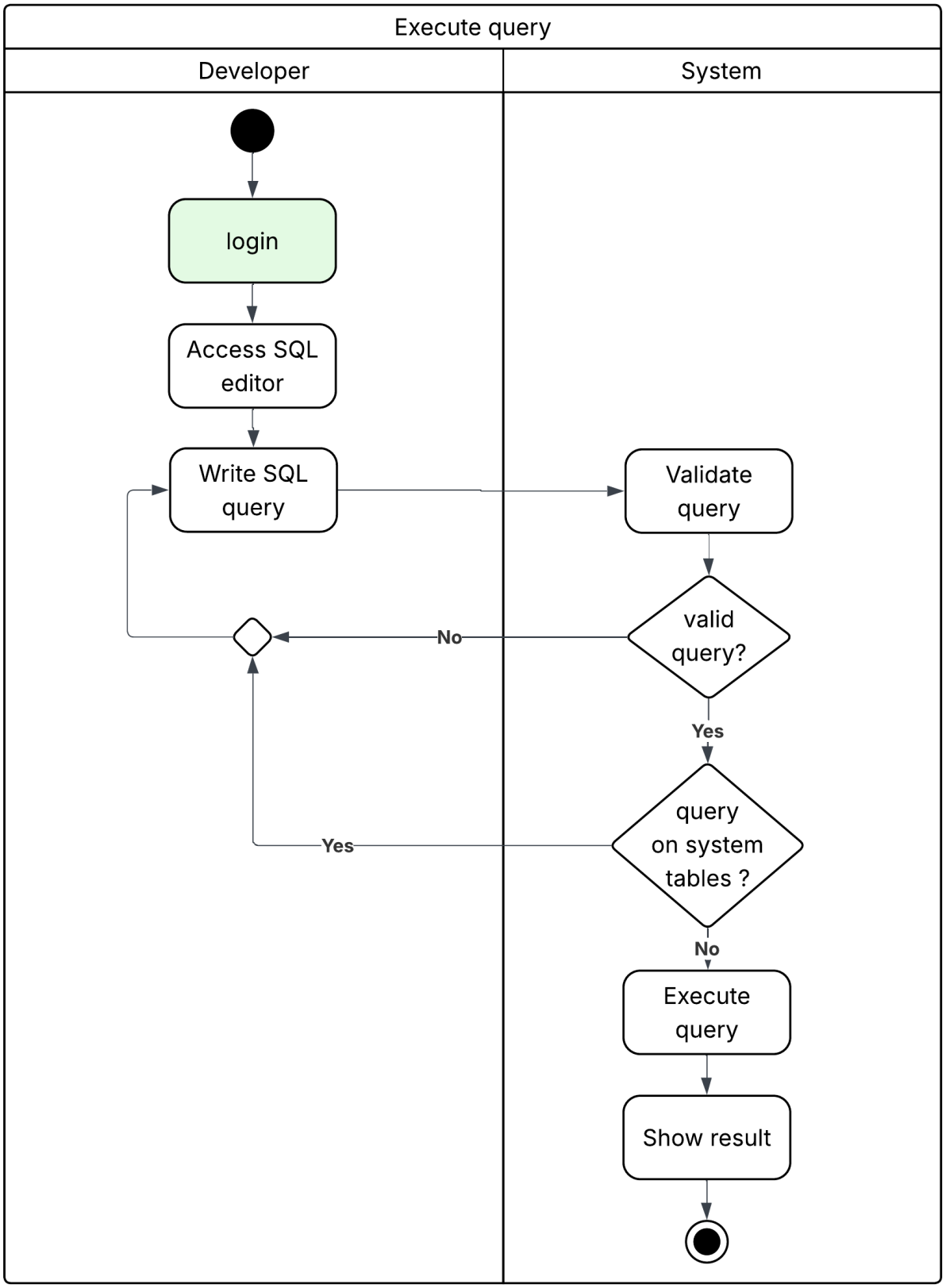


Figure 3.8: Execute SQL query activity diagram

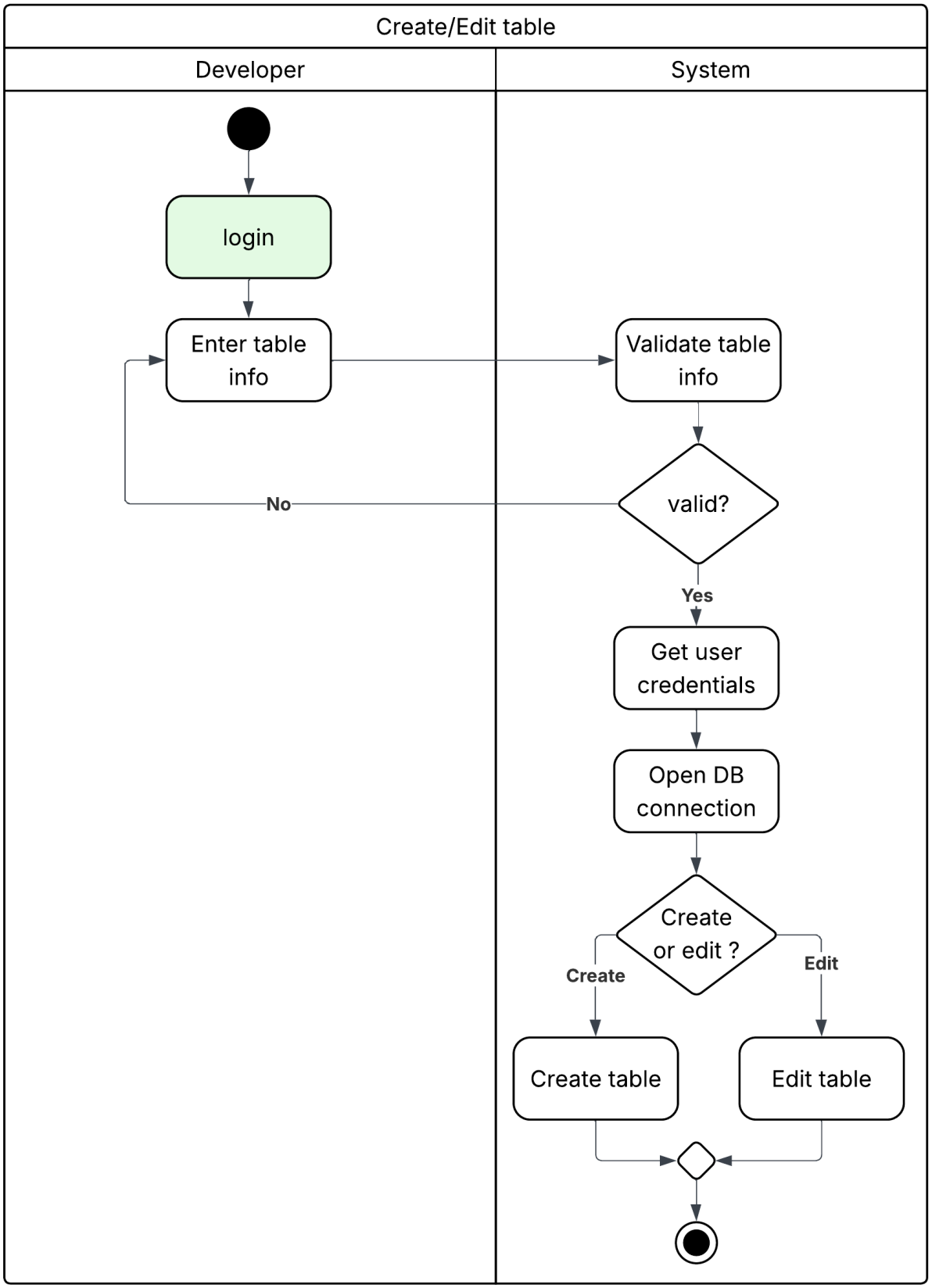


Figure 3.9: Create & edit SQL table activity diagram

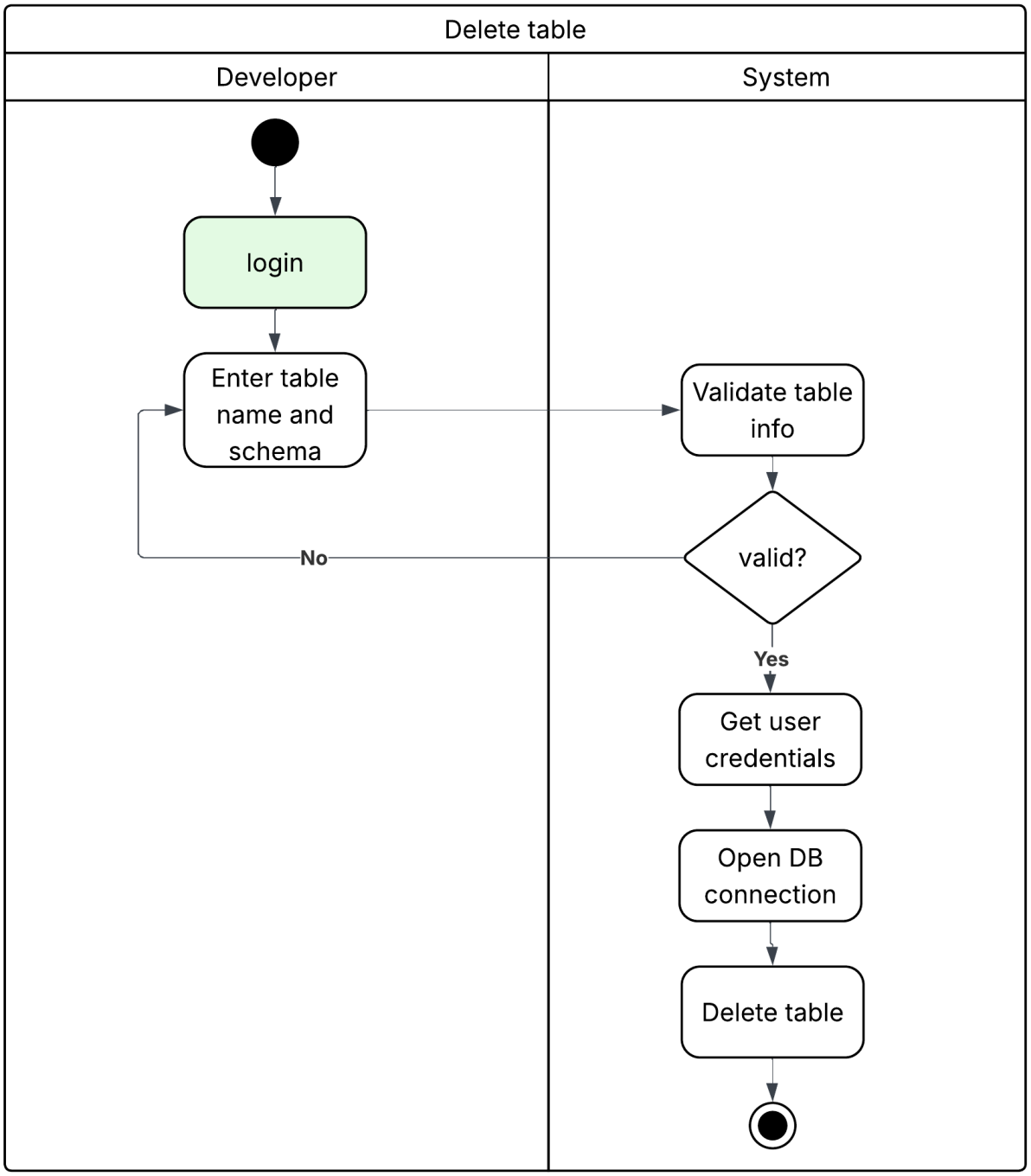


Figure 3.10: Delete SQL table activity diagram

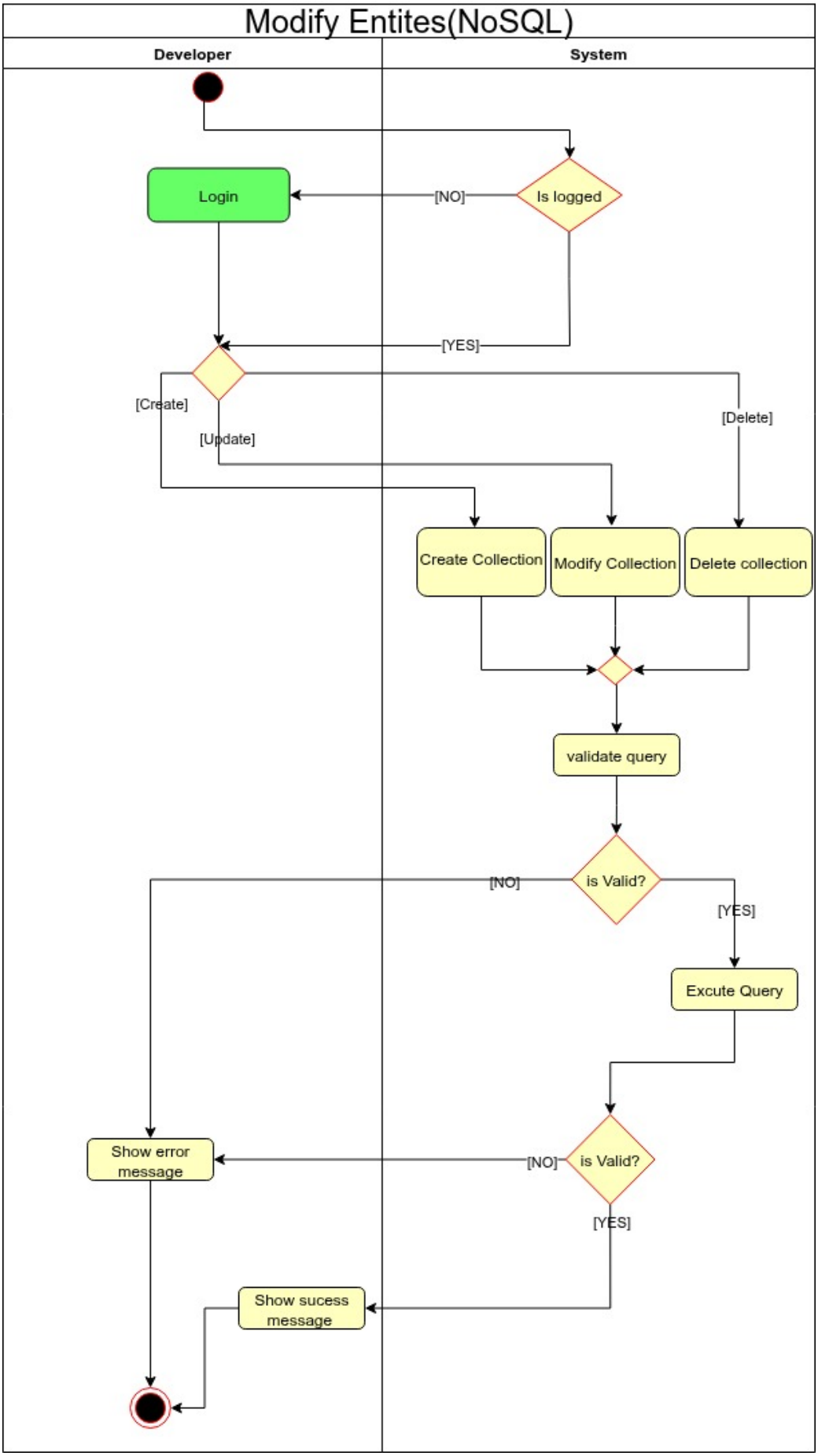


Figure 3.11: Modify entity (NOSQL) activity diagram

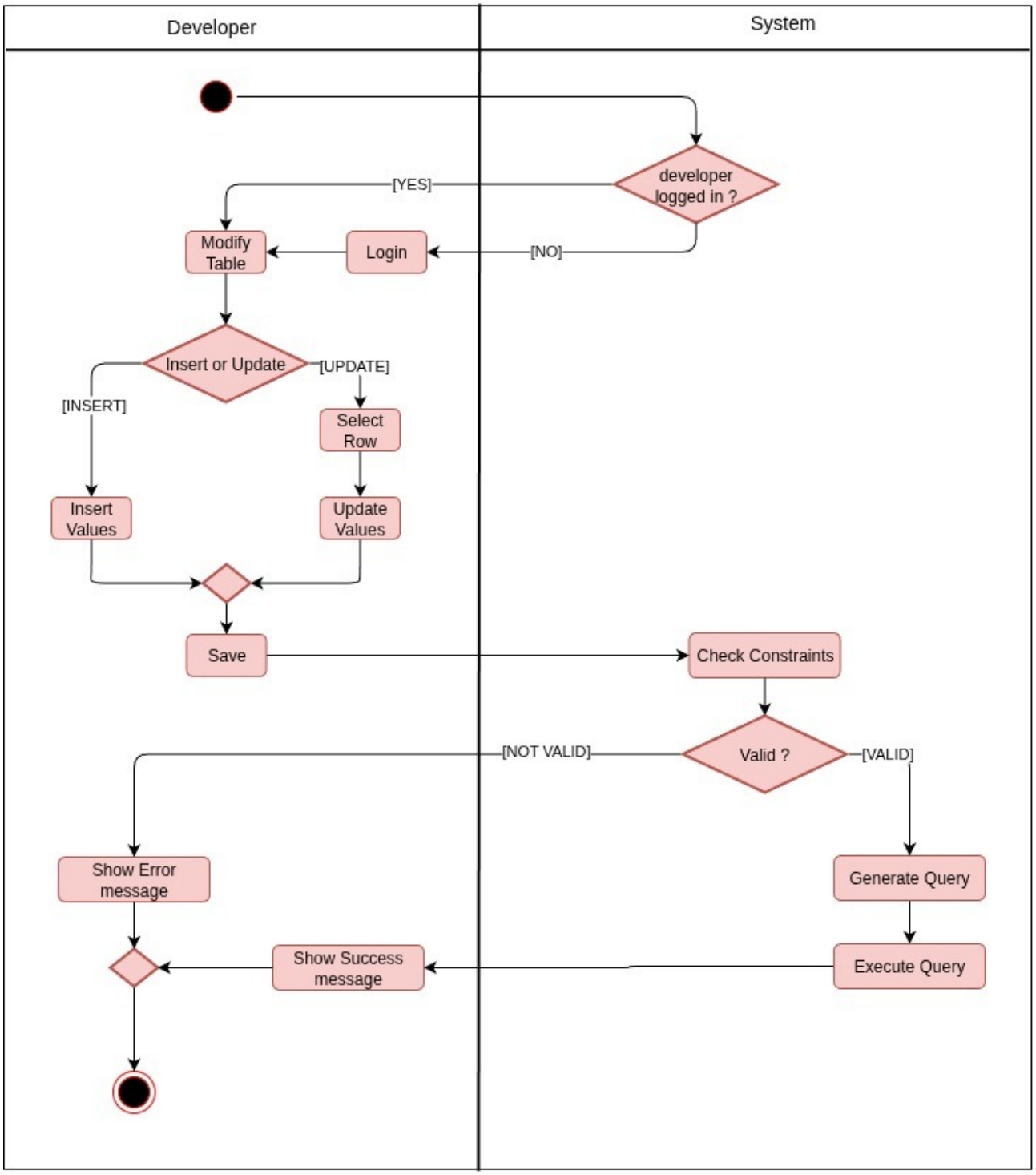


Figure 3.12: Insert row(s) activity diagram

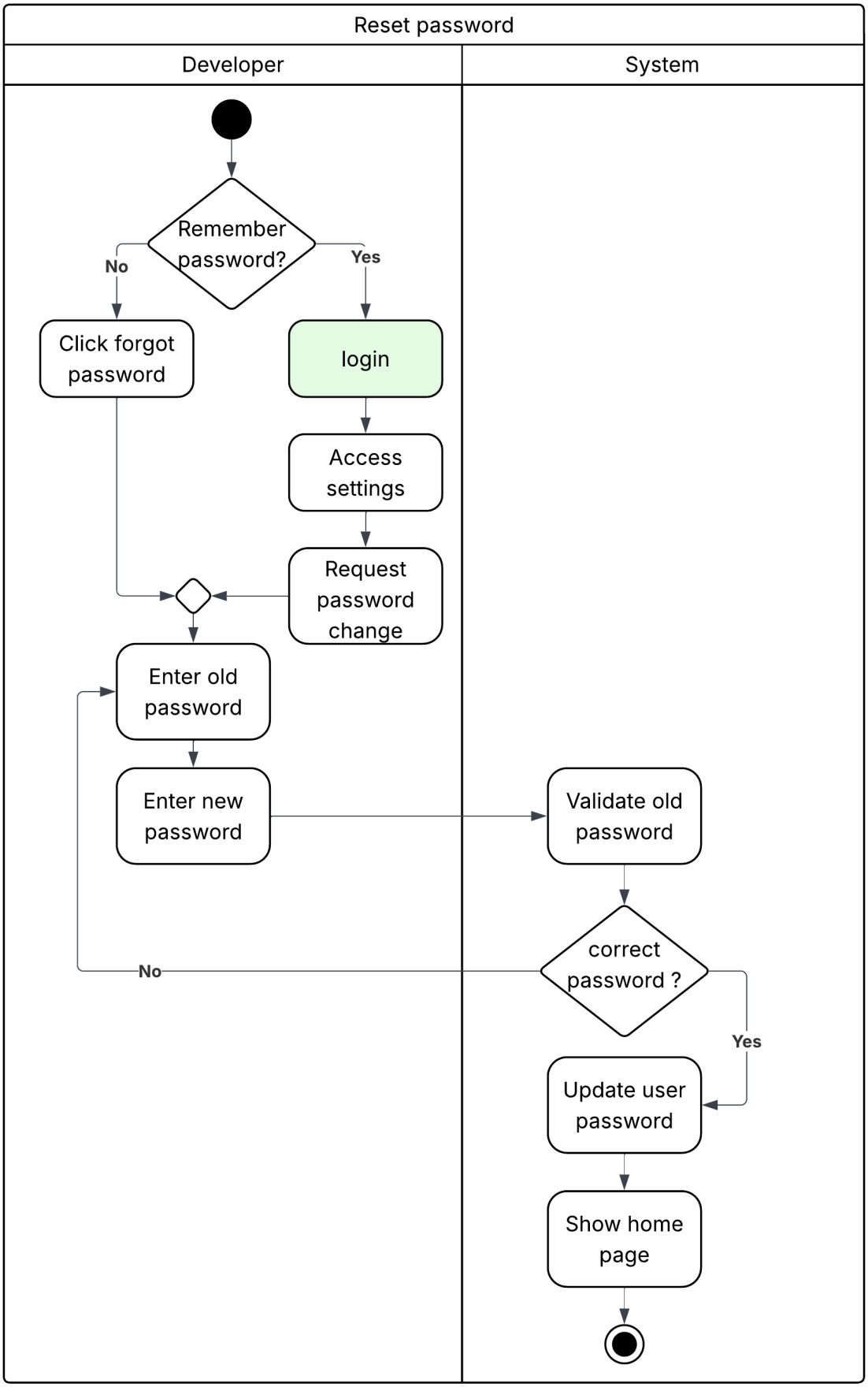


Figure 3.13: Reset password activity diagram

3.4.3 Use case

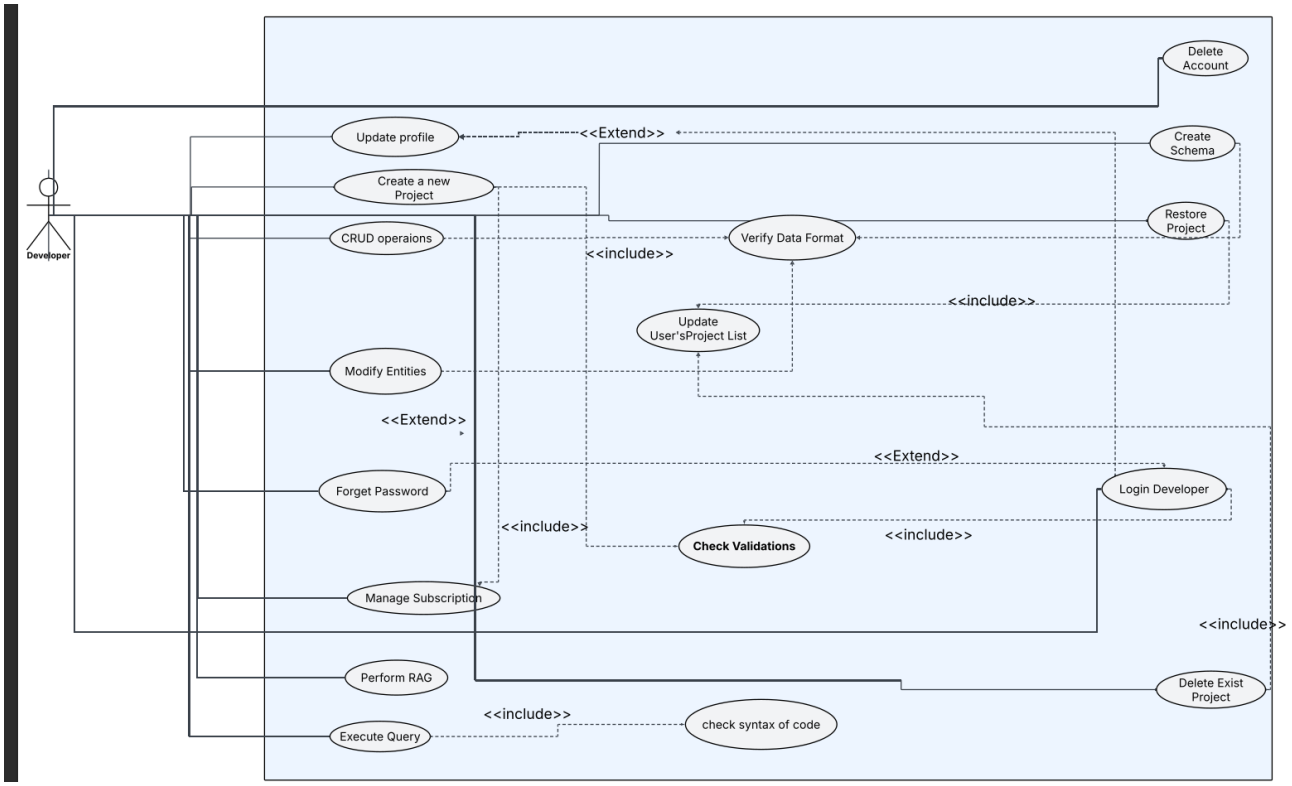


Figure 3.14: Use case diagram

3.4.4 Sequence Diagrams

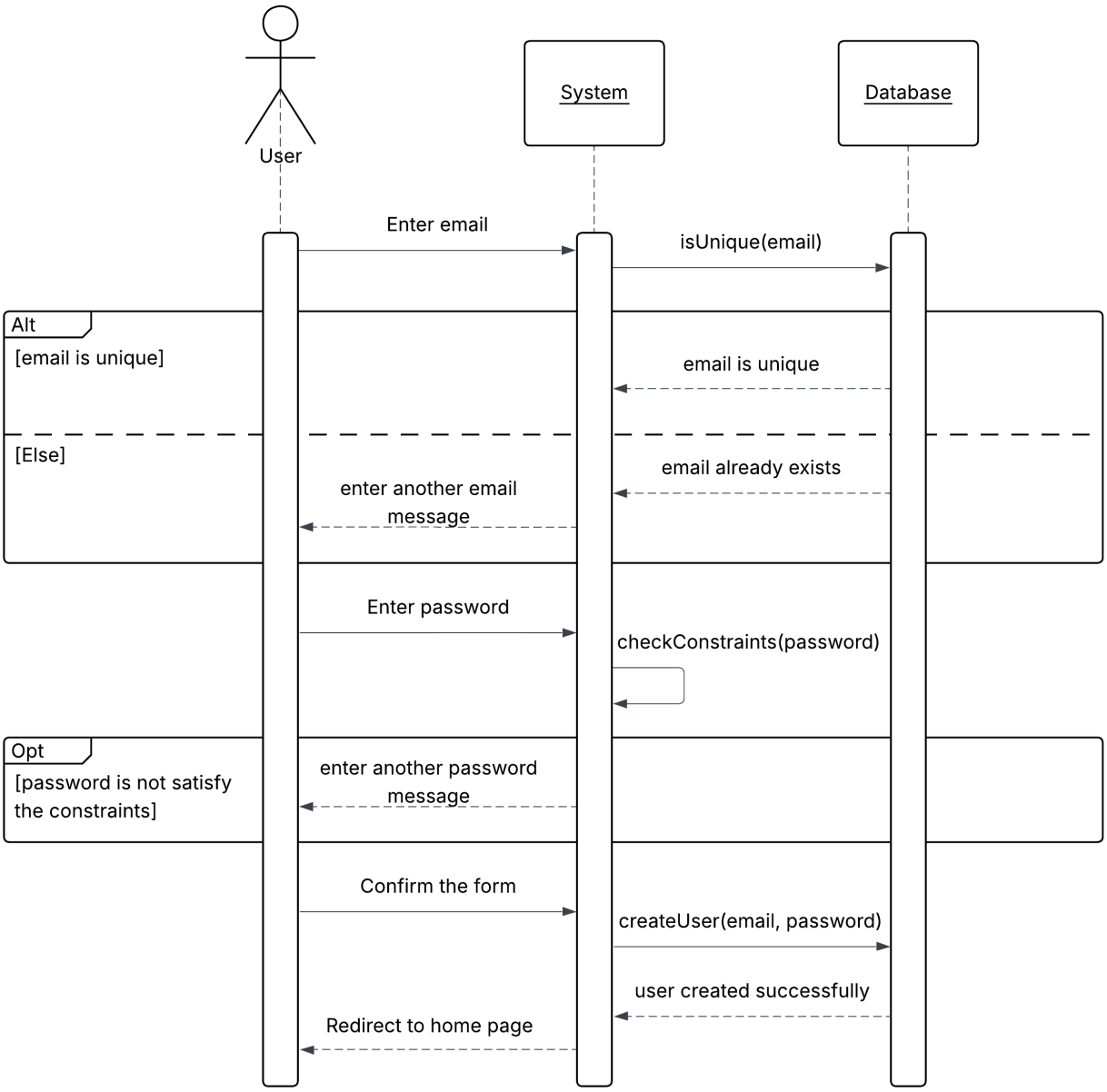


Figure 3.15: Register sequence diagram

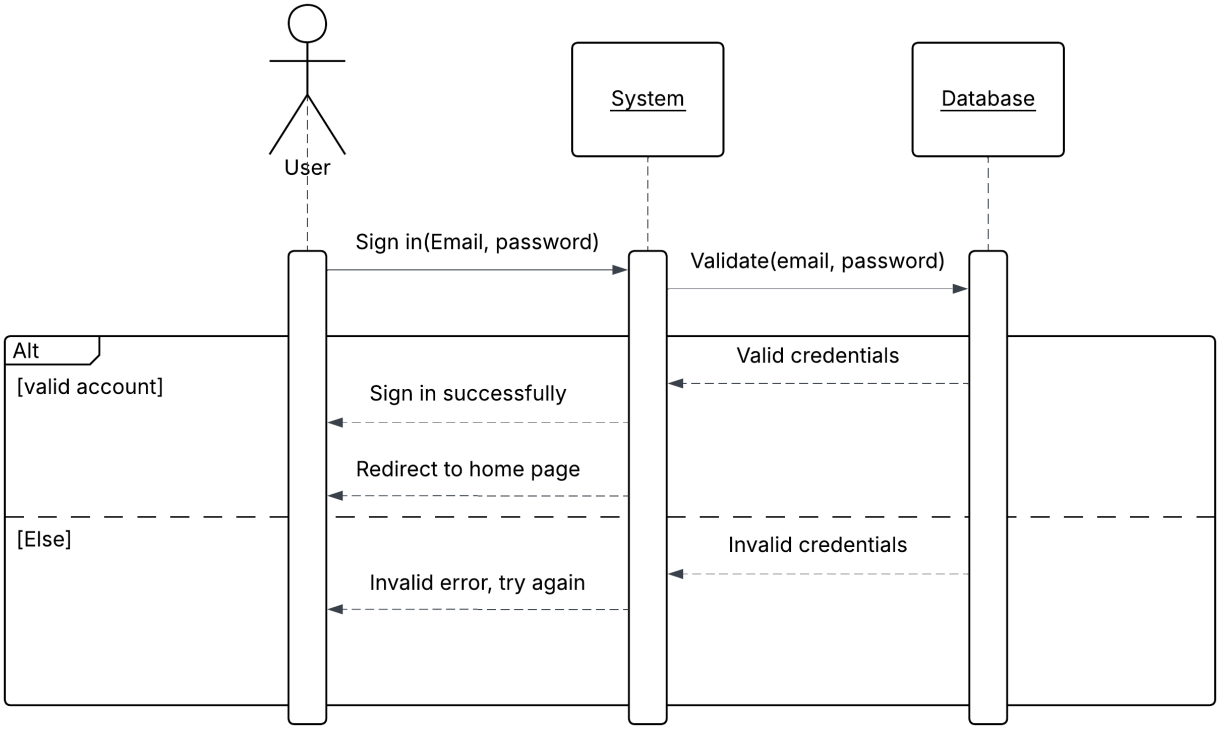


Figure 3.16: Login sequence diagram

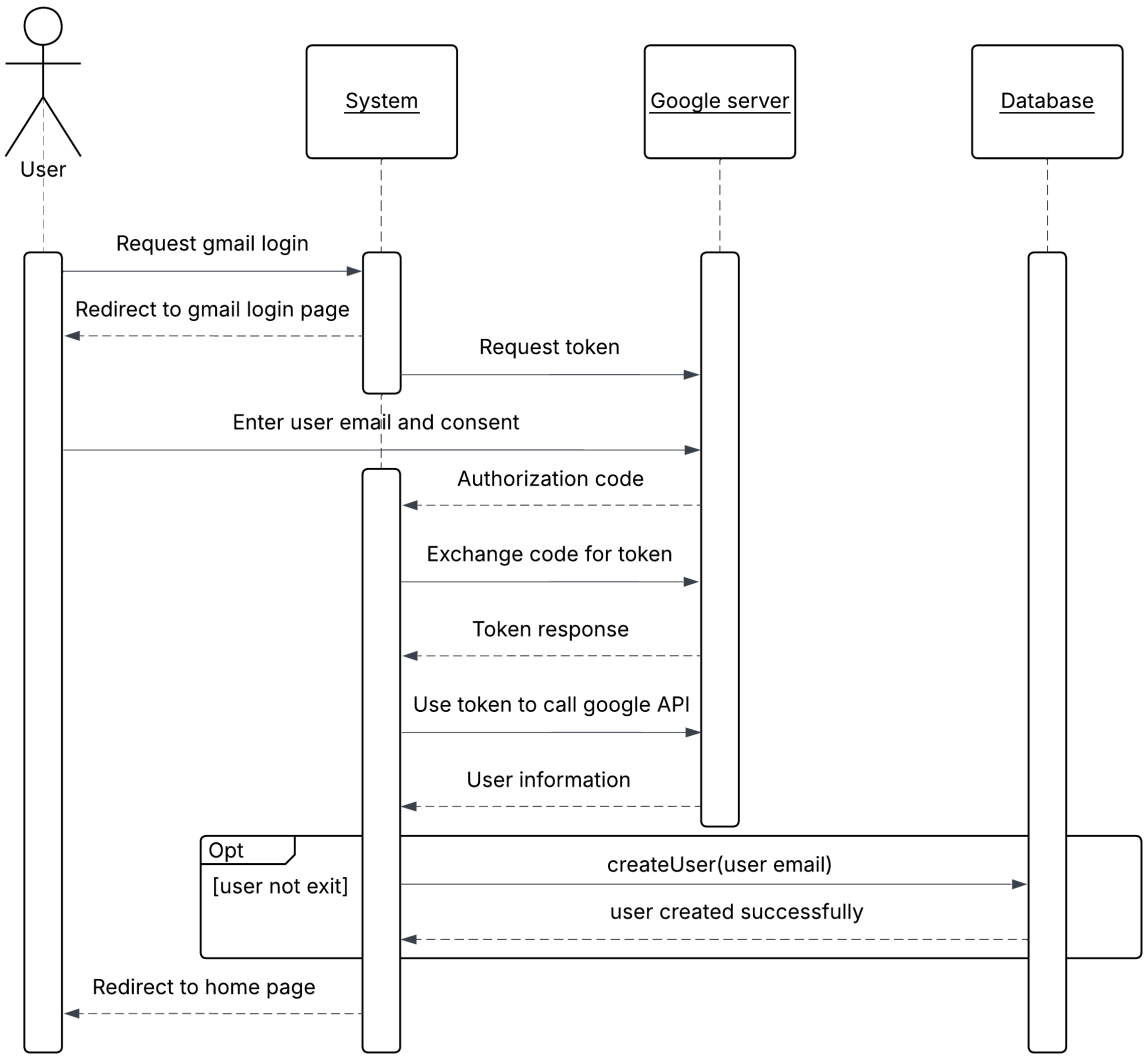


Figure 3.17: Outh sequence diagram

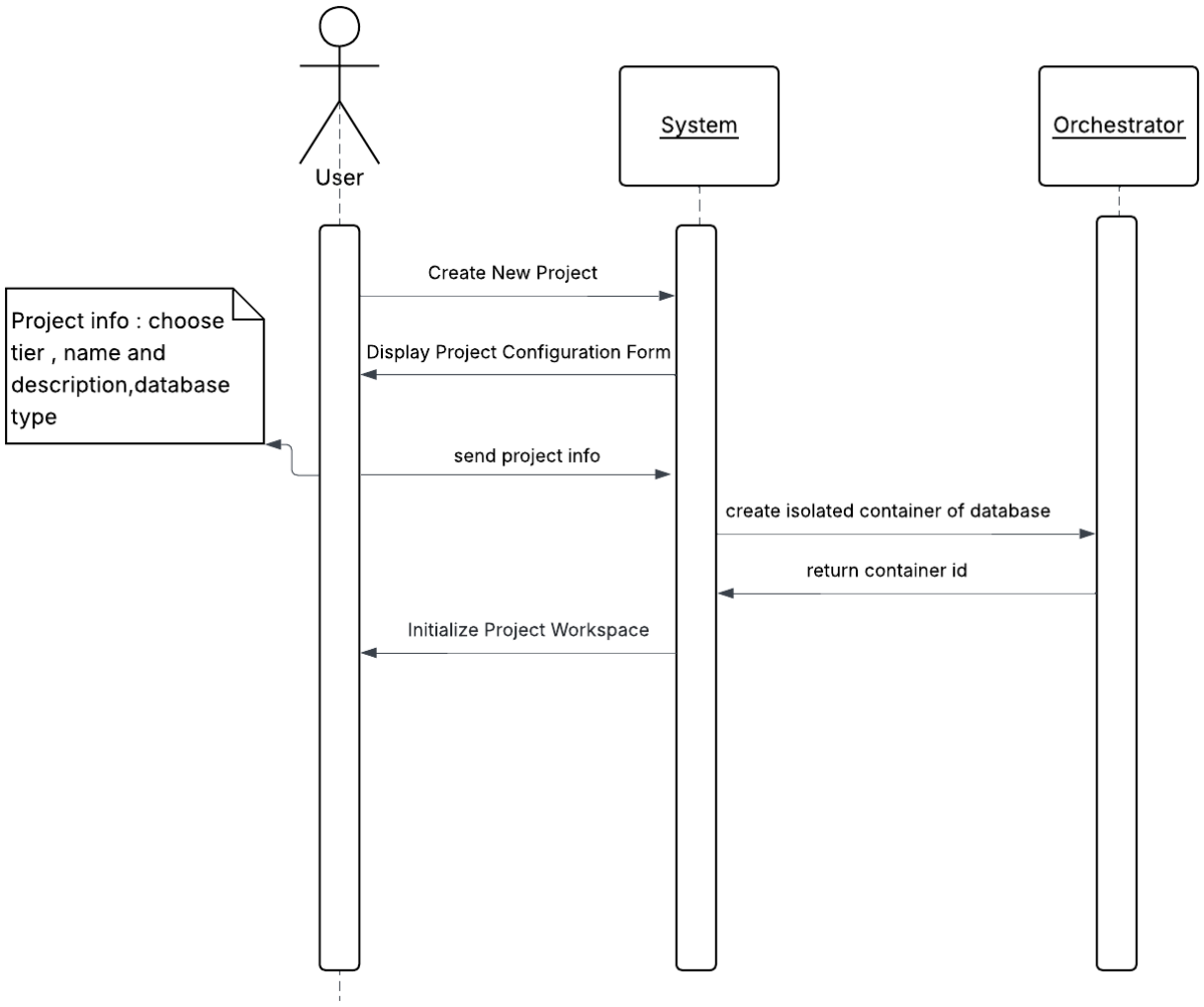


Figure 3.18: Create project sequence diagram

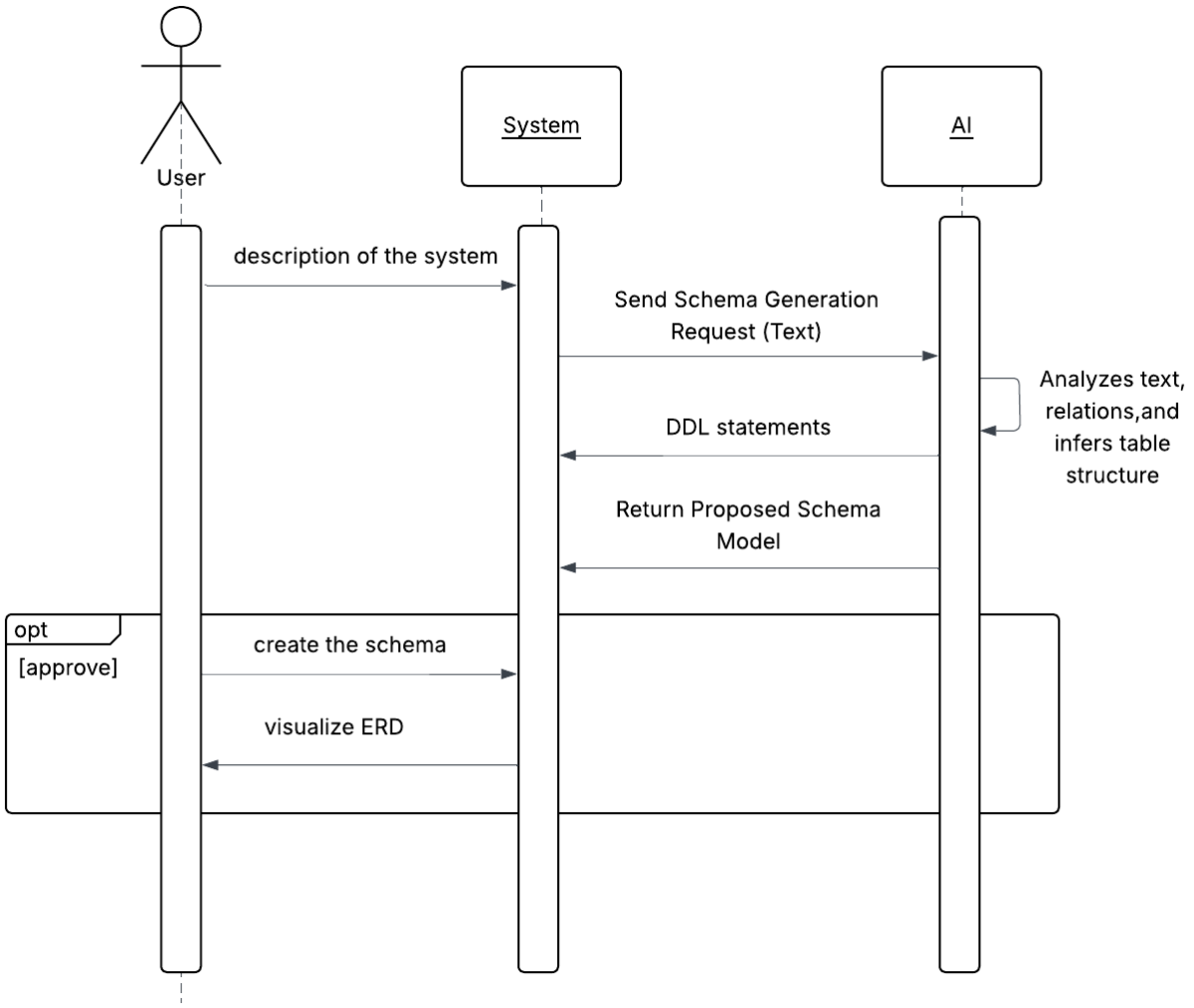


Figure 3.19: Create schema sequence diagram

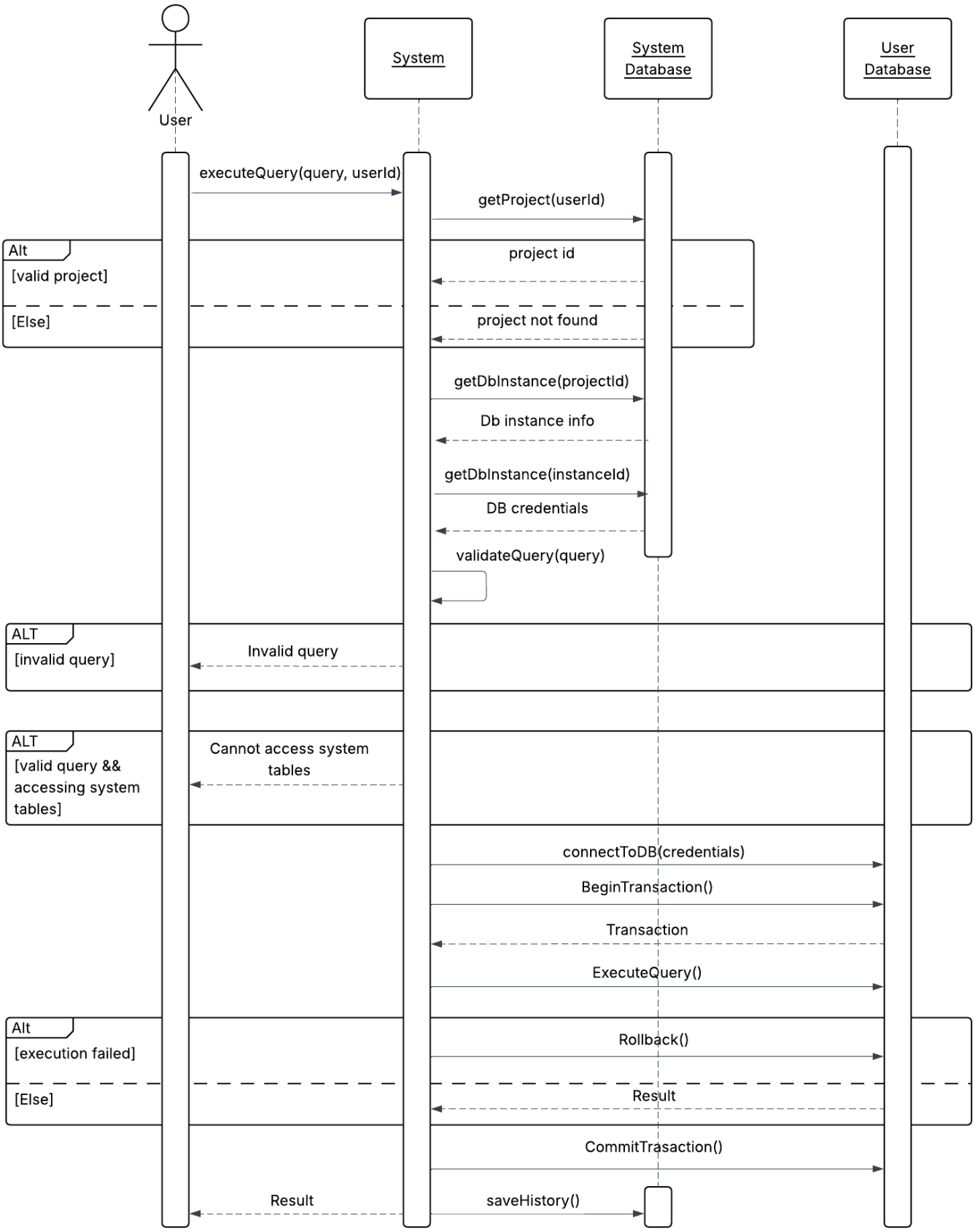


Figure 3.20: Execute SQL query sequence diagram

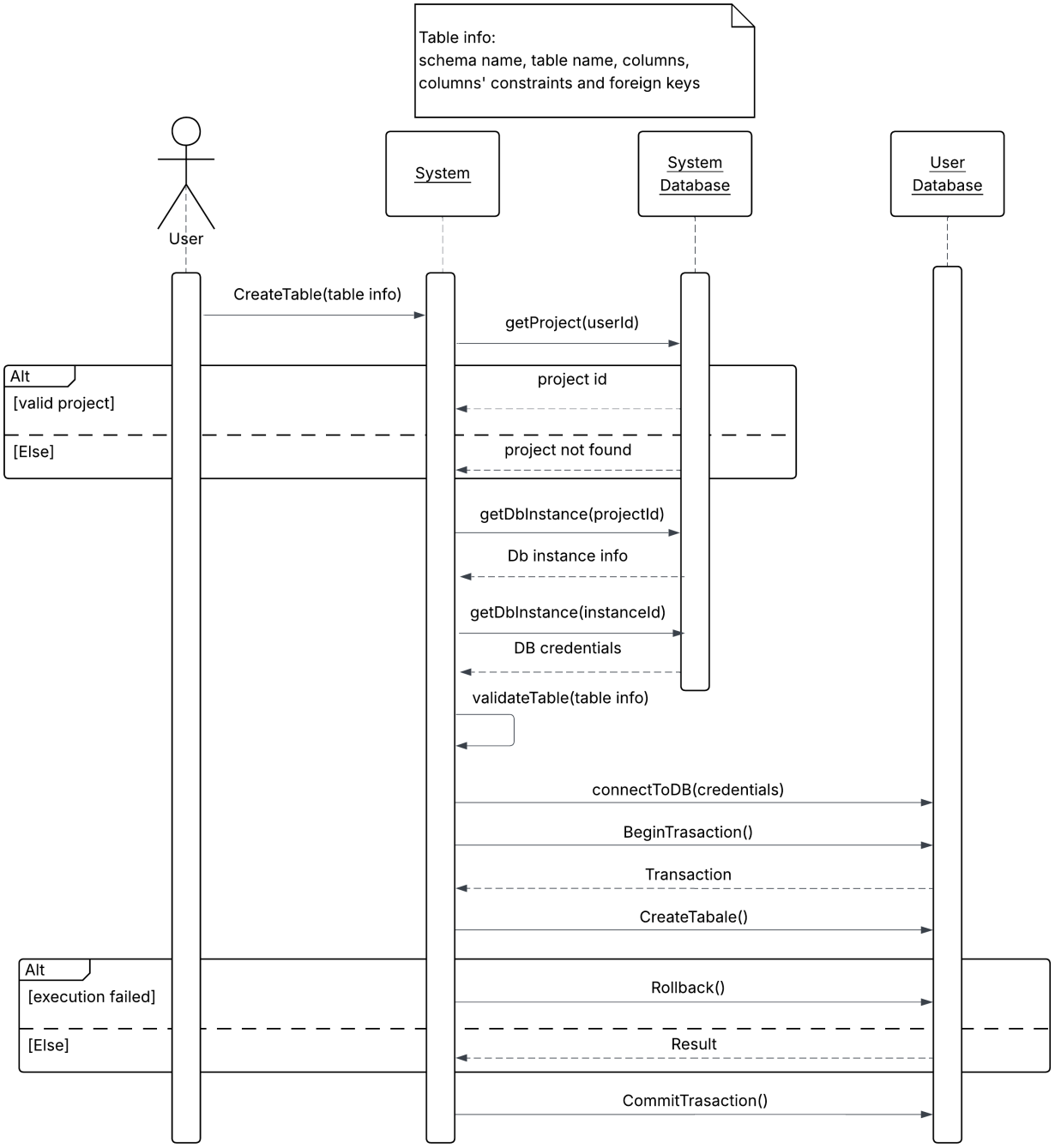


Figure 3.21: Create table sequence diagram

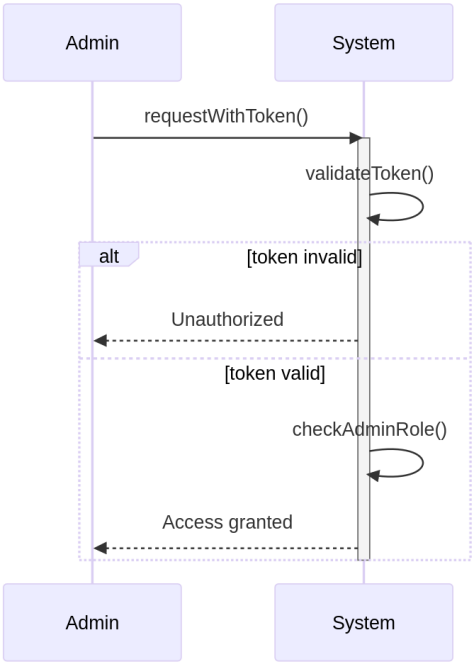


Figure 3.22: Admin authorization sequence diagram

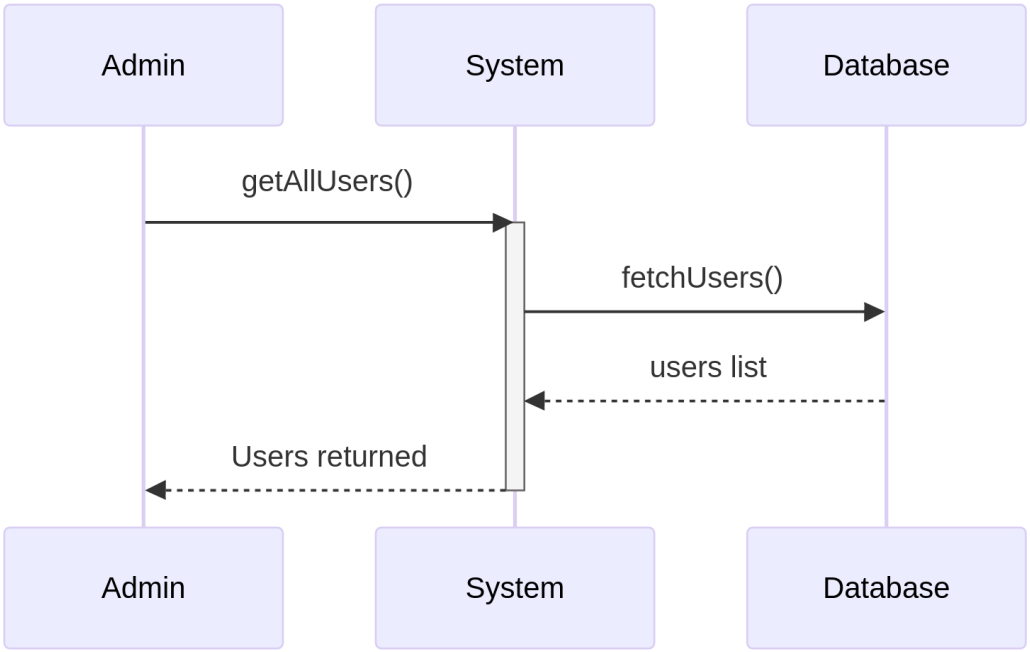


Figure 3.23: Admin read users sequence diagram

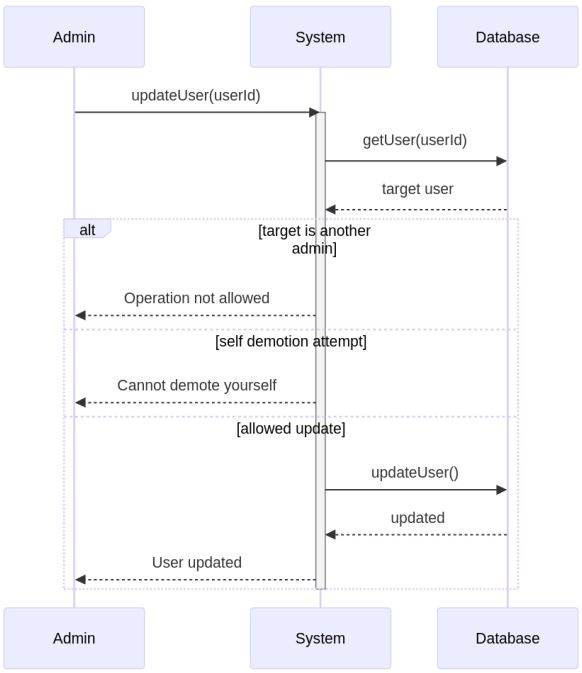


Figure 3.24: Admin update users sequence diagram

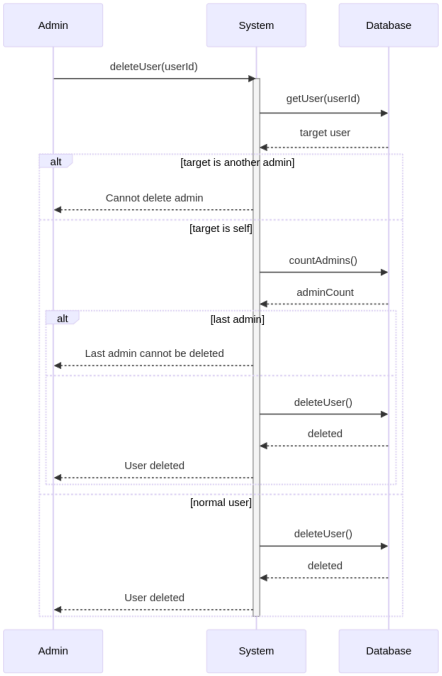


Figure 3.25: Admin delete users sequence diagram

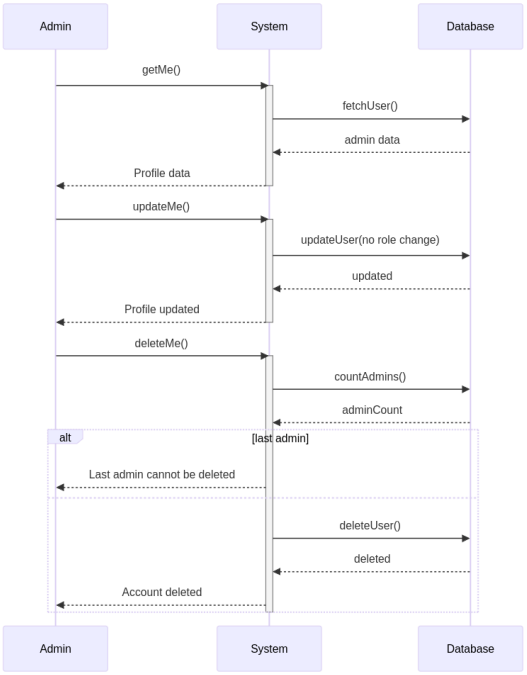


Figure 3.26: Admin self management sequence diagram

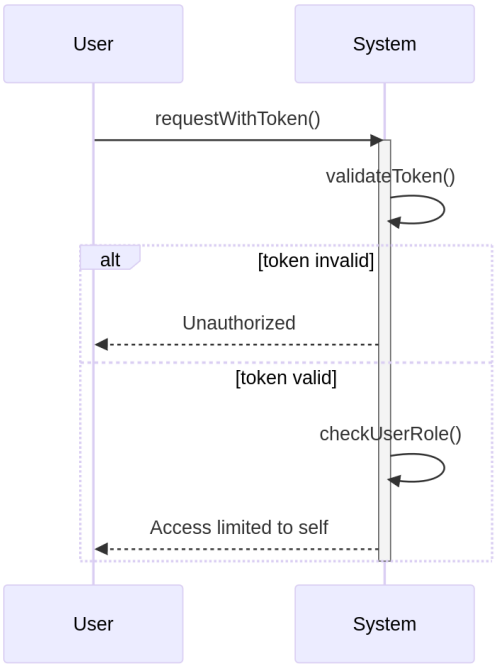


Figure 3.27: User authorization sequence diagram

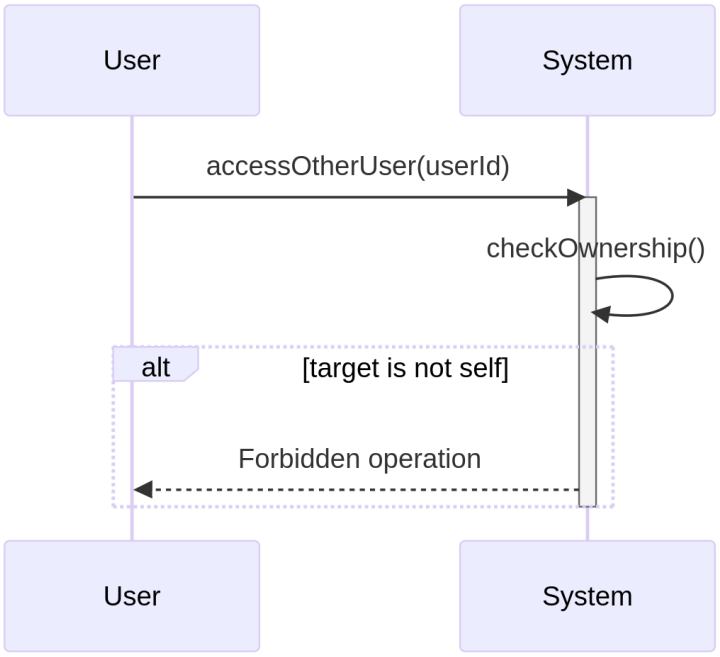


Figure 3.28: User forbidden operations sequence diagram

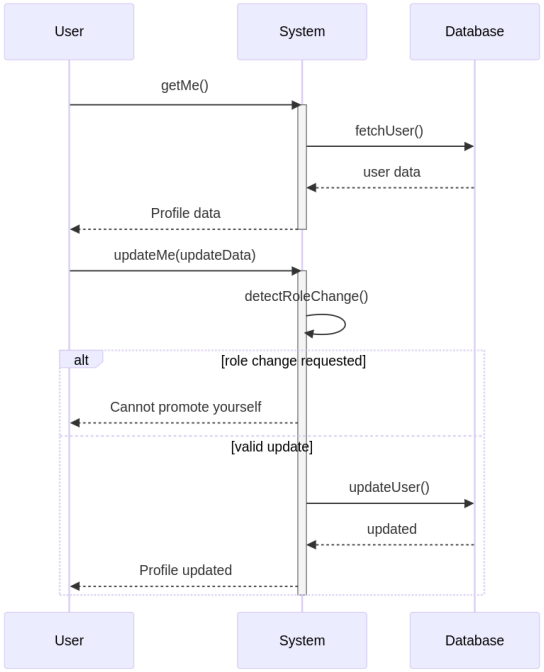


Figure 3.29: User self mangement sequence diagram

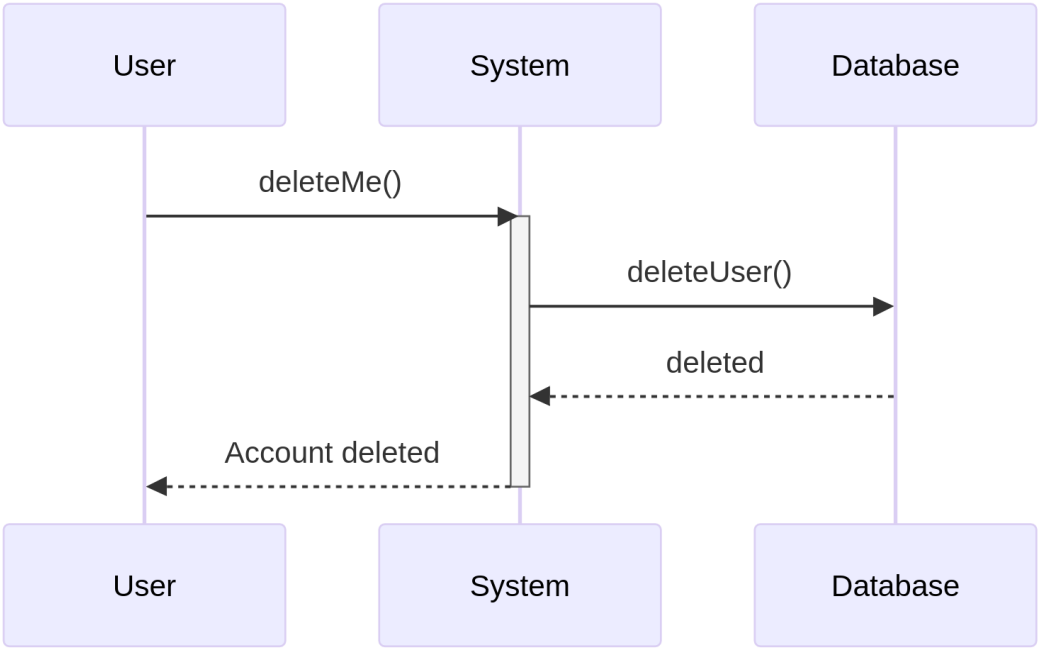


Figure 3.30: User self delete sequence diagram

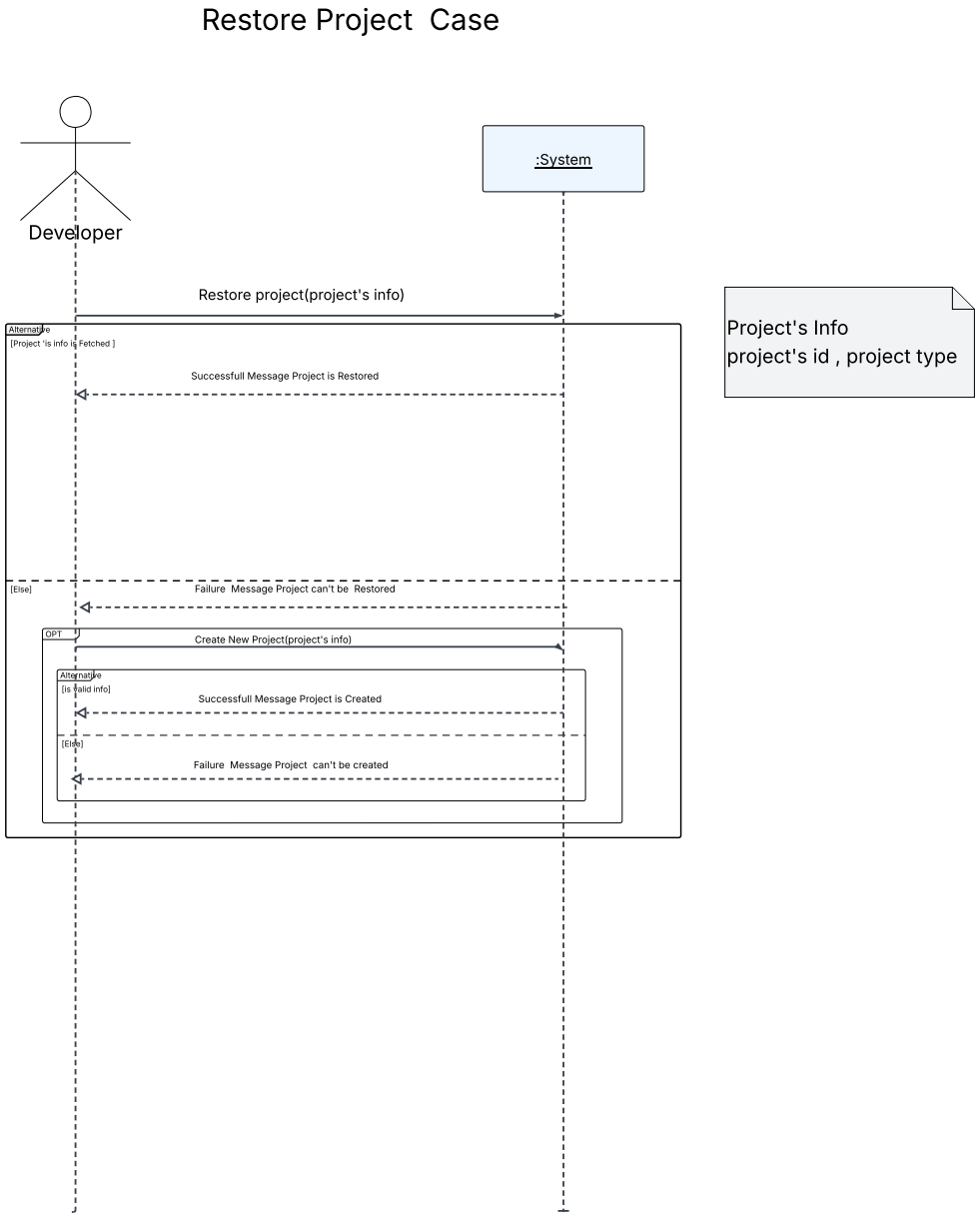


Figure 3.31: Restore project sequence diagram

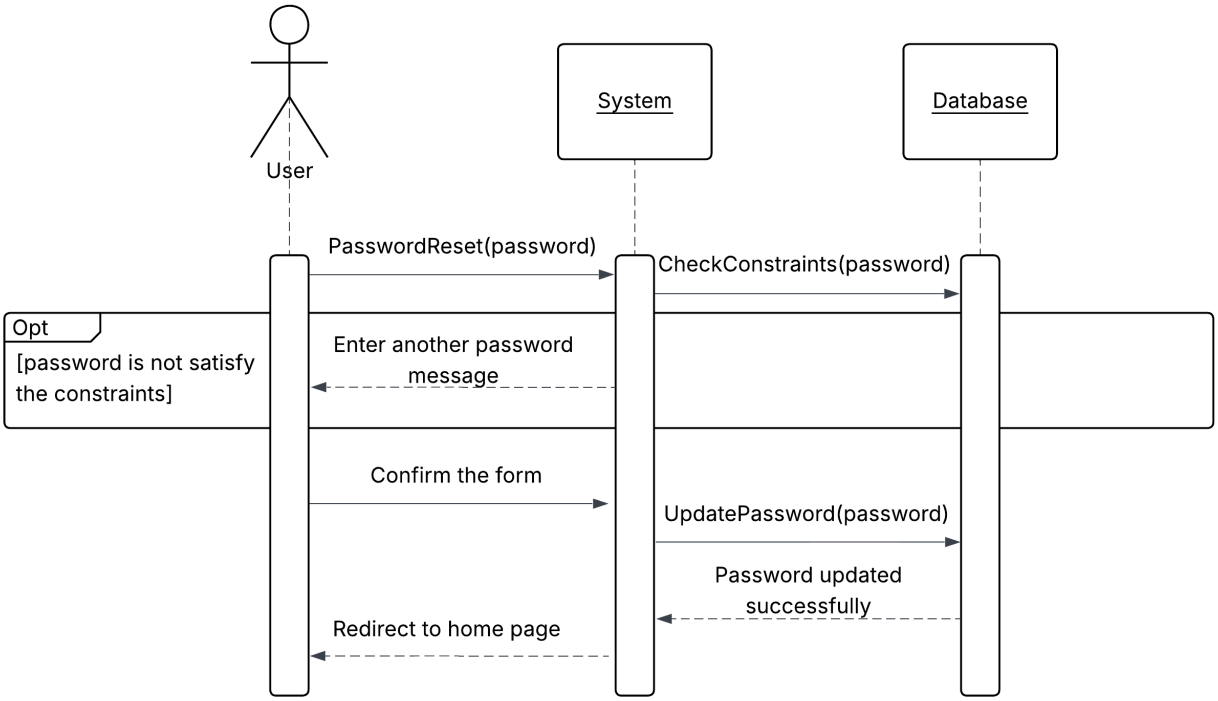


Figure 3.32: Reset password sequence diagram

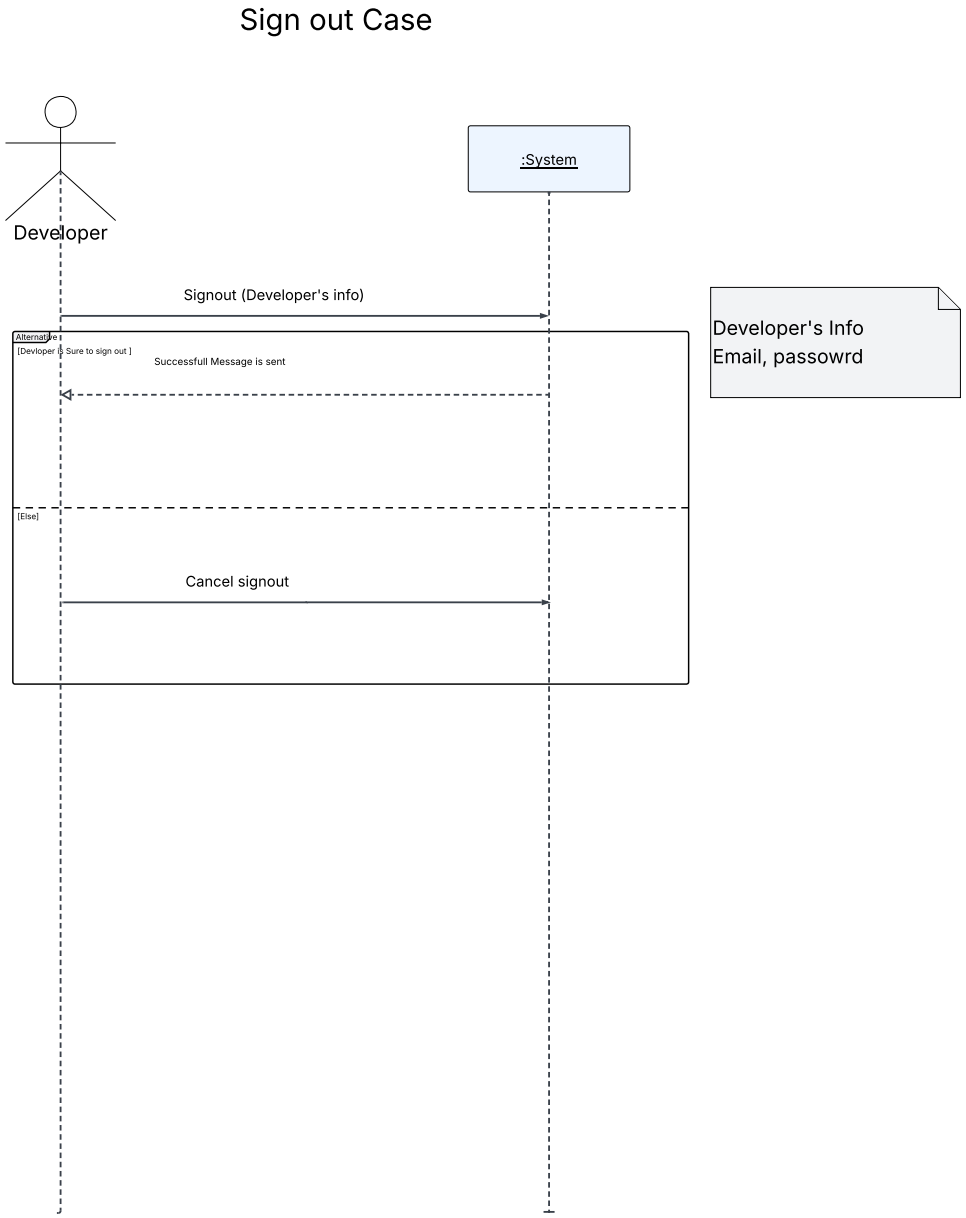


Figure 3.33: Signout sequence diagram

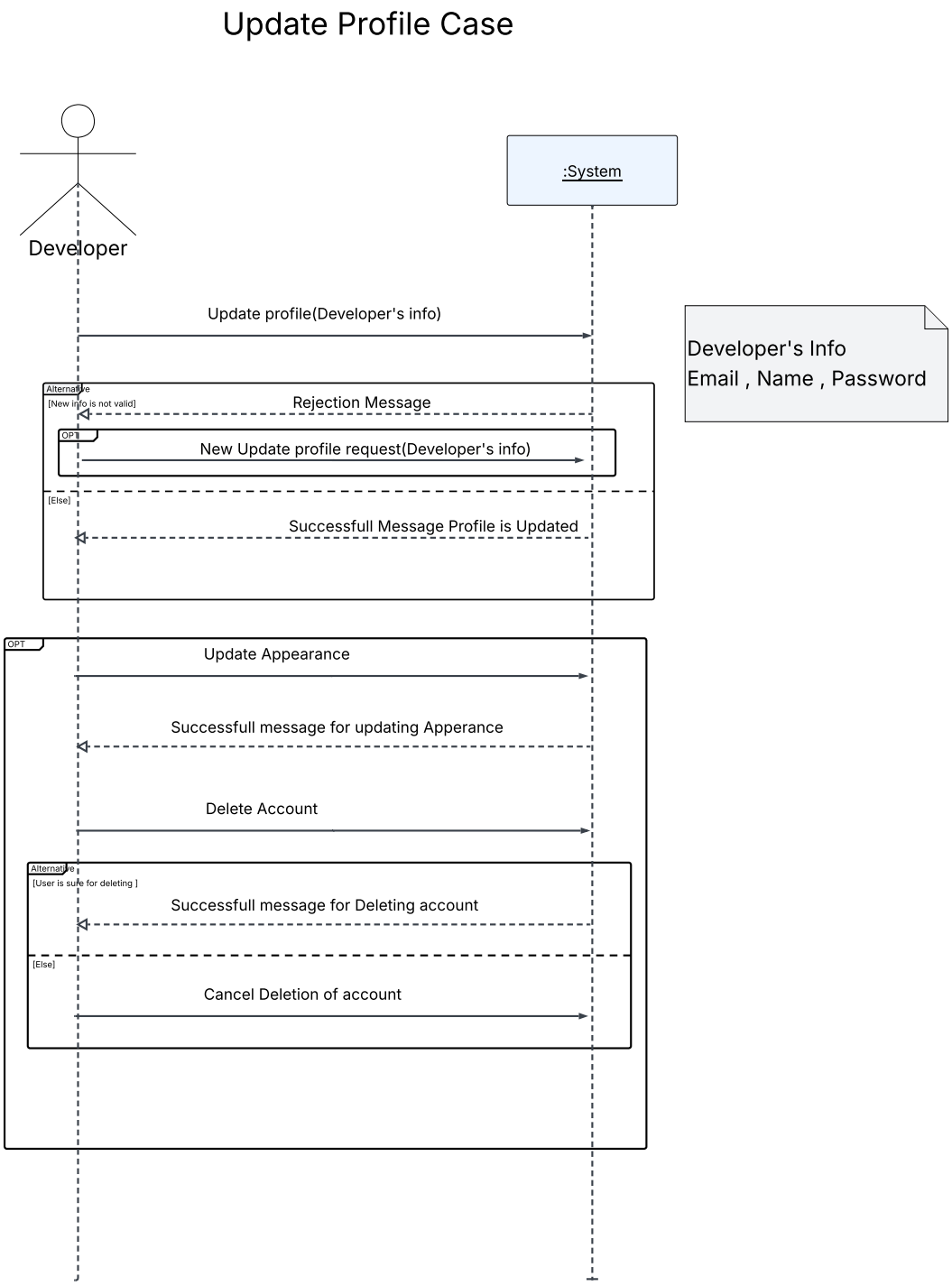


Figure 3.34: Update profile sequence diagram

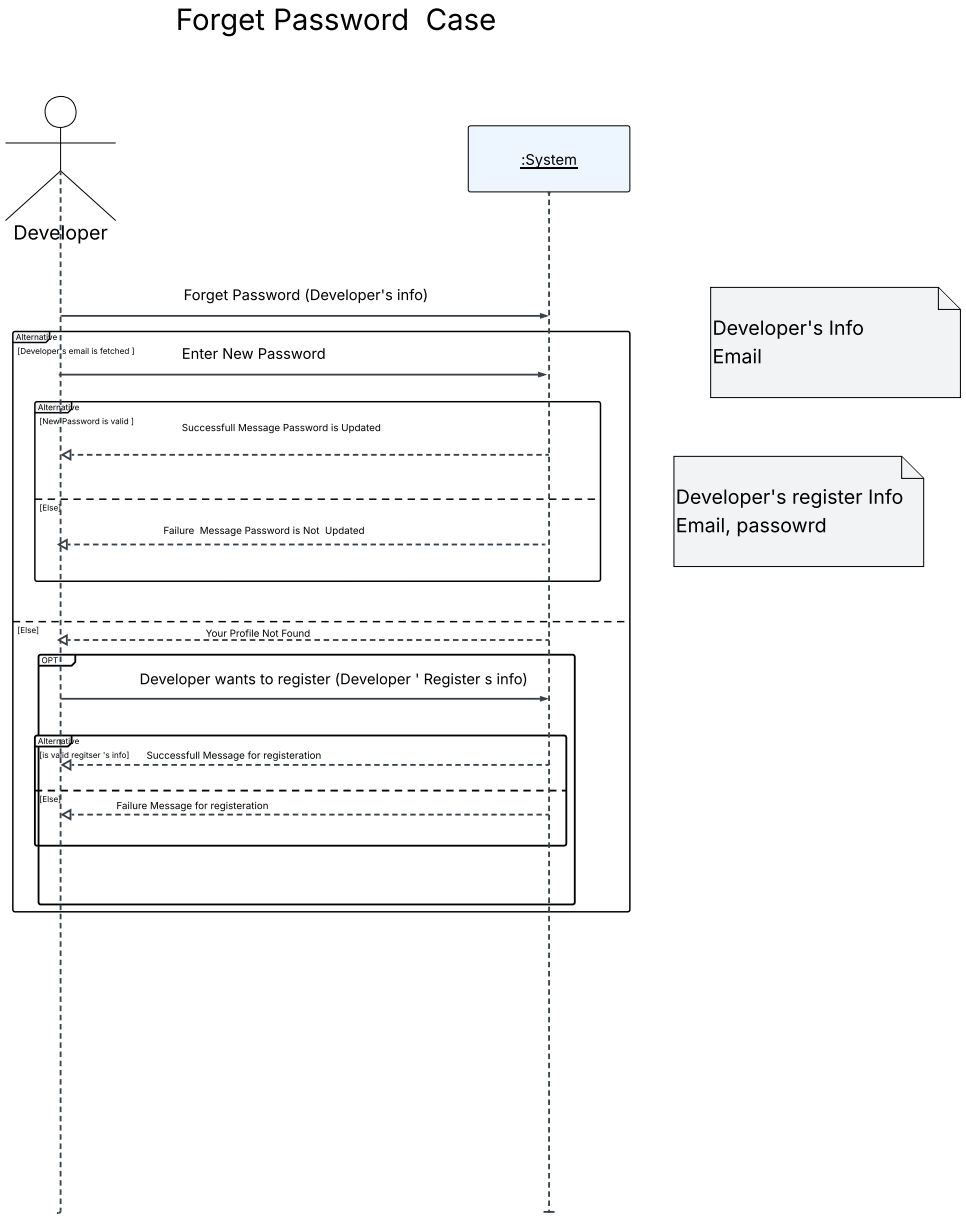


Figure 3.35: Forget password sequence diagram

Chapter 4:System Implementation

4.1 System Architecture

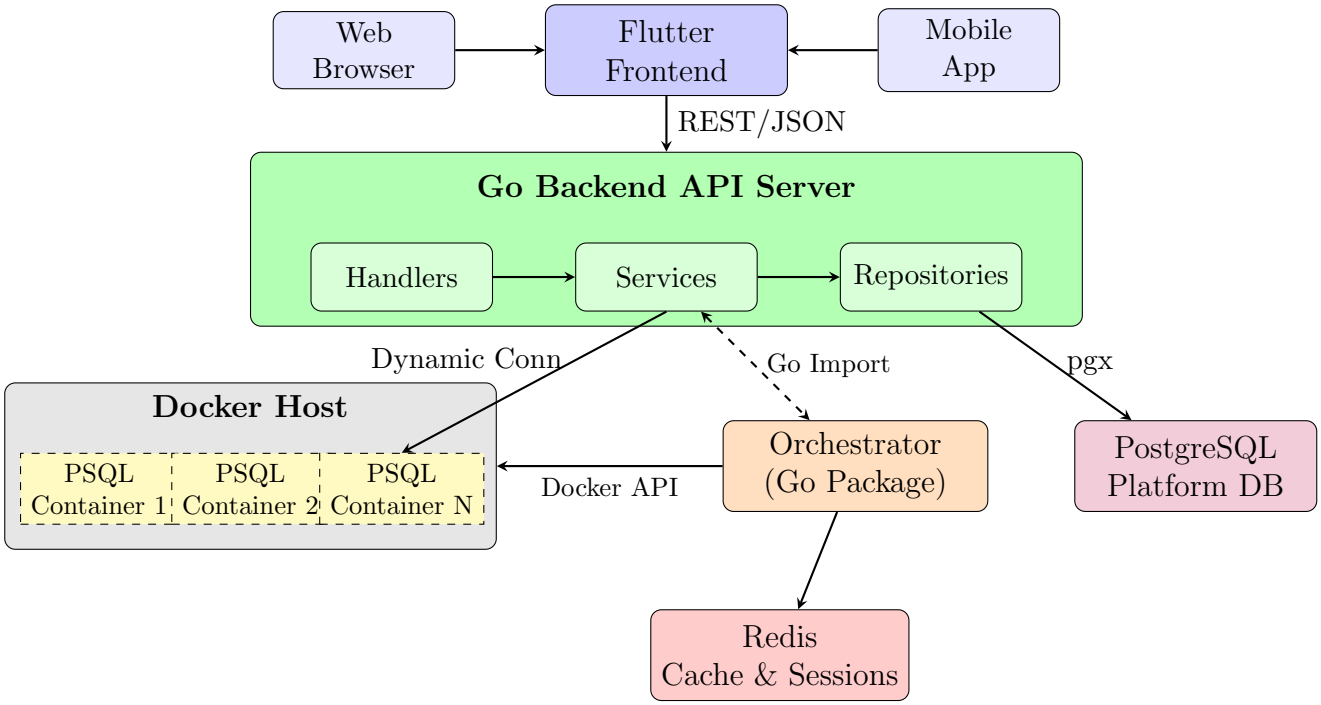


Figure 4.1: High-Level System Architecture

- **Service Decomposition**: Functionality is divided into loosely coupled services (backend API, orchestrator, database instances)
- **Single Responsibility**: Each component has a well-defined purpose (separation of concerns)
- **API-first Design**: Services communicate through well-defined interfaces (REST APIs)

4.2 Technologies Used

The DBaaS platform relies on a modern and scalable technology stack to support efficient database management. Although the system consists of multiple components developed by different team members, this subsection focuses on the technologies related to the frontend layer and its integration with backend and AI services.

Flutter

Flutter is a UI development framework created by Google that enables the development of cross-platform applications using a single codebase. It was selected for the DBaaS frontend due to several key advantages:

- **Cross-Platform Capability:** Enables building web-based applications without the need for separate platform-specific implementations.
- **Rapid Development Cycle:** The hot reload feature allows instant visualization of UI changes, accelerating development and debugging.
- **Flexible and Reusable Widgets:** Provides a wide range of customizable widgets for building dashboards, tables, and interactive forms.
- **Backend Communication:** Supports seamless interaction with RESTful and GraphQL APIs for executing database operations.
- **Responsive Design:** Ensures consistent user experience across different screen sizes and devices.

Backend Technologies

Golang for Backend Development

The selection of Go as the primary implementation language is justified by several theoretical and practical considerations:

1. **Concurrency Model**: Go's goroutines implement lightweight threads (green threads) multiplexed onto OS threads by the Go runtime. This M:N threading model provides efficient concurrent execution without the overhead of OS-level thread management.
2. **Memory Safety**: Go's garbage collector prevents common memory errors (dangling pointers, buffer overflows) while maintaining performance through generational and concurrent collection algorithms.
3. **Static Typing**: Compile-time type checking prevents entire classes of runtime errors, improving system reliability.
4. **Compiled Execution**: Ahead-of-time compilation to native machine code provides performance comparable to C/C++ while maintaining higher-level abstractions.

Containerization with Docker

Docker was selected over alternatives (LXC, rkt, Podman) based on:

- Industry standard with extensive tooling and ecosystem
- Well-documented API for programmatic container management
- Mature implementation of Linux namespaces and cgroups
- Wide platform support and community knowledge base

PostgreSQL as Database Engine

PostgreSQL was chosen for tenant databases due to:

- ACID compliance ensuring data consistency and durability
- MVCC (Multi-Version Concurrency Control) enabling high concurrency
- Rich SQL compliance and extensibility
- Small footprint in Alpine Linux containers (200MB)

Redis for State Persistence

Redis serves as the persistence layer for orchestrator metadata based on:

- In-memory architecture providing O(1) lookup complexity
- Persistence mechanisms (RDB/AOF) for durability
- Atomic operations preventing race conditions
- Simplicity compared to distributed databases

Cloud and DevOps

To ensure scalability, reliability, and efficient deployment, the system adopts modern cloud and DevOps technologies:

- **Docker:** Used for containerizing application components to ensure consistency across development and production environments.
- **AWS / GCP:** Cloud platforms used to securely host application services, user data, and database resources.
- **Git and GitHub:** Used for version control, collaboration, and team-based development.

4.3 UI in the Technology Stack

The frontend development of the DBaaS platform focused on creating an intuitive and responsive user interface. Key components, including dashboards, table editors, query editors, and other interactive elements, were designed to provide a consistent experience across multiple screen sizes and devices.

Collaboration between the frontend, backend, and AI teams facilitated the integration of APIs and RAG-powered features, ensuring smooth communication with both SQL and NoSQL database systems. This integrated approach allows users to perform database operations efficiently within a unified and cohesive platform.

4.3.1 Comparison: Flutter Web vs Other Web Technologies

To support the choice of Flutter for the frontend implementation, the team conducted a comparison between Flutter Web and other widely used web technologies, such as ReactJS, Angular, and VueJS. Table 4.1 summarizes the key differences across essential features for modern web applications.

Table 4.1: Comparison of Flutter Web with Other Web Technologies

Aspect	**Flutter Web**	**ReactJS**	**Angular**	**VueJS**
Single Codebase	✓	✗	✗	✗
Pixel-perfect UI	✓	✗	✗	✗
Hot Reload / Fast Development	✓	✓	✗	✓
Enterprise Ready	✗	✓	✓	✗
Lightweight	✗	✗	✗	✓
Large Community Support	✓	✓	✓	✓
Best for Animations	✓	✗	✗	✗
Easy to Learn	✗	✗	✗	✓

4.4 Completed and Under Development Features

This section highlights the core functional modules of the DBaaS platform that have been successfully implemented and verified.

4.4.1 Implemented Features

Authentication Interface

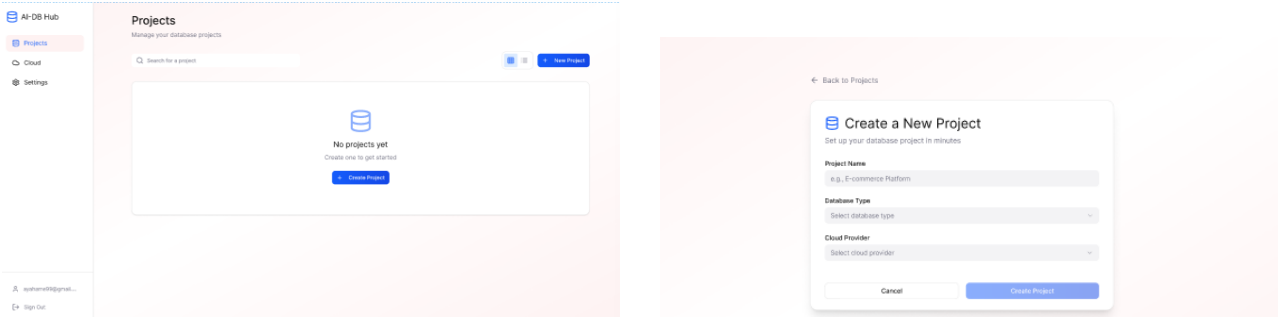
A secure entry point featuring dual-mode authentication (Sign In/Sign Up), email validation, and password recovery options.

(a) Login Screen (b) Sign Up Screen

Figure 4.2: Authentication screens of the DBaaS system.

Project Dashboard & Provisioning

The central hub for managing the database lifecycle, including a guided modal for provisioning new cloud instances.

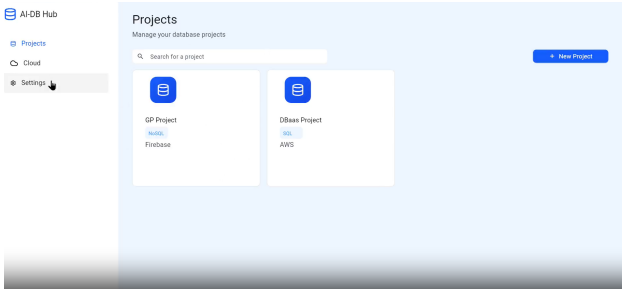


(a) Central Dashboard (b) Project Creation Modal

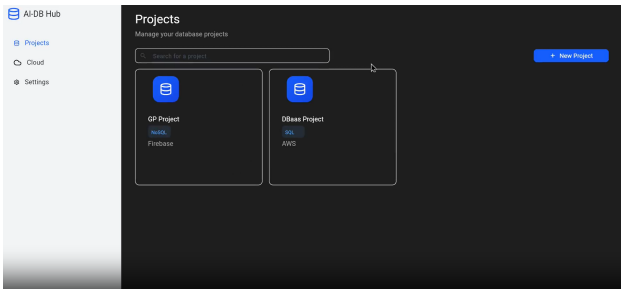
Figure 4.3: Project management and cloud provisioning interfaces.

Advanced Filtering & Theme Support

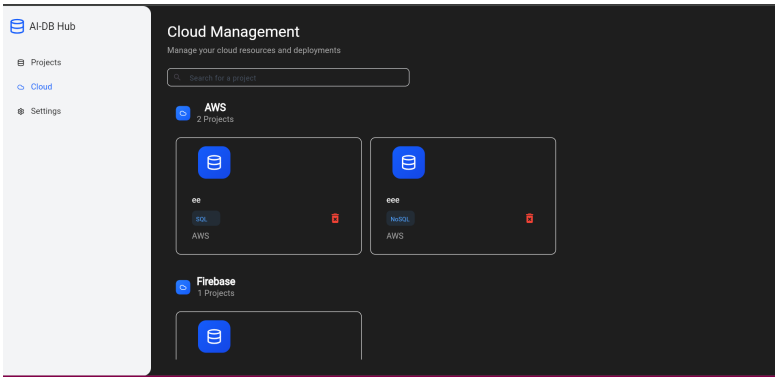
Supports dynamic resource tracking with provider-based filtering and optimization for both Light and Dark modes.



(a) Light Mode View



(b) Dark Mode View

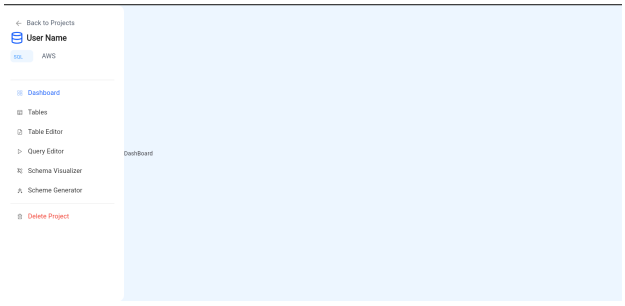


(c) Cloud Provider Filtering

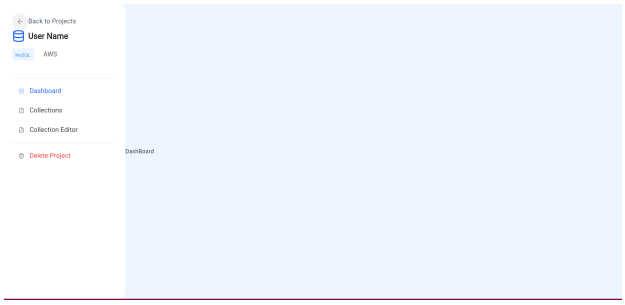
Figure 4.4: Project display with theme toggling and provider filtering.

Database Workspace Sidebars

Context-aware navigation for SQL and NoSQL environments, providing access to query editors and schema tools.



(a) SQL Sidebar



(b) NoSQL Sidebar

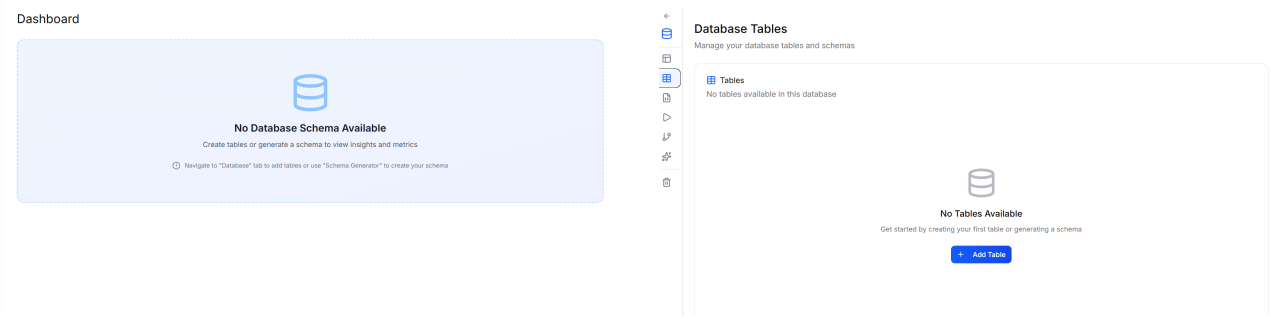
Figure 4.5: Database workspace sidebars for specialized project management.

4.4.2 SQL Project Management Tools

The platform provides a specialized suite of tools dedicated to **SQL-based projects**. These features are designed to handle relational structures, ensuring data integrity through schema enforcement and providing visual interfaces for complex relational operations.

Initial Setup and User Guidance

When a new SQL project is provisioned, the system provides intuitive empty states direct the user to begin by defining their schema or creating their first table.

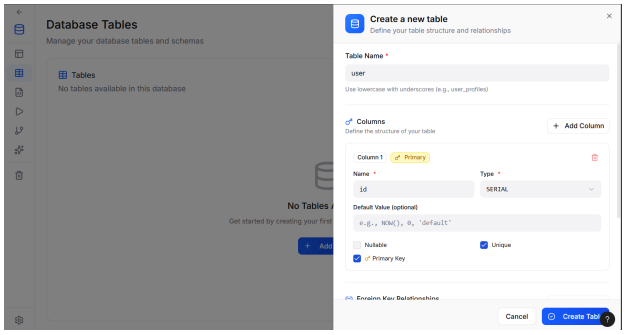


(a) Initial Dashboard State (b) Empty Tables View

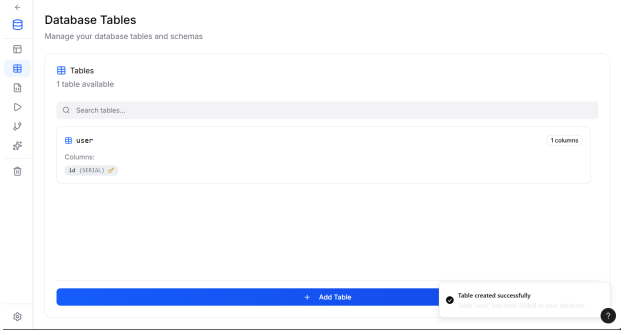
Figure 4.6: Guided onboarding for newly provisioned SQL databases.

Relational Table Management

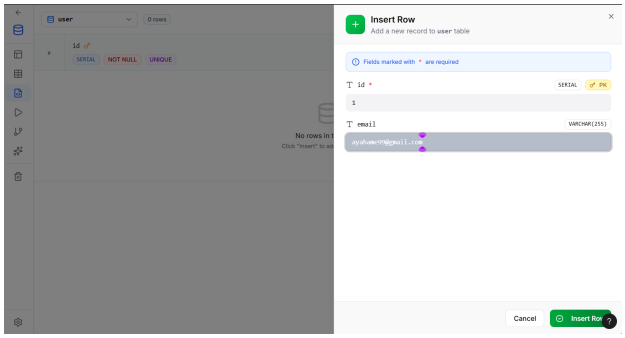
Users can define relational tables through a visual builder. This includes setting primary keys, auto-incrementing sequences (SERIAL), and managing data rows through an interactive grid.



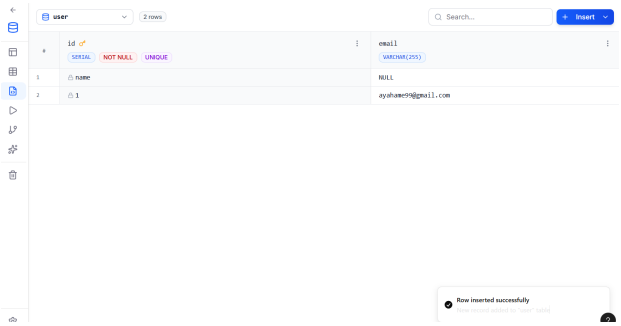
(a) Creating a new SQL Table



(b) Table Schema Overview



(c) Data Insertion Interface

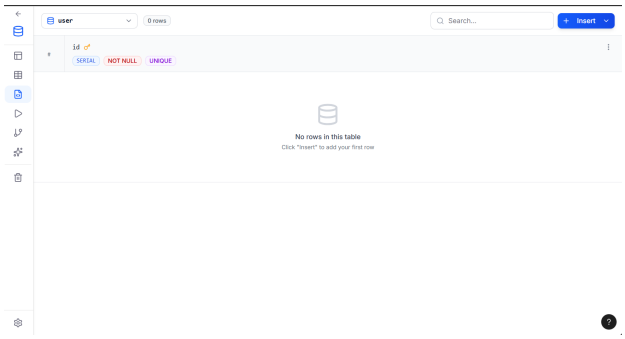


(d) Populated Data Grid

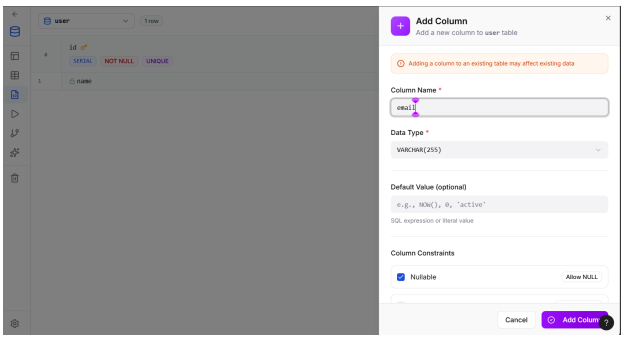
Figure 4.7: Visual workflow for managing SQL tables and records.

Schema Evolution and Column Modification

To support evolving requirements, the platform allows users to alter table structures dynamically, such as adding new columns with specific constraints and data types like VARCHAR.



(a) Structural Edit Mode

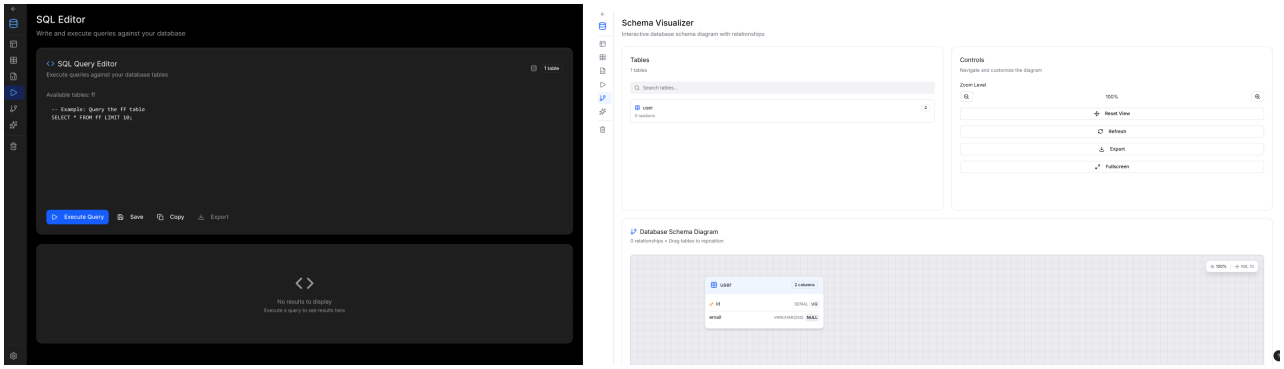


(b) Adding New Columns

Figure 4.8: Interfaces for modifying existing relational schemas.

SQL Workbench and Schema Visualization

For advanced users, a full SQL editor to execute different queries on created schemas.



(a) Dark-themed SQL Editor (b) Interactive Schema Diagram

Figure 4.9: Advanced querying and architectural visualization tools.

Comprehensive System Settings

Beyond database operations, the platform includes a full settings dashboard where users can manage their account, UI themes, and global system preferences.

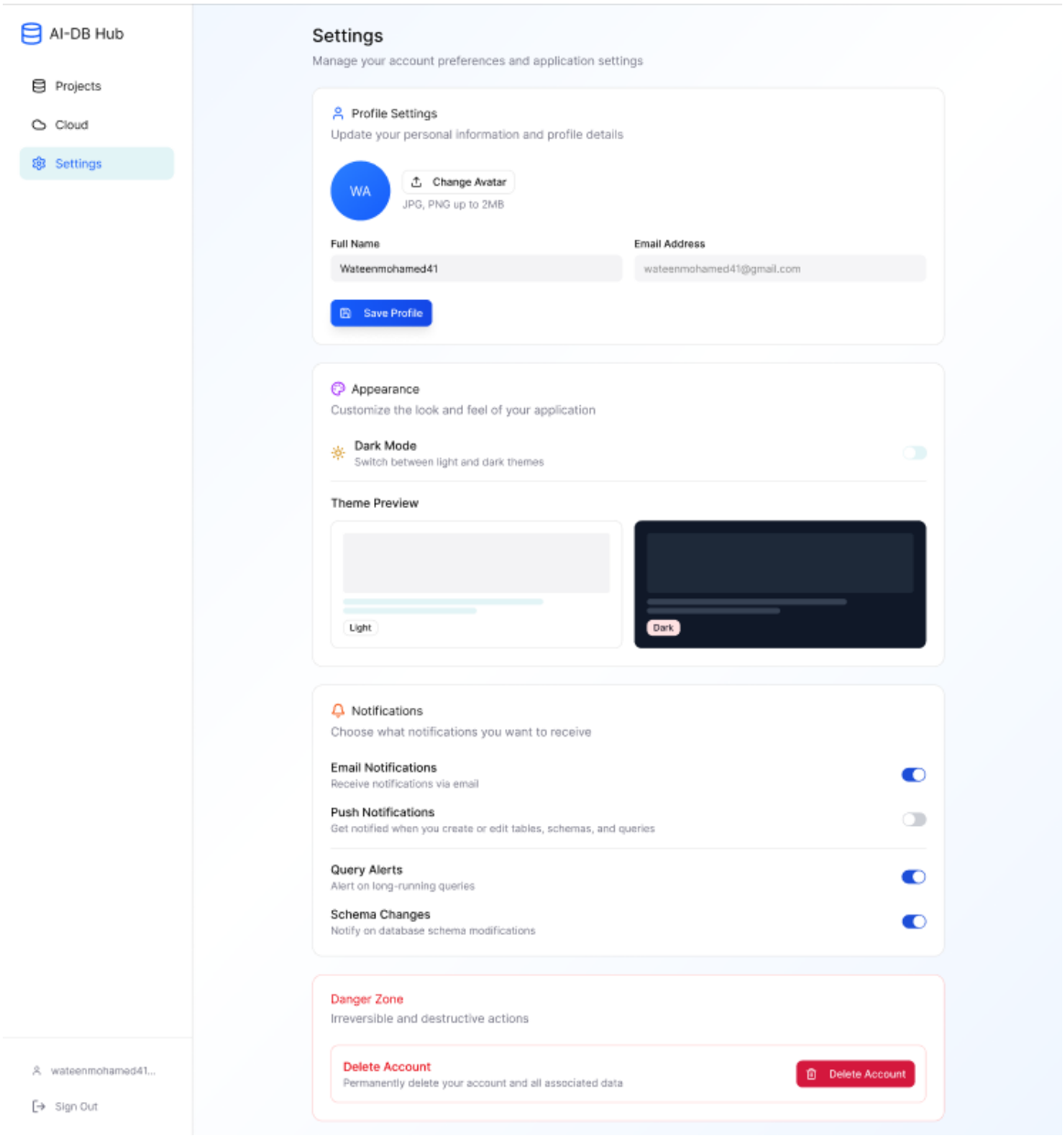


Figure 4.10: System-wide settings and personalization interface.

Bibliography

[1] Z. Łata and M. Skublewska-Paszkowska, "Performance analysis of databases created in virtualized and containerized environment," *Journal of Computer Sciences Institute*, vol. 28, pp. 264–272, 2023.

[2] I. Astrova, A. Koschel, C. Eickemeyer, and N. Offel, "Comparison of dbaas architectures," in *2018 9th International Conference on Information, Intelligence, Systems and Applications (IISA)*. IEEE, 2018, pp. 1–5.

[3] M. Husain, I. Hussain, S. Tanweer, I. R. Khan, and A. Khalique, "Analyzing the impact of performance metrics on database speedup in dbaas environment."

[4] A. C. König, Y. Shan, K. Newatia, L. Marshall, and V. Narasayya, "Solver-in-the-loop cluster resource management for database-as-a-service," *Proceedings of the VLDB Endowment*, vol. 16, no. 13, pp. 4254–4267, 2023.

[5] M. B. F. Sanjeetha, P. Sumathipala, M. S. M. Siriwardane, and R. J. P. K. Rajakaruna, "Database-as-a-service (dbaas) in cloud computing: Security issues and policies," *Asian Journal of Computer Science and Technology*, vol. 14, no. 1, pp. 15–19, 2025.

[6] W. Lehner and K.-U. Sattler, "Database as a service (dbaas)," in *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*. IEEE, 2010, pp. 1216–1217.

[7] M. Aniruddh, A. Dinkar, S. C. Mouli, B. Sahana, and A. A. Deshpande, "Comparison of containerization and virtualization in cloud architectures," in *2021 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*. IEEE, 2021, pp. 1–5.

[8] A. A. A. Mardan and K. Kono, "Containers or hypervisors: Which is better for database consolidation?" in *2016 IEEE international conference on cloud computing technology and science (CloudCom)*. IEEE, 2016, pp. 564–571.

[9] J. Watada, A. Roy, R. Kadikar, H. Pham, and B. Xu, "Emerging trends, techniques and open issues of containerization: A review," *IEEE Access*, vol. 7, pp. 152 443–152 472, 2019.

[10] V. Narasayya, S. Das, M. Syamala, B. Chandramouli, and S. Chaudhuri, "Sqlvm: Performance isolation in multi-tenant relational database-as-a-service," in *CIDR 2013*, 2013.

[11] N. Wiener, *Cybernetics; or, Control and Communication in the Animal and the Machine*. Cambridge, MA: MIT Press, 1948.

[12] J. Weizenbaum, "Eliza—a computer program for the study of natural language communication between man and machine," *Communications of the ACM*, vol. 9, no. 1, pp. 36–45, 1966. [Online]. Available: https://dl.acm.org/doi/10.1145/365153.365168

[13] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 1998.

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[16] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *NeurIPS*, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, U. Khandelwal, W.-t. Yih, T. Rocktäschel, S. Riedel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *NeurIPS*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[18] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," 2022/2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[20] S. Kapoor, B. Stroebl, and Z. S. Siegel, "Survey on large language model-based multi-agent systems," *arXiv preprint arXiv:2407.05012*, 2024. [Online]. Available: https://arxiv.org/abs/2407.05012

[21] R. Sapkota and K. I. Roumeliotis, "Formulating large language model agents as stochastic games," *arXiv preprint arXiv:2505.10468*, 2025. [Online]. Available: https://arxiv.org/abs/2505.10468

[22] Q. Wu, P. Liu, H. Zhang, and Z. Chen, "Hugginggpt: Solving ai tasks with chatgpt and hugging face models," *arXiv preprint arXiv:2303.17580*, 2023. [Online]. Available: https://arxiv.org/abs/2303.17580

[23] Y. Li, T. Sun, and M. Zhou, "Autogen: Multi-agent ai coordination via large language models," 2024. [Online]. Available: https://arxiv.org/abs/2408.11234

[24] Z. Z. Wang, Y. Shao, and G. Neubig, "Measuring emergent coordination and synergy in llm multi-agent systems," *arXiv preprint arXiv:2510.22780*, 2025. [Online]. Available: https://arxiv.org/abs/2510.22780

[25] M. R. A. Team, "Autogen: Enabling next-gen llm applications via multi-agent conversation framework," arXiv preprint arXiv:2308.08155, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[26] M. Research, "Autogen: An agentic open-source framework for intelligent automation," Microsoft Research publication page, 2024. [Online]. Available: https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/

[27] M. A. Team, "Autogen documentation and user guide," GitHub / AutoGen Docs, 2024. [Online]. Available: https://microsoft.github.io/autogen/

[28]

[29] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[30] X. Zheng, "Database as a service-current issues and its future," *arXiv preprint arXiv:1804.00465*, 2018.

[31] P. P.-S. Chen, "The entity-relationship model—toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[32] Y. Huhtala, J. Kärkkäinen, P. Porkka, and H. Toivonen, "Tane: An efficient algorithm for discovering functional and approximate dependencies," in *Proceedings of the International Conference on Data Engineering (ICDE)*, 1999, pp. 106–115.

[33] A. Kumar, Y. Bajpai, S. Gulwani, G. Soares, and E. Murphy-Hill, "Why ai agents still need you: Findings from developer-agent collaborations in the wild," 2025. [Online]. Available: https://arxiv.org/abs/2506.12347

[34] M. Robeer, G. Lucassen, J. M. E. van der Werf, F. Dalpiaz, and S. Brinkkemper, "Automated extraction of conceptual models from user stories via nlp," in *Proceedings of the International Conference on Requirements Engineering (selected workshop/venue versions)*, 2015, visual Narrator tool; heuristics for concept/relationship extraction. [Online]. Available: https://research-portal.uu.nl/en/publications/automated-extraction-of-conceptual-models-from-user-stories-via-nlp

[35] E. Navarro and P. Letelier, "From requirements to relational database schemas: A systematic approach," *Data & Knowledge Engineering*, vol. 68, no. 11, pp. 1162–1180, 2009.

[36] S. Ghosh, S. Rana, and D. Sarkar, "Automated extraction of conceptual models from natural language requirements," *Journal of Systems and Software*, vol. 117, pp. 433–451, 2016.

[37] M. Parciak, B. Vandevoort, F. Neven, L. M. Peeters, and S. Vansummeren, "Schema matching with large language models: An experimental study," *Proceedings of the VLDB Endowment / TaDA workshop (arXiv preprint)*, 2024, empirical analysis of LLMs for schema matching; useful for LLM→schema tasks. [Online]. Available: https://arxiv.org/abs/2407.11852

[38] F. Zhou *et al.*, "Large language models for structured data: A survey," *arXiv preprint arXiv:2402.11014*, 2024, survey summarizing LLM applications to structured-data tasks (schema/query/code generation).

[39] Q. Wu, P. Liu, H. Zhang, and Z. Chen, "Hugginggpt: Solving ai tasks with chatgpt and hugging face models," *arXiv preprint arXiv:2303.17580*, 2023.

[40] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan *et al.*, "React: Synergizing reasoning and acting in language models," in *arXiv preprint*, 2022/2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[41] Z. Abedjan, L. Golab, and F. Naumann, "Detecting data errors: Where are we and what needs to be done?" *Proceedings of the VLDB Endowment*, vol. 9, no. 12, pp. 993–1004, 2016.

[42] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," in *Proceedings of EMNLP*, 2018. [Online]. Available: https://yale-lily.github.io/spider

[43] P. A. Bernstein, "Synthesizing third normal form relations from functional dependencies," *ACM Transactions on Database Systems*, vol. 1, no. 4, pp. 277–298, 1976.

[44] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, and Z. Li, "Mac-sql," 2025. [Online]. Available: https://arxiv.org/abs/2312.11242

[45] X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Zhang, J. Fan, G. Li, N. Tang, and Y. Luo, "A survey of text-to-sql in the era of llms: Where are we, and where are we going?" 2025. [Online]. Available: https://arxiv.org/abs/2408.05109

[46] J. Cen, J. Liu, Z. Li, and J. Wang, "Sqlfixagent," 2025. [Online]. Available: https://arxiv.org/abs/2406.13408

[47] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 28 691–28 735.

[48] S. Ojuri, T. A. Han, R. Chiong, and A. Di Stefano, "Optimizing text-to-sql conversion techniques through the integration of intelligent agents and large language models," *Information Processing & Management*, vol. 62, no. 5, p. 104136, 2025.

[49] R. G. Aparicio and I. C. Coz, "Database on demand: insight how to build your own dbaas," in *Journal of Physics: Conference Series*, vol. 664, no. 4. IOP Publishing, 2015, p. 042021.

[50] N. Krishnan, "Ai agents: Evolution, architecture, and real-world applications," *arXiv preprint arXiv:2503.12687*, 2025.

[51] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "Ai agents that matter," *arXiv preprint arXiv:2407.01502*, 2024.

[52] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges," *arXiv preprint arXiv:2505.10468*, 2025.

[53] N. Kolt, "Governing ai agents," *arXiv preprint arXiv:2501.07913*, 2025.

[54] Z. Z. Wang, Y. Shao, O. Shaikh, D. Fried, G. Neubig, and D. Yang, "How do ai agents do human work? comparing ai and human workflows across diverse occupations," *arXiv preprint arXiv:2510.22780*, 2025.

[55] T. Schick, J. Dwivedi-Yu, R. Dess
i, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, pp. 68 539–68 551, 2023.

[56] K. Awala, "Comparing the performance of flutter and reactjs for web development," *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, vol. 3, no. 2, 2023, pDF available online. [Online]. Available: https://ijarsct.co.in/Paper8892.pdf

[57] "Flutter a boom to web development technology," *International Journal of Research Publication and Reviews (IJRPR)*, vol. 5, no. 4, 2024, pDF available online. [Online]. Available: https://ijrpr.com/uploads/V5ISSUE4/IJRPR24974.pdf

[58] N. Krishnan, "Coordination mechanisms in multi-agent systems powered by large language models," *arXiv preprint arXiv:2503.12687*, 2025. [Online]. Available: https://arxiv.org/abs/2503.12687

[59] B. Nuseibeh and S. Easterbrook, "Requirements engineering: a roadmap," *Proceedings of the Conference on The Future of Software Engineering*, pp. 35–46, 2000.